

Prelude Teacher's Edition

A Complete Course in Functional Python

sixteen chapters, 193 homework problems, 74 applied labs

J. L. Clements III

September 17, 2026

How to Take This Course

This book teaches `prelude.py` – a Haskell-style prelude in pure Python, no imports anywhere – until it is not reference material but reflex. The order is the mastery order, not the file order: the spine is **fold** → **scan** → **unfold** → **dict fold** → **memo** – one concept in five costumes – and every chapter hangs off it.

Each chapter has three movements. **Concepts:** every function introduced with its purpose, worked examples (all of them runnable doctests), deep dives where the idea is subtle, and a “roll your own” walkthrough of the implementation itself. **Homework:** twelve problems – five drills, five applied scenarios, two interview-grade challenges. **Discipline:** solve in a scratch file with the printed doctests pasted in; `python3 -m doctest` is the referee, and an answer is not done until it is N/N.

Part III then applies the whole toolkit to seventy-four classic problems organised by class – the HackerRank/LeetCode canon reworked in prelude style. This Teacher’s Edition prints every solution and a teaching note inline; grade against the doctests, then against the style: two or three prelude names and one lambda beat a page of loops.

The recognition ladder

Before any problem, name the INPUT shape, the OUTPUT shape, and the quantity word – then walk this ladder; the first row whose trigger words match names your tools.

Shape	Trigger words	Reach for
SORT FIRST	“sorted”, “k-th”, “closest pair”, “intervals”	<code>sorted</code> , <code>sortOn</code> , <code>bisect_left</code> , <code>merge</code>
FOLD	“total”, “count”, “largest”, “best single ...”	<code>foldl</code> , <code>minOn</code> / <code>maxOn</code>
SCAN	“running”, “so far”, “at each step”, “prefix sums”	<code>scanl</code> , <code>scanl1</code>
DICT FOLD	“group by”, “frequency”, “most common”	<code>fromListWith</code> , <code>Counter</code> , <code>unionWith</code>
WINDOW	“substring”, “contiguous”, “longest run”	<code>longest_window</code> , <code>pairwise</code>
STACK	“nested”, “matching pairs”, “next greater”	<code>list</code> as <code>stack</code> ; <code>foldl</code> with <code>stack acc</code>
UNFOLD	“digits of”, “split into pieces”, “repeat until”	<code>unfoldr</code> , <code>iterate + take</code>
ENUMERATE	“all subsets”, “every way to pick”	<code>subsequences</code> , <code>replicateM</code> , <code>cross</code>
TREE / PATHS	“nested”, “prefix” (trie), “directories”	<code>ITree</code> , <code>paths</code> , <code>setpath</code> , <code>tfold</code>
GRAPH	“prerequisites”, “network”, “shortest route”	<code>Tree(1,list)</code> <code>adjacency</code> , <code>deque BFS</code> , <code>heap</code> , <code>until</code>
MEMOIZE	“fewest”, “max value”, “count the ways”	<code>memo over indices</code>

Contents

How to Take This Course	iii
I The Sixteen Chapters	1
1 Grammar, Pairs and Maybe	3
1.1 Concepts	3
1.2 Homework	6
2 Arithmetic and Logic	11
2.1 Concepts	11
2.2 Homework	15
3 Lists I: The Basics	21
3.1 Concepts	21
3.2 Homework	26
4 Lists II: Higher-Order	31
4.1 Concepts	31
4.2 Homework	34
5 The Fold	39
5.1 Concepts	39
5.2 Homework	42
6 Scans	47
6.1 Concepts	47
6.2 Homework	49
7 Streams and Unfolds	55
7.1 Concepts	55
7.2 Homework	57
8 Spans and Views	63
8.1 Concepts	63
8.2 Homework	66
9 Order and Text	71
9.1 Concepts	71
9.2 Homework	74
10 Dict Folds and Bags	79

10.1	Concepts	79
10.2	Homework	83
11	Trees and Tries	89
11.1	Concepts	89
11.2	Homework	91
12	Machines: Deque, DSU, Heap	97
12.1	Concepts	97
12.2	Homework	100
13	Nodes: Linked Lists and Binary Trees	107
13.1	Concepts	107
13.2	Homework	111
14	Grids and Windows	117
14.1	Concepts	117
14.2	Homework	118
15	Control and Dynamic Programming	125
15.1	Concepts	125
15.2	Homework	126
16	Combinators Capstone	133
16.1	Concepts	133
16.2	Homework	136
II	Applied Labs: The Nine Problem Classes	141
	About the labs	143
17	Arith And Unfolds	145
17.1	sign: Sign of a number	145
17.2	rect_area: Rectangle area	145
17.3	reverse_digits: Reverse an integer's digits	146
17.4	to_base: Render n in base b	146
17.5	to_roman: Integer to Roman numeral, $1 \leq n \leq 3999$	146
17.6	from_roman: Parse a valid Roman numeral	147
18	Lists And Strings	149
18.1	greet: Greeting string	149
18.2	count_vowels: Count vowels, case-insensitive	149
18.3	fizzbuzz: FizzBuzz as data	149
18.4	word_lengths: Map each word to its length	150
18.5	flatten: Flatten one level of nesting	151
18.6	caesar: Caesar cipher, letters only, case preserved	151
18.7	contains_duplicate: NeetCode: Contains Duplicate	151
18.8	chunks: Split a list into pieces of length k	152
18.9	permutations: All distinct orderings, sorted	152
18.10	reverse_words: Reverse the word order	153
18.11	transpose_matrix: Flip rows and columns	153

18.12	rotate_right: Rotate a list right by k	153
19	Folds Scans Streams	155
19.1	two_sum: NeetCode: Two Sum	155
19.2	best_time_stock: NeetCode: Best Time to Buy and Sell Stock	155
19.3	maximum_subarray: NeetCode: Maximum Subarray (Kadane)	156
19.4	product_except_self: NeetCode: Product of Array Except Self	156
19.5	moving_average: Average of every k-window	156
19.6	pascal: First n rows of Pascal's triangle	157
19.7	climbing_stairs: NeetCode: Climbing Stairs	157
19.8	primes_up_to: All primes $\leq n$, ascending	158
19.9	totals_and_deltas: A scan and its inverse	158
19.10	paren_depth: Deepest paren nesting	159
20	Sorting And Searching	161
20.1	is_anagram_sorted: Anagrams via canonical form	161
20.2	second_largest: Second largest distinct value	161
20.3	binary_search: Index of target in a sorted list, else -1	162
20.4	merge_intervals: Merge overlapping (and touching) intervals	162
20.5	photo_lineup: The reported Anduril prompt, prelude-sized	163
20.6	merge_sort: A real $O(n \log n)$ sort, by hand	163
20.7	lis: Longest strictly increasing subsequence (length)	164
20.8	russian_doll: LC 354: Russian Doll Envelopes	164
20.9	pairs_within_budget: Count pairs whose sum fits a budget	165
20.10	sensor_pairing: The screen's domain problem, function form	165
21	Bags And Grouping	167
21.1	valid_anagram: NeetCode: Valid Anagram	167
21.2	ransom_note: HackerRank: Ransom Note	167
21.3	most_frequent: Most frequent value; ties $\rightarrow$ smallest	168
21.4	top_k_words: The k most frequent words	168
21.5	group_words: Group words by first letter	168
21.6	group_anagrams: NeetCode: Group Anagrams	169
21.7	merge_sum: Merge two dicts of numbers, summing shared keys	169
21.8	rle_encode: Run-length encode	170
21.9	lru_class: An LRU cache class, dict-only	170
21.10	first_unique: First non-repeating character	171
21.11	log_report: Error report from messy logs	171
21.12	invert_dict: Swap a dict's keys and values	172
22	Tries And Paths	173
22.1	deepest_directory: Challenge: deepest path in a directory forest	173
22.2	autocomplete: Challenge: prefix trie with completion	173
22.3	subset_sum_paths: Challenge: all paths in the decision tree summing to a total	174
22.4	subset_sum_trie: Challenge: subset-sum, winners recorded in a solution trie	175
23	Graphs Grids Heaps	177
23.1	longest_unique: Longest substring without repeats	177
23.2	grid_path: Shortest path through a grid	177
23.3	rate_limiter: Sliding-window rate limiter	178
23.4	running_median: Median after each arrival	179

23.5	dijkstra: Challenge: shortest paths from weighted (from, to, dist) edges	179
23.6	dag_longest_path: Challenge: longest path in a weighted DAG	180
23.7	bellman_ford: Challenge: shortest paths as a fixed point (negative weights ok)	181
23.8	course_schedule: NeetCode: Course Schedule (cycle detection via Kahn)	182
24	Dp And Control	185
24.1	coin_change: NeetCode: Coin Change	185
24.2	lcs: NeetCode: Longest Common Subsequence	185
24.3	edit_distance: Levenshtein distance	186
24.4	knapsack: 0/1 knapsack	187
24.5	calculator: Integer expression evaluator (capstone)	187
24.6	max_height_cuboids: LC 1691: Maximum Height by Stacking Cuboids	188
24.7	guillotine_cut: Max value cut from a sheet, guillotine cuts only	189
24.8	partition_k_subsets: LC 698: Partition to K Equal Sum Subsets	190
24.9	min_path_sum: Cheapest right/down path through a grid	190
25	Stack Folds	193
25.1	balanced_brackets: Matching pairs: valid bracket nesting	193
25.2	eval_rpn: Evaluate Reverse Polish notation	193
25.3	next_greater: First later element that's larger	194
III	Appendices	195
A	prelude.py, Complete	197

Part I

The Sixteen Chapters

Chapter 1

Grammar, Pairs and Maybe

Every course opens with its grammar. Here it is the conventions the whole prelude leans on: functions come first and data second in every call; `None` is `Nothing` – the universal “no answer” – and `fromMaybe` is how a pipeline lands a default, and `isJust/isNothing` make the `None` test itself a predicate you can hand to other tools; `NOTHING` is the other absence, the “no argument supplied” sentinel. The pair accessors look trivial, and are: that is what makes them readable inside sort keys and maps where an index would be noise.

1.1 Concepts

1.1.1 `fst`

the first element of a pair. Reads better than `p[0]` in code that is all tuples, and slots into higher-order positions.

```
>>> fst((1, 2))
1
>>> map_(fst, [(1, 'a'), (2, 'b')])
[1, 2]
```

Roll your own — the definition:

```
fst = lambda p: p[0] # first element of a pair
```

Used before it is rolled: `map_` in §4.1.1 (p. 31).

1.1.2 `snd`

the second element of a pair. The classic sort key for (key, count) items.

```
>>> snd((1, 2))
2
>>> sortOn(snd, [('a', 3), ('b', 1)])
[('b', 1), ('a', 3)]
```

Roll your own — the definition:

```
snd = lambda p: p[1] # second element of a pair
```

Used before it is rolled: `sortOn` in §9.1.1 (p. 71).

1.1.3 `swap`

exchanges a pair’s elements. One honest use: inverting a dict by swapping its items.

```
>>> swap((1, 2))
(2, 1)
>>> dict(map_(swap, [('a', 1), ('b', 2)]))
{'1': 'a', '2': 'b'}
```

Roll your own — the definition:

```
| swap = compose(tuple, reversed) # Data.Tuple
```

Used before it is rolled: *map_* in §4.1.1 (p. 31), *compose* in §16.1.8 (p. 136).

1.1.4 NOTHING

a private sentinel object meaning "no argument was supplied". Why not `None`? Because `None` is a legitimate VALUE a caller might pass; `NOTHING` can only mean "omitted", since nobody else has a reference to it. Always test with `'is'`. This is how `foldl` and `accumulate` tell an absent initial value from a real one.

```
>>> x = NOTHING
>>> x is NOTHING
True
>>> def greet(name=NOTHING):
...     return "hello, stranger" if name is NOTHING else "hello, " + name
>>> greet()
'hello, stranger'
>>> greet("ada")
'hello, ada'
```

How it works: `object()` creates a fresh, featureless object whose only property is its IDENTITY – nothing else in the program can ever be it. Testing `x is NOTHING` therefore asks exactly one question: "was I handed this precise sentinel?", which makes it a safe flag for "no argument was supplied".

Why not `None`, demonstrated: `None` is data (from `Maybe`'s whole job is handling it), so using it to also mean "argument omitted" would blur two different situations. The sentinel keeps them apart: `None` is `Nothing` in the DATA; `NOTHING` is `Nothing` in the ARGUMENT LIST.

```
>>> def describe(x=NOTHING):
...     if x is NOTHING:
...         return "no argument"
...     return "got " + repr(x)
>>> describe()
'no argument'
>>> describe(None)
'got None'
```

Where the prelude leans on it: `foldl` and `accumulate` take an optional seed. With `NOTHING` as the default they can tell "seed omitted – use the first element" from "the seed IS `None`", which a `None` default could not.

Remember it as: `None` is a value; `NOTHING` is the absence of one.

Roll your own — the definition:

```
| NOTHING = object() # Maybe's Nothing: "no arg given"; test with `is`
# hoisted ahead of foldl, which needs it as a sentinel
```

Used before it is rolled: *foldl* in §5.1.1 (p. 39).

1.1.5 fromMaybe

lands a default: `fromMaybe(d, x)` is `x` unless `x` is `None`, in which case it is `d`. The standard way to finish a pipeline built on the `None-is-Nothing` convention (`find`, `lookup`, `getPath`, `stripPrefix` all return `None` on failure).

```
>>> fromMaybe(0, None), fromMaybe(0, 5)
(0, 5)
>>> fromMaybe('_', find(lambda c: c == 'q', "abc"))
'_'
>>> fromMaybe(99, lookup('b', [('a', 1), ('b', 2)]))
2
```

Start from a problem you already have: `find` looks for something that might not be there, and when it is not, you are left holding `None` – which you cannot print to a user, add, or put in a report. So you write a backup plan:

```
>>> result = find(even, [3, 5, 7])
```

```
>>> if result is None:
...     result = 0
>>> result
0
```

fromMaybe is those three lines as one word. Argument order: the backup plan first, then the risky thing – said aloud it parses: "use 0 if this comes back empty".

```
>>> fromMaybe(0, find(even, [3, 5, 7]))
0
>>> fromMaybe(0, find(even, [3, 5, 8]))
8
```

Versus x or d: Python's or falls back on ANYTHING falsy – 0, "", [], False – not just None, so it silently destroys legitimate zero-ish answers. fromMaybe asks the only correct question: is this the absence marker?

```
>>> fromMaybe(99, 0), (0 or 99)
(0, 99)
```

Remember it as: ifNoneUse – the three-line backup plan as one word; or-with-a-conscience.

Roll your own — the definition:

```
fromMaybe = lambda d, x: d if x is None else x # Data.Maybe: default for every None-returning tool
```

Used before it is rolled: *even* in §2.1.3 (p. 11), *lookup* in §3.1.12 (p. 23), *find* in §4.1.3 (p. 31).

1.1.6 isJust

the None test as a predicate you can hand to other tools. The lambda it replaces, 'lambda v: v is not None', kept reappearing wherever a Maybe-producing step feeds a predicate slot; naming it also locks in the discipline: identity against None, never truthiness, so 0 and "" count as present.

```
>>> isJust(0), isJust(''), isJust(None)
(True, True, False)
>>> find(isJust, [None, None, 7, None])
7
>>> all(map(isJust, [1, 0, 'x']))
True
```

Roll your own — the definition:

```
isJust = lambda x: x is not None # Data.Maybe: None test as a handable predicate
```

Used before it is rolled: *find* in §4.1.3 (p. 31).

1.1.7 isNothing

the mirror: True only for None. Counting failures reads as one word.

```
>>> isNothing(None), isNothing(0)
(True, False)
>>> len(filter_(isNothing, [None, 3, None]))
2
```

Together they finish the Maybe family: fromMaybe LANDS an optional value, mapMaybe/catMaybes FILTER optional values, isJust/isNothing TEST one. Haskell's Data.Maybe has all five under the same names.

```
>>> readings = [(1, None), (2, 7.5), (3, None)]
>>> fromMaybe(-1, find(isJust, map(snd, readings)))
7.5
```

Roll your own — the definition:

```
isNothing = lambda x: x is None # Data.Maybe: true when x is Nothing (None)
```

Used before it is rolled: *filter_* in §4.1.2 (p. 31), *find* in §4.1.3 (p. 31).

1.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

1.1 Mirror a pair

drill

Write `mirror(p)`: the pair with its elements exchanged. Applying it twice must give the original pair back.

`mirror(p) -> pair`

```
>>> mirror((1, 2))
(2, 1)
>>> mirror(('lo', 'hi'))
('hi', 'lo')
>>> mirror(mirror((3, 4)))
(3, 4)
```

Solution.

```
| mirror = swap
```

Note. Recognize-it drill: mirror IS swap. Involution property (twice = identity) is worth saying.

1.2 Record projections

drill

A leaderboard record is a (name, score) pair. Write `name_of(r)` and `score_of(r)` using the pair accessors, not indexing.

`name_of(r) -> value; score_of(r) -> value`

```
>>> r = ('ada', 95)
>>> name_of(r)
'ada'
>>> score_of(r)
95
>>> name_of(swap(r))
95
```

Solution.

```
| name_of = fst
| score_of = snd
```

Note. `fst/snd` read better than `[0]/[1]` in pair-heavy code and slot into higher-order positions.

1.3 Default when missing

drill

Write `or_default(d, x)`: `x` itself unless `x` is `None`, in which case `d`. Falsy values like `0` and `''` are real data and must survive.

`or_default(d, x) -> value`

```
>>> or_default(0, None)
0
>>> or_default(0, 5)
5
>>> or_default(9, 0)
0
>>> or_default('-', None)
'-'
```

Solution.

```
| or_default = fromMaybe
```

Note. `fromMaybe` verbatim. The teaching point is the third test: only `None` triggers the default, unlike `'x` or `d`' which would destroy the `0`.

1.4 The sentinel argument

drill

Write `greet(name=NOTHING)` returning `'hello, stranger'` when NO argument was passed, and `'hello, ' + str(name)` otherwise – including when the caller passes `None` on purpose.

`greet(name=NOTHING) -> str`

```
>>> greet()
'hello, stranger'
>>> greet('ada')
'hello, ada'
>>> greet(None)
'hello, None'
```

Solution.

```
def greet(name=NOTHING):
    return 'hello, stranger' if name is NOTHING else 'hello, ' + str(name)
```

Note. `NOTHING` distinguishes omitted from `None`-as-data; test with `'is'`. A `None` default could not pass the third doctest.

1.5 Manufactured defaults

drill

Write `make_default(v)`: a function that returns a one-argument function answering `v` no matter what it is asked.

`make_default(v) -> function`

```
>>> make_default(7)('anything')
7
>>> f = make_default(0)
>>> f(99)
0
>>> f('x')
0
```

Solution.

```
make_default = const
```

Note. `const` verbatim – the constant function factory that fills callback slots.

Rolled later: `const` in §16.1.2 (p. 133).

1.6 Leaderboard by name

apply

The scoring service emits `(score, name)` pairs, but the display wants `(name, score)`. Write `by_name(pairs)` converting the whole list, order preserved.

`by_name(pairs) -> list of pairs`

```
>>> by_name([(95, 'ada'), (87, 'bob')])
[('ada', 95), ('bob', 87)]
>>> by_name([])
[]
>>> by_name([(1, 'solo')])
[('solo', 1)]
```

Solution.

```
by_name = lambda ps: [swap(p) for p in ps]
```

Note. `swap` mapped over the list. $O(n)$; the common slip is reversing the list instead of each pair.

1.7 Config with fallbacks

apply

Settings live in a dict; an absent key means 'use the default'. Write `setting(cfg, key, default)`. A stored falsy value (`False`, `0`, `''`) is a deliberate choice and must be returned as-is.

`setting(cfg, key, default) -> value`

```
>>> setting({'retries': 3}, 'retries', 1)
3
>>> setting({}, 'retries', 1)
1
>>> setting({'debug': False}, 'debug', True)
False
```

Solution.

```
| setting = lambda cfg, key, default: fromMaybe(default, cfg.get(key))
```

Note. dict.get is already a None-returning lookup, so fromMaybe finishes it. 'cfg.get(key) or default' would fail the False test.

1.8 Invert the phone book

apply

A phone book maps names to numbers, and numbers are unique. Produce the reverse index mapping each number back to its name.

```
invert_book(d) -> dict
```

```
>>> invert_book({'ada': '555-01', 'bob': '555-02'})
{'555-01': 'ada', '555-02': 'bob'}
>>> invert_book({})
{}
>>> invert_book({'x': 1})
{1: 'x'}
```

Solution.

```
| invert_book = lambda d: dict(swap(kv) for kv in d.items())
```

Note. A dict is a list of pairs wearing a hat: items, swap each, dict. Uniqueness of values is what makes the inversion lossless.

1.9 First good sensor fix

apply

A sensor log is a list of (timestamp, reading) pairs where a failed sample records None. Return the first non-None reading, or the supplied default when every sample failed (or the log is empty). A reading of 0 is a real fix.

```
first_fix(default, log) -> value
```

```
>>> first_fix(-1, [(1, None), (2, 7.5), (3, 8.0)])
7.5
>>> first_fix(-1, [(1, None), (2, None)])
-1
>>> first_fix(-1, [])
-1
>>> first_fix(-1, [(1, 0)])
0
```

Solution.

```
| first_fix = lambda default, log: fromMaybe(
|     default, find(isJust, map(snd, log)))
```

Note. Reshape then search: bare map keeps the scan lazy, find(isJust, ...) stops at the first real reading, fromMaybe lands the caller's default. The zero test is the None-vs-falsy discipline again – isJust asks identity against None, never truthiness.

Rolled later: find in §4.1.3 (p. 31).

1.10 Effective badge ids

apply

Each visitor holds (badge_or_None, temp_id). The effective id is the badge when one was issued, else the temp id. Map a whole visitor list to effective ids; an empty-string badge is a real (blank) badge, not a missing one.

```
effective_ids(pairs) -> list
```

```
>>> effective_ids([(None, 'T1'), ('B2', 'T2'), (None, 'T3')])
['T1', 'B2', 'T3']
```

```
>>> effective_ids([])
[]
>>> effective_ids(['', 'T9'])
['']
```

Solution.

```
effective_ids = lambda ps: [fromMaybe(snd(p), fst(p)) for p in ps]
```

Note. fromMaybe per element with the pair accessors supplying both sides. 'badge or temp' would fail the blank-badge test.

1.11 Ledger reconciliation

challenge

Ledger rows are (txn_id, debit_or_None, credit_or_None); a None side simply was not recorded and counts as 0 toward the balance. Return the pair (net, incomplete): net is total credits minus total debits; incomplete counts rows where EITHER side is missing.

```
reconcile(rows) -> (net, incomplete)
>>> reconcile([('a', 5, None), ('b', None, 10), ('c', 2, 3)])
(6, 2)
>>> reconcile([])
(0, 0)
>>> reconcile([('x', 1, 4)])
(3, 0)
>>> reconcile([('y', None, None)])
(0, 1)
```

Solution.

```
def reconcile(rows):
    net = sum(fromMaybe(0, r[2]) - fromMaybe(0, r[1]) for r in rows)
    incomplete = sum(1 for r in rows if r[1] is None or r[2] is None)
    return (net, incomplete)
```

Note. Two passes, each a one-liner: fromMaybe turns absence into the additive identity; the incomplete count keeps the audit trail honest. $O(n)$.

1.12 Three-level settings cascade

challenge

Configuration resolves through a cascade: an environment dict overrides a config-file dict, which overrides a built-in default. Absence at a level means None from .get; stored falsy values win like any other value. Write resolve(key, env, cfg, default).

resolve(key, env, cfg, default) -> value

```
>>> resolve('p', {'p': 1}, {'p': 2}, 9)
1
>>> resolve('p', {}, {'p': 2}, 9)
2
>>> resolve('p', {}, {}, 9)
9
>>> resolve('debug', {}, {'debug': 0}, 1)
0
```

Solution.

```
resolve = lambda key, env, cfg, default: fromMaybe(
    fromMaybe(default, cfg.get(key)), env.get(key))
```

Note. fromMaybe NESTS: the inner call resolves cfg-vs-default, and that result becomes the outer default for env. Reading it inside-out is the exercise.

Chapter 2

Arithmetic and Logic

Small tools with sharp edges. The floor-versus-truncate split (`div/mod` against `quot/rem`) decides real interview questions the moment a negative number appears; `signum` compresses three-way comparison into arithmetic; `isqrt` shows Newton's method surviving entirely inside the integers. Master the edges here and modular-arithmetic problems stop being scary.

2.1 Concepts

2.1.1 `succ`

successor: one more; on a character (Haskell's `Char` instance of `Enum`), the next codepoint.

```
>>> succ(41)
42
>>> succ('a')
'b'
>>> map_(succ, [1, 2, 3])
[2, 3, 4]
```

Roll your own — the definition:

```
succ = lambda x: chr(ord(x) + 1) if isinstance(x, str) else x + 1 # next: increments, advances chars
```

Used before it is rolled: `map_` in §4.1.1 (p. 31).

2.1.2 `pred`

predecessor: one less; characters step back one codepoint.

```
>>> pred(43)
42
>>> pred('b')
'a'
```

Roll your own — the definition:

```
pred = lambda x: chr(ord(x) - 1) if isinstance(x, str) else x - 1 # Enum: numbers and Chars
```

2.1.3 `even`

is `n` divisible by two? A predicate shaped for `filter_` and friends.

```
>>> even(4), even(7)
(True, False)
>>> filter_(even, [1, 2, 3, 4])
[2, 4]
```

Roll your own — the definition:

```
| even = lambda n: n % 2 == 0 # true for even numbers
```

Used before it is rolled: *filter_* in §4.1.2 (p. 31).

2.1.4 odd

the complement of even.

```
>>> odd(4), odd(7)
(False, True)
>>> partition(odd, [1, 2, 3, 4])
([1, 3], [2, 4])
```

Roll your own — the definition:

```
| odd = lambda n: n % 2 == 1 # true for odd numbers
```

Used before it is rolled: *partition* in §4.1.4 (p. 32).

2.1.5 not_

logical negation as a function ("not" is a keyword, so it cannot be passed around; not_ can). Composes predicates.

```
>>> not_(True)
False
>>> is_odd = compose(not_, even)
>>> is_odd(3)
True
```

Roll your own — the definition:

```
| not_ = lambda b: not b # `not` is a Python keyword
```

Used before it is rolled: *compose* in §16.1.8 (p. 136).

2.1.6 otherwise

just True, wearing Haskell's name for the catch-all guard. Its habitat is the RULES TABLE: a long if/elif ladder rewritten as data – (predicate, result) pairs tried in order, find picking the first match. One care: Haskell guards are EVALUATED, so bare otherwise works there; a table's guards are APPLIED to the argument, so wrap it as const(otherwise) – the guard that ignores its input and says yes. Result slots follow the same rule: fixed answers wear const, computed ones (like str below) go bare.

```
>>> otherwise
True
>>> RULES = [(lambda n: n % 15 == 0, const("FizzBuzz")),
...          (lambda n: n % 3 == 0, const("Fizz")),
...          (lambda n: n % 5 == 0, const("Buzz")),
...          (const(otherwise), str)]
>>> speak = lambda n: snd(find(lambda rule: fst(rule)(n), RULES))(n)
>>> map_(speak, [1, 3, 5, 15])
['1', 'Fizz', 'Buzz', 'FizzBuzz']
```

Roll your own — the definition:

```
| otherwise = True # guard fallback: always true
```

Used before it is rolled: *map_* in §4.1.1 (p. 31), *find* in §4.1.3 (p. 31), *const* in §16.1.2 (p. 133).

2.1.7 signum

the sign of a number as -1, 0 or 1, computed by arithmetic on two comparisons – no branches. Also a ready-made comparator result.

```
>>> signum(-7), signum(0), signum(3)
(-1, 0, 1)
```

```
>>> sortBy(lambda a, b: signum(a - b), [3, 1, 2])
[1, 2, 3]
```

Roll your own — the definition:

```
signum = lambda x: (x > 0) - (x < 0) # -1, 0, or 1 by the sign of x
```

Used before it is rolled: *sortBy* in §9.1.5 (p. 72).

2.1.8 div

integer division that FLOORS (rounds toward negative infinity), like Haskell's `div` and Python's `//`. The `div/mod` pair and the `quot/rem` pair agree on positives and split on negatives – interviews live in that gap.

```
>>> div(7, 2), div(-7, 2)
(3, -4)
```

Roll your own — the definition:

```
div = lambda a, b: a // b # floors, like Haskell div
```

2.1.9 mod

the remainder that matches `div`: its sign follows the DIVISOR, and `div(a,b)*b + mod(a,b) == a` always holds.

```
>>> mod(7, 2), mod(-7, 2), mod(7, -2)
(1, 1, -1)
```

Roll your own — the definition:

```
mod = lambda a, b: a % b # remainder, sign follows b
```

2.1.10 quot

integer division that TRUNCATES (rounds toward zero), like Haskell's `quot` and C's `/`. Same as `div` for positives; differs by one for negatives.

```
>>> quot(7, 2), quot(-7, 2)
(3, -3)
```

Roll your own — the definition:

```
quot = lambda a, b: -(-a // b) if (a < 0) != (b < 0) else a // b # truncates, like Haskell quot
```

2.1.11 rem

the remainder that matches `quot`: its sign follows the DIVIDEND.

```
>>> rem(7, 2), rem(-7, 2), rem(7, -2)
(1, -1, 1)
```

Roll your own — the definition:

```
rem = lambda a, b: a - b * quot(a, b) # remainder, sign follows a
```

2.1.12 quotRem

truncating quotient and remainder at once.

```
>>> quotRem(-7, 2)
(-3, -1)
```

Roll your own — the definition:

```
quotRem = lambda a, b: (quot(a, b), rem(a, b)) # (quot, rem) in one call
```

2.1.13 gcd

greatest common divisor by Euclid's recursion, always ≥ 0 like Haskell's. Recursion is safe here: the depth is at most ~ 90 even for 64-bit inputs (worst case: consecutive Fibonacci).

```
>>> gcd(48, 18)
6
>>> gcd(-4, 6), gcd(17, 5)
(2, 1)
```

Roll your own — the definition:

```
gcd = lambda a, b: abs(a) if b == 0 else gcd(b, a % b) #  $\geq 0$  like Haskell; depth  $\leq 90$ 
```

2.1.14 lcm

least common multiple via gcd, also always ≥ 0 ; defined as 0 when either input is 0.

```
>>> lcm(4, 6)
12
>>> lcm(-4, 6), lcm(0, 5)
(12, 0)
```

Roll your own — the definition:

```
lcm = lambda a, b: abs(a // gcd(a, b) * b) if a and b else 0 #  $\geq 0$  like Haskell
```

2.1.15 hypot

the Euclidean distance $\sqrt{x^2 + y^2}$. Note it returns a float.

```
>>> hypot(3, 4)
5.0
```

Roll your own — the definition:

```
hypot = lambda x, y: (x*x + y*y) ** 0.5 # straight-line distance
```

2.1.16 isqrt

floor square root using Newton's method on INTEGERS – no floats, so no precision loss on huge numbers; a negative input raises rather than lying.

```
>>> isqrt(16), isqrt(17)
(4, 4)
>>> isqrt(10**18)
10000000000
```

Roll your own: Newton's iteration on integers – repeatedly replace x with the average of x and $n // x$. It converges from above, so loop while the new estimate still shrinks; all arithmetic is $//$, so no float (and no float rounding lie) ever appears.

```
def isqrt(n): # floor sqrt without floats (Newton)
    if n < 0:
        raise ValueError("isqrt of negative")
    x = n
    y = halve(x + 1)
    while y < x:
        x, y = y, halve(y + n // y)
    return x if n else 0
```

2.1.17 add

(+) as a value, for slots that want a function, not an operator: scanl1, zipWith, foldl.

```
>>> add(2, 3)
5
>>> scanl1(add, [1, 2, 3])
[1, 3, 6]
```

Roll your own — the definition:

```
add = lambda a, b: a + b # (+) as a value: scanl1(add, xs)
```

Used before it is rolled: *scanl1* in §6.1.3 (p. 48).

2.1.18 sub

(-) as a value, same habitat as add.

```
>>> sub(5, 2)
3
>>> zipWith(sub, [10, 20], [1, 2])
[9, 18]
```

Roll your own — the definition:

```
sub = lambda a, b: a - b # (-) as a value: zipWith(sub, a, b)
```

Used before it is rolled: *zipWith* in §4.1.10 (p. 34).

2.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

2.1 Parity census

drill

Count how many values are even and how many are odd, as a pair (evens, odds). Zero is even; negatives carry their usual parity.

`parity_counts(xs) -> (evens, odds)`

```
>>> parity_counts([1, 2, 3, 4])
(2, 2)
>>> parity_counts([])
(0, 0)
>>> parity_counts([-2, -3])
(1, 1)
>>> parity_counts([0])
(1, 0)
```

Solution.

```
parity_counts = lambda xs: (sum(1 for x in xs if even(x)),
                             sum(1 for x in xs if odd(x)))
```

Note. Two generator sums over the parity predicates. One-pass with a fold is a fine alternative; the trap is forgetting that `even(-2)` and `even(0)` are `True`.

2.2 Floor versus truncate

drill

Write `both_divs(a, b)` returning `(div(a, b), quot(a, b))` so the two integer divisions can be compared side by side. They agree on positives and split on mixed signs.

`both_divs(a, b) -> (floored, truncated)`

```
>>> both_divs(7, 2)
(3, 3)
>>> both_divs(-7, 2)
(-4, -3)
```

```
>>> both_divs(7, -2)
(-4, -3)
>>> both_divs(-7, -2)
(3, 3)
```

Solution.

```
both_divs = lambda a, b: (div(a, b), quot(a, b))
```

Note. `div` floors toward negative infinity (Haskell/Python); `quot` truncates toward zero (C). Interviews live in the mixed-sign rows of this table.

2.3 Character neighbours

drill

Using the Enum behaviour of `succ` and `pred` on characters, return the pair of codepoint neighbours of a character: (previous, next). Works on letters and digits alike.

`char_neighbours(c) -> (prev_char, next_char)`

```
>>> char_neighbours('b')
('a', 'c')
>>> char_neighbours('m')
('l', 'n')
>>> char_neighbours('5')
('4', '6')
```

Solution.

```
char_neighbours = lambda c: (pred(c), succ(c))
```

Note. `succ`/`pred` dispatch on type: strings step through codepoints, numbers add one. That Enum duality is the whole drill.

2.4 Perfect square test

drill

Decide whether a non-negative integer is a perfect square, with no floating point anywhere – float `sqrt` lies for large inputs, `isqrt` does not.

`is_square(n) -> bool`

```
>>> is_square(16)
True
>>> is_square(17)
False
>>> is_square(0)
True
>>> is_square(10**18)
True
>>> is_square(10**18 + 1)
False
```

Solution.

```
is_square = lambda n: isqrt(n) ** 2 == n
```

Note. Round-trip through the integer square root. The `10**18` tests are exactly where float-based checks start failing.

2.5 Coprime pairs

drill

Two integers are coprime when they share no factor beyond 1 – that is, their gcd is exactly 1. Signs do not matter.

`coprime(a, b) -> bool`

```
>>> coprime(8, 15)
True
>>> coprime(6, 9)
False
>>> coprime(1, 99)
```

```
True
>>> coprime(-3, 4)
True
```

Solution.

```
| coprime = lambda a, b: gcd(a, b) == 1
```

Note. gcd is always ≥ 0 (Haskell semantics), so negative inputs need no special casing.

2.6 Meeting end on a 24-hour clock

apply

A meeting starts at `start_hour` (0..23) and runs for `duration` hours, possibly negative (rescheduled earlier) or longer than a day. Return the ending hour on the 24-hour clock. Flooring mod does all the wrap-around work.

`end_hour(start, duration) -> hour in 0..23`

```
>>> end_hour(23, 2)
1
>>> end_hour(9, 8)
17
>>> end_hour(0, 48)
0
>>> end_hour(10, -12)
22
```

Solution.

```
| end_hour = lambda start, duration: mod(start + duration, 24)
```

Note. `mod`'s sign follows the divisor, so negative totals land in 0..23 automatically – the reason `floor-mod`, not truncating `rem`, is the clock operator.

2.7 Pages needed

apply

A report has `n` items and each page holds `k`. How many pages are needed? This is ceiling division, and the branch-free trick is to floor-divide the NEGATION: `-div(-n, k)`.

`pages(n, k) -> int`

```
>>> pages(10, 3)
4
>>> pages(9, 3)
3
>>> pages(0, 5)
0
>>> pages(1, 10)
1
```

Solution.

```
| pages = lambda n, k: -div(-n, k)
```

Note. `ceil(n/k) == -floor(-n/k)`. The common alternative `div(n + k - 1, k)` also works; both beat importing `math.ceil` and converting through floats.

2.8 Reduced aspect ratio

apply

Given a screen's width and height in pixels, report its aspect ratio in lowest terms as a pair. 1920x1080 is famously 16:9.

`ratio(w, h) -> (w_reduced, h_reduced)`

```
>>> ratio(1920, 1080)
(16, 9)
>>> ratio(100, 100)
(1, 1)
>>> ratio(7, 3)
```

```
| (7, 3)
```

Solution.

```
| def ratio(w, h):
|     g = gcd(w, h)
|     return (div(w, g), div(h, g))
```

Note. Divide both sides by the gcd – the definition of lowest terms. $O(\log \min(w, h))$ from Euclid.

2.9 Cron co-firing

apply

Two cron jobs fire every a minutes and every b minutes, and both just fired together. In how many minutes do they next fire together again? That is the least common multiple.

`next_together(a, b) -> minutes`

```
| >>> next_together(4, 6)
| 12
| >>> next_together(5, 7)
| 35
| >>> next_together(10, 10)
| 10
| >>> next_together(1, 9)
| 9
```

Solution.

```
| next_together = lcm
```

Note. Recognize-it: the question IS lcm. Knowing $\text{lcm}(a, b) = a * b / \text{gcd}(a, b)$ is the interview follow-up.

2.10 Signal triage

apply

Sensor deltas arrive as integers. Triage them into (negative, zero, positive) counts using `signum` so the branching lives in arithmetic, not in if-chains.

`sign_census(xs) -> (negatives, zeros, positives)`

```
| >>> sign_census([-2, 0, 3, 4])
| (1, 1, 2)
| >>> sign_census([])
| (0, 0, 0)
| >>> sign_census([0, 0])
| (0, 2, 0)
```

Solution.

```
| def sign_census(xs):
|     signs = [signum(x) for x in xs]
|     return (signs.count(-1), signs.count(0), signs.count(1))
```

Note. `signum` compresses the three-way comparison; `count` reads the census. One map, three counts, $O(n)$.

Rolled later: count in §7.1.1 (p. 55).

2.11 Digital root

challenge

The digital root of a non-negative integer repeatedly replaces the number by the sum of its digits until a single digit remains: 38 -> 11 -> 2. Compute it with integer arithmetic only – no string conversion.

`digital_root(n) -> digit`

```
| >>> digital_root(0)
| 0
| >>> digital_root(9)
| 9
| >>> digital_root(38)
```

```

2
>>> digital_root(12345)
6
>>> digital_root(999)
9

```

Solution.

```

def digital_root(n):
    while n > 9:
        m, s = n, 0
        while m:
            m, r = divmod(m, 10)
            s += r
        n = s
    return n

```

Note. Digit-peeling via divmod inside a convergence loop. The slick closed form is 0 if $n == 0$ else $1 + \text{mod}(n - 1, 9)$ – casting out nines; mentioning it is a senior signal.

2.12 Visible lattice points

challenge

Standing at the origin of an integer grid, the point (x, y) is visible when no other lattice point lies on the straight segment between you and it – which happens exactly when $\text{gcd}(x, y) == 1$. The origin itself ($\text{gcd } 0$) is not visible. Count the visible points in a list.

```

visible_count(points) -> int

>>> visible_count([(1, 1), (2, 2), (0, 1), (0, 0), (3, 5)])
3
>>> visible_count([])
0
>>> visible_count([(0, 0)])
0
>>> visible_count([(1, 0), (0, 1)])
2

```

Solution.

```

visible_count = lambda points: sum(1 for (x, y) in points if gcd(x, y) == 1)

```

Note. A blocked point's coordinates share a factor $g > 1$ – the blocker is $(x/g, y/g)$. gcd's ≥ 0 semantics make negative quadrants free; $\text{gcd}(0, 0) == 0$ excludes the origin.

Chapter 3

Lists I: The Basics

The list is the prelude’s universal substrate, and this chapter is its anatomy: take lists apart at the ends, cut them at positions, pair them against each other – and against their own tails, which is what `pairwise` does and why every rule about “adjacent elements” starts there. `nub`, `transpose` and the prefix family each hide one idea worth owning outright.

3.1 Concepts

3.1.1 `head`

the first element. A PARTIAL function: on an empty list it raises, so guard with `null()` or reach for `find/fromMaybe` when emptiness is possible.

```
>>> head([1, 2, 3])
1
>>> head([])
Traceback (most recent call last):
...
IndexError: list index out of range
```

Roll your own — the definition:

```
head = lambda xs: xs[0] # first element
```

Used before it is rolled: last in §3.1.4 (p. 22).

3.1.2 `tail`

everything after the first element. `head` and `tail` together are the grammar of structural recursion: do something with `head`, recurse on `tail`.

```
>>> tail([1, 2, 3])
[2, 3]
>>> tail([1])
[]
```

Roll your own — the definition:

```
tail = lambda xs: xs[1:] # all but the first element
```

3.1.3 `init`

everything before the last element (the mirror of `tail`).

```
>>> init([1, 2, 3])
[1, 2]
```

Roll your own — the definition:

```
| init = lambda xs: xs[:-1] # all but the last element
```

Used before it is rolled: *last* in §3.1.4 (p. 22).

3.1.4 last

the final element. Partial, like head.

```
| >>> last([1, 2, 3])
| 3
```

Roll your own — the definition:

```
| last = lambda xs: xs[-1] # last element
```

3.1.5 null

is the list empty? The guard that makes head/tail safe to use.

```
| >>> null([], null([1])
| (True, False)
```

Roll your own — the definition:

```
| null = lambda xs: len(xs) == 0 # true for an empty list
```

3.1.6 elem

membership test, argument order (needle, haystack).

```
| >>> elem(2, [1, 2, 3])
| True
| >>> elem('q', "abc")
| False
```

Roll your own — the definition:

```
| elem = lambda x, xs: x in xs # true if x is in xs
```

3.1.7 notElem

non-membership.

```
| >>> notElem(9, [1, 2, 3])
| True
```

Roll your own — the definition:

```
| notElem = lambda x, xs: x not in xs # true if x is not in xs
```

3.1.8 replicate

n copies of one value, as a list.

```
| >>> replicate(3, 'x')
| ['x', 'x', 'x']
| >>> replicate(0, 'x')
| []
```

Roll your own — the definition:

```
| replicate = lambda n, x: [x] * n # x repeated n times
```

3.1.9 drop

discard the first *n* elements, keep the rest; finite lists only (it materialises the input), and a negative *n* drops nothing.

```
>>> drop(2, [1, 2, 3, 4])
[3, 4]
>>> drop(10, [1, 2]), drop(-3, [1, 2])
([], [1, 2])
```

Roll your own — the definition:

```
drop = lambda n, xs: list(xs)[max(n, 0):] # finite lists; n<0 drops none
```

3.1.10 splitAt

cut a list into (first *n*, rest) in one call; a negative *n* splits at the front. The pieces glue back with `+`, which is why rotations and chunking fall out of it.

```
>>> splitAt(2, [1, 2, 3, 4])
([1, 2], [3, 4])
>>> splitAt(0, [1, 2]), splitAt(-2, [1, 2])
(([], [1, 2]), ([1, 2], []))
```

Roll your own — the definition:

```
splitAt = lambda n, xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs) # n<0 splits at the front
```

3.1.11 nub

deduplicate, FIRST occurrence wins, order preserved. The trick: `dict.fromkeys` builds a dict (whose keys are unique and insertion-ordered) and throws away the values. Elements must be hashable. Compare `set(xs)`, which destroys order.

```
>>> nub([3, 1, 3, 2, 1])
[3, 1, 2]
>>> nub("mississippi")
['m', 'i', 's', 'p']
```

Roll your own — the definition:

```
nub = compose(list, dict.fromkeys) # unique; first seen wins
```

Used before it is rolled: *compose* in §16.1.8 (p. 136).

3.1.12 lookup

the first value paired with a key in an association list (a list of (key, value) tuples), or `None` if absent. First match wins, so an assoc list can shadow like a scope chain.

```
>>> lookup('b', [('a', 1), ('b', 2)])
2
>>> lookup('a', [('a', 1), ('a', 99)])
1
>>> lookup('z', [('a', 1)]) is None
True
```

Roll your own — the definition:

```
lookup = lambda k, pairs: next((v for kk, v in pairs if kk == k), None) # Nothing -> None
```

3.1.13 zip_

pair two iterables positionally, stopping at the shorter; returns a real list (the builtin `zip` returns a lazy iterator), and either side may be any iterable, streams included.

```
>>> zip_([1, 2], [10, 20, 30])
[(1, 10), (2, 20)]
>>> zip_("ab", count(0))
[('a', 0), ('b', 1)]
```

Roll your own — the definition:

```
zip_ = lambda a, b: list(zip(a, b)) # shortest wins, any iterable
```

Used before it is rolled: *count* in §7.1.1 (p. 55).

3.1.14 zip3

the three-list version.

```
>>> zip3([1, 2], ['a', 'b'], [True, False])
[(1, 'a', True), (2, 'b', False)]
```

Roll your own — the definition:

```
zip3 = lambda a, b, c: list(zip(a, b, c)) # zip three lists together
```

3.1.15 unzip

a list of pairs back into a pair of lists; the inverse of `zip_`.

```
>>> unzip([(1, 'a'), (2, 'b')])
([1, 2], ['a', 'b'])
>>> unzip([])
([], [])
```

Roll your own — the definition:

```
unzip = lambda ps: tuple(map(list, zip(*ps))) if ps else ([], []) # inverse of zip: pairs -> two lists
```

3.1.16 transpose

rows become columns, RAGGED-SAFE exactly like Haskell's: short rows simply drop out of later columns; nothing is silently truncated.

```
>>> transpose([[1, 2, 3], [4, 5, 6]])
[[1, 4], [2, 5], [3, 6]]
>>> transpose([[1, 2, 3], [4, 5]])
[[1, 4], [2, 5], [3]]
>>> transpose([])
[]
```

Rows may be any iterables, even one-shot ones – the rows are materialised first, so a generator of iterators transposes like a list of lists (without that step the generator is spent by the length scan):

```
>>> transpose(iter([1, 2]) for _ in range(2))
[[1, 1], [2, 2]]
>>> transpose(["abc", "de"])
[['a', 'd'], ['b', 'e'], ['c']]
```

Roll your own: materialise the rows, find the longest, then for each column index keep `xs[i]` only from rows that reach it – the `'if i < len(xs)'` filter is precisely where short rows drop out of later columns instead of truncating everyone.

```
def transpose(xss): # Data.List: rows <-> cols, RAGGED-SAFE like Haskell
    xss = [list(xs) for xs in xss] # any iterables in (rows may be one-shot); materialise once
    n = max(map(len, xss), default=0) # short rows just drop out of later columns (no truncation)
    return [[xs[i] for xs in xss if i < len(xs)] for i in range(n)]
```

3.1.17 enum

pair every element with its index: a list-returning enumerate. The shape that feeds index-aware folds (`two_sum`'s `seen-dict`, `next_greater`'s stack).

```
>>> enum(['a', 'b'])
[(0, 'a'), (1, 'b')]
>>> enum(['a', 'b'], start=1)
[(1, 'a'), (2, 'b')]
```

Roll your own — the definition:

```
enum = lambda xs, start=0: zip_(list(range(start, start + len(xs))), xs) # pairs each x with its index
```

3.1.18 pairwise

every element with its RIGHT neighbour: `zip xs (tail xs)`. The tool for any rule about adjacency – deltas, local comparisons, Roman numeral subtraction.

```
>>> pairwise([1, 4, 9])
[(1, 4), (4, 9)]
>>> map_(lambda p: p[1] - p[0], pairwise([3, 10, 6]))
[7, -4]
```

Roll your own — the definition:

```
pairwise = lambda xs: zip_(xs, xs[1:]) # zip xs (tail xs)
```

Used before it is rolled: *map_* in §4.1.1 (p. 31).

3.1.19 isPrefixOf

does `xs` start with `p`? Works on strings and lists alike (both are normalised through `list()`).

```
>>> isPrefixOf("re", "rebuild")
True
>>> isPrefixOf([1, 2], [1, 2, 3])
True
>>> isPrefixOf("", "anything")
True
```

Roll your own — the definition:

```
isPrefixOf = lambda p, xs: list(xs[:len(p)]) == list(p) # Data.List; strings or lists
```

3.1.20 isSuffixOf

does `xs` end with `p`? Implemented by slicing from `len(xs) - len(p)`, NOT by `xs[-len(p):]`, because when `p` is empty that slice is `xs[0:]` == the WHOLE list. A scar worth keeping visible.

```
>>> isSuffixOf("ing", "folding")
True
>>> isSuffixOf("", "x")
True
>>> isSuffixOf("abc", "bc")
False
```

Roll your own — the definition:

```
isSuffixOf = lambda p, xs: list(xs[len(xs) - len(p):]) == list(p) # not [-len(p):]: [-0:] is ALL
```

3.1.21 stripPrefix

Just the rest after the prefix, or Nothing: returns `xs[len(p):]` when the prefix matches, `None` when it does not. Pairs with `fromMaybe` and `mapMaybe`.

```
>>> stripPrefix("re", "rebuild")
'build'
>>> stripPrefix("un", "rebuild") is None
True
>>> mapMaybe(partial(stripPrefix, "err:"), ["err:a", "ok:b", "err:c"])
['a', 'c']
```

Roll your own — the definition:

```
stripPrefix = lambda p, xs: xs[len(p):] if isPrefixOf(p, xs) else None # Just the rest, or Nothing
```

Used before it is rolled: *mapMaybe* in §4.1.5 (p. 32), *partial* in §16.1.6 (p. 135).

3.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

3.1 Swap the ends

drill

Return a copy of a list (length ≥ 2) with its first and last elements swapped and everything in between left alone. Build it from the end-taking tools, not by index.

```
swap_ends(xs: list) -> list
```

```
>>> swap_ends([1, 2, 3, 4])
[4, 2, 3, 1]
>>> swap_ends([1, 2])
[2, 1]
>>> swap_ends([5, 9, 9, 5])
[5, 9, 9, 5]
```

Solution.

```
swap_ends = lambda xs: [last(xs)] + init(tail(xs)) + [head(xs)]
```

Note. `head/last` read the ends; `init(tail(xs))` is the middle (drop one from each side). $O(n)$ to rebuild. The classic slip is writing `xs[1:-1]` and forgetting it is empty for length-2 input – `init(tail(...))` handles that for free.

3.2 Deal the deck

drill

Deal a list into two halves. With an odd length the extra element goes to the SECOND half. Return the pair of halves.

```
deal(xs: list) -> tuple
```

```
>>> deal([1, 2, 3, 4])
([1, 2], [3, 4])
>>> deal([1, 2, 3])
([1], [2, 3])
>>> deal([])
([], [])
```

Solution.

```
deal = lambda xs: splitAt(len(xs) // 2, xs)
```

Note. `splitAt(n, xs)` cuts into `(xs[:n], xs[n:])` in one call; floor division sends the odd element rightward. $O(n)$.

3.3 Adjacent duplicate

drill

Report whether any two neighbouring elements of a list are equal. Empty and single-element lists have no neighbours, so the answer is False.

```
has_adjacent_dup(xs: list) -> bool
```

```
>>> has_adjacent_dup([1, 2, 2, 3])
True
>>> has_adjacent_dup([1, 2, 3])
False
>>> has_adjacent_dup([])
False
>>> has_adjacent_dup([5])
False
```

Solution.

```
| has_adjacent_dup = lambda xs: any(a == b for a, b in pairwise(xs))
```

Note. pairwise turns a list into its consecutive pairs – exactly the shape any rule about neighbours wants. It yields nothing for length < 2, so any(...) is False there. O(n).

3.4 Count the distinct

drill

Count how many distinct values a list contains. Order does not matter; duplicates count once.

```
count_distinct(xs: list) -> int
```

```
>>> count_distinct([1, 2, 2, 3, 1])
3
>>> count_distinct([])
0
>>> count_distinct([7, 7, 7])
1
```

Solution.

```
| count_distinct = lambda xs: len(nub(xs))
```

Note. nub keeps first occurrences (elements must be hashable); its length is the distinct count. O(n). set(xs) would work too but nub is the order-preserving habit.

3.5 Phone book lookup

drill

A phone book is a list of (name, number) pairs. Look up a name and return its number, or a supplied default string when the name is absent.

```
lookup_or(book: list, name, default) -> value
```

```
>>> book = [("ada", "123"), ("bob", "456")]
>>> lookup_or(book, "bob", "unknown")
'456'
>>> lookup_or(book, "cy", "unknown")
'unknown'
>>> lookup_or([], "ada", "unknown")
'unknown'
```

Solution.

```
| lookup_or = lambda book, name, default: fromMaybe(default, lookup(name, book))
```

Note. lookup returns the first match or None; fromMaybe supplies the fallback. The pairing of a None-returning search with fromMaybe is the whole Maybe idiom in one line.

3.6 Spreadsheet column totals

apply

A spreadsheet is a list of rows, each row a list of numbers. Rows may be ragged (some shorter than others). Return the total of each column; a short row simply contributes nothing to columns it does not reach. An empty sheet has no columns.

`column_sums(grid: list) -> list`

```
>>> column_sums([[1, 2, 3], [4, 5, 6]])
[5, 7, 9]
>>> column_sums([[1, 2, 3], [4, 5]])
[5, 7, 3]
>>> column_sums([])
[]
```

Solution.

```
| column_sums = lambda grid: [] if null(grid) else [sum(col) for col in transpose(grid)]
```

Note. `transpose` is ragged-safe: short rows drop out of later columns instead of truncating every column to the shortest row. `null` guards the empty sheet. $O(\text{cells})$.

3.7 Mastermind hits

apply

In Mastermind a guess and the secret code are equal-length sequences. Count the positions where the guess matches the secret exactly (the 'exact hits').

`matches(guess: list, secret: list) -> int`

```
>>> matches([1, 2, 3, 4], [1, 0, 3, 0])
2
>>> matches("abc", "abc")
3
>>> matches([], [])
0
```

Solution.

```
| matches = lambda guess, secret: sum(1 for a, b in zip_(guess, secret) if a == b)
```

Note. `zip_` walks both sequences in lockstep, stopping at the shorter; count the agreeing positions. $O(n)$. `zip_` returns a real list, so it also works on strings.

3.8 Strip a log tag

apply

Log lines sometimes begin with a fixed tag such as 'ERROR:'. Remove that leading tag if the line starts with it; otherwise return the line unchanged.

`strip_tag(tag: str, line: str) -> str`

```
>>> strip_tag("ERROR:", "ERROR:disk full")
'disk full'
>>> strip_tag("ERROR:", "all good")
'all good'
>>> strip_tag("", "unchanged")
'unchanged'
```

Solution.

```
| strip_tag = lambda tag, line: fromMaybe(line, stripPrefix(tag, line))
```

Note. `stripPrefix` returns the remainder after the tag, or `None` when the tag is absent; `fromMaybe` falls back to the whole line. An empty tag matches and strips nothing.

3.9 Filter by prefix and suffix

apply

Given a list of filenames, keep exactly those that both start with a required prefix and end with a required extension. Empty prefix or suffix matches everything.

```
by_affix(names: list, prefix: str, suffix: str) -> list
```

```
>>> by_affix(["test_a.py", "test_b.txt", "main.py"], "test_", ".py")
['test_a.py']
>>> by_affix(["a", "b"], "", "")
['a', 'b']
>>> by_affix(["main.py"], "test_", ".py")
[]
```

Solution.

```
by_affix = lambda names, prefix, suffix: [n for n in names
                                           if isPrefixOf(prefix, n)
                                           and isSuffixOf(suffix, n)]
```

Note. isPrefixOf / isSuffixOf work on strings and lists alike. Note isSuffixOf uses the len-based slice, not xs[-len(p):], so an empty suffix correctly matches all. O(n*k).

3.10 Pad a scoreboard

apply

A scoreboard must show exactly n slots. Given the current entries, pad on the right with a fill value until the list has length n. If it is already n or longer, leave it unchanged (never truncate).

```
pad_to(xs: list, n: int, fill) -> list
```

```
>>> pad_to([1, 2], 4, 0)
[1, 2, 0, 0]
>>> pad_to([1, 2, 3], 2, 0)
[1, 2, 3]
>>> pad_to([], 3, "-")
['-', '-', '-']
```

Solution.

```
pad_to = lambda xs, n, fill: xs + replicate(max(n - len(xs), 0), fill)
```

Note. replicate(k, fill) makes k copies; max(..., 0) means an already-long list gets zero padding rather than a negative count. O(n).

3.11 Roster reconciliation

challenge

An event keeps a list of previous attendees and a list of this time's check-ins (which may contain repeats). Produce three things: the DISTINCT current attendees who also attended before ('returning'), the distinct current attendees who did not ('newcomers'), and a flag that is True when there were no previous attendees at all. Preserve the first-seen order of the current check-ins in both lists.

```
roster_diff(prev: list, curr: list) -> tuple
```

```
>>> roster_diff(["ada", "bob"], ["bob", "cy", "cy", "ada", "dee"])
(['bob', 'ada'], ['cy', 'dee'], False)
>>> roster_diff([], ["ada", "ada", "bob"])
([], ['ada', 'bob'], True)
>>> roster_diff(["x"], [])
([], [], False)
```

Solution.

```
def roster_diff(prev, curr):
    distinct = nub(curr)
```

```

    returning = [x for x in distinct if elem(x, prev)]
    newcomers = [x for x in distinct if notElem(x, prev)]
    return (returning, newcomers, null(prev))

```

Note. nub gives the distinct current names in first-seen order; elem / notElem split them against the previous roster; null reports the empty-history case. $O(\text{len}(\text{curr}) * \text{len}(\text{prev}))$ with lists – a set(prev) would make membership $O(1)$ if prev were large.

3.12 Shipping manifest

challenge

A shipping manifest is built from three parallel arrays: item names, weights, and bin labels (equal length). Combine them into records, skip the first 'skip' records (already shipped), then number the remaining records from 1. Return the pair (numbered, total) where numbered is a list of (rank, name) and total is the summed weight of the kept records. Skipping past the end yields ([], 0).

```
manifest(names: list, weights: list, bins: list, skip: int) -> tuple
```

```

>>> manifest(["a", "b", "c"], [10, 20, 30], ["x", "y", "z"], 1)
([(1, 'b'), (2, 'c')], 50)
>>> manifest(["a", "b", "c"], [10, 20, 30], ["x", "y", "z"], 0)
([(1, 'a'), (2, 'b'), (3, 'c')], 60)
>>> manifest(["a"], [10], ["x"], 5)
([], 0)

```

Solution.

```

def manifest(names, weights, bins, skip):
    records = zip3(names, weights, bins)
    kept = drop(skip, records)
    numbered = [(i, r[0]) for i, r in enum(kept, 1)]
    cols = unzip(kept)
    total = sum(cols[1]) if kept else 0
    return (numbered, total)

```

Note. zip3 stitches the parallel arrays into records; drop discards the shipped prefix; enum(..., 1) numbers from 1; unzip splits the kept records back into columns so the weight column can be summed. $O(n)$.

Chapter 4

Lists II: Higher-Order

Where loops go to be named. `map_` transforms, `filter_` selects, `concatMap` enumerates, `zipWith` combines – and two tools do what loops do badly: `find` stops at the first hit (even on an infinite stream), and `mapMaybe` runs a parser over messy input and keeps only what parsed, one pass, no flags. This chapter is the single highest-frequency vocabulary in the whole course.

4.1 Concepts

4.1.1 `map_`

apply a function to every element; returns a real list, eagerly (the builtin `map` is lazy). The workhorse transformation.

```
>>> map_(lambda x: x * x, [1, 2, 3])
[1, 4, 9]
>>> map_(str.upper, ["ab", "cd"])
['AB', 'CD']
```

Roll your own — the definition:

```
map_ = lambda f, xs: [f(x) for x in xs] # apply f to every element
```

4.1.2 `filter_`

keep the elements satisfying a predicate. Scans the WHOLE list; if you only want the first hit, that is `find`.

```
>>> filter_(even, [1, 2, 3, 4])
[2, 4]
>>> filter_(str.isdigit, "a1b22")
['1', '2', '2']
```

Roll your own — the definition:

```
filter_ = lambda cond, xs: [x for x in xs if cond(x)] # keep elements where cond holds
```

4.1.3 `find`

the first element satisfying the predicate, else `None`. Lazy: it stops at the first hit, which folds cannot do, and therefore works even on infinite streams. Finish with `fromMaybe` if you need a default.

```
>>> find(even, [1, 3, 4, 6])
4
>>> find(even, [1, 3]) is None
True
>>> find(lambda x: x % 7 == 0, count(1))
7
```

How it works: `find` wraps a generator expression in `next()` with a `None` default. The generator is the point – elements are tested one at a time, on demand, and the search STOPS at the first hit. Nothing after the match is ever examined.

Why it earns its place: folds and `filter_` consume the whole list; they cannot stop early. `find` can – which makes it the right tool the moment the word "first" appears in a statement, and the only tool that survives an infinite stream. `Land` defaults with `fromMaybe`.

```
>>> find(lambda x: x * x > 50, count(1))
8
>>> fromMaybe('?', find(str.isdigit, "abc7de"))
'7'
```

Versus `filter_`: `filter_` answers "which ones?"; `find` answers "who was first, if anyone?". Taking `filter_(p, xs)[0]` instead does wasted work and explodes when nothing matches; `find` does neither.

Remember it as: `find` is the early exit that folds cannot perform.

Roll your own: a generator expression handed to `next()` with a default. The generator is what makes it lazy – elements are produced one at a time, so the first hit stops everything – and the default is what makes it total.

```
find = lambda cond, xs: next((x for x in xs if cond(x)), None) # first match, else None
```

Used before it is rolled: `count` in §7.1.1 (p. 55).

4.1.4 partition

`split` into (satisfy, don't) in ONE pass, both sides in original order; the predicate runs exactly once per element, so it may be expensive or even stateful.

```
>>> partition(even, [1, 2, 3, 4])
([2, 4], [1, 3])
>>> partition(str.isalpha, "a1b2")
(['a', 'b'], ['1', '2'])
```

Roll your own: two buckets and one conditional append – (yes if `cond(x)` else no).`append(x)`. The conditional expression picks the LIST, then the append mutates it; one pass, exactly one predicate call per element.

```
def partition(cond, xs): # (keepers, rest) in ONE pass; cond called once per element
    yes, no = [], []
    for x in xs:
        (yes if cond(x) else no).append(x)
    return (yes, no)
```

4.1.5 mapMaybe

`map` a function that returns a value or `None`, and keep only the values: parse-and-filter in ONE pass. The shape for messy input – write a parser that answers `None` for garbage, then `mapMaybe` it over everything. `dict.get` is already such a function.

```
>>> mapMaybe(lambda s: int(s) if s.isdigit() else None, ["3", "x", "7"])
[3, 7]
>>> mapMaybe({'a': 1, 'b': 2}.get, ['a', 'x', 'b'])
[1, 2]
```

How it works: run `f` over every element; `f` answers a VALUE when it can make sense of the input and `None` when it cannot; keep only the values. `Map` and `filter` fused into one pass, with the parser itself deciding what survives.

```
>>> parse_int = lambda s: int(s) if s.lstrip('-').isdigit() else None
>>> mapMaybe(parse_int, ["12", "x", "-3", ""])
[12, -3]
```

Why it earns its place: messy input. The alternative is a loop with an if, a flag and an append – three chances for a bug. Here the contract is crisp: write `parse(item) -> value-or-None` once, and `mapMaybe` handles every item. `dict.get` is already such a function, so lookups that may miss compose directly.

```
>>> legend = {'a': 1, 'b': 2}
>>> mapMaybe(legend.get, "cabbage")
[1, 2, 2, 1]
```

The Haskell connection: `Data.Maybe`'s `mapMaybe`, with `None` standing in for `Nothing` – the "parse, don't validate" discipline: turn questionable data into clean data at the boundary, and everything downstream stops worrying.

Remember it as: map with a parser, drop the Nothings, one pass.

Roll your own — the definition:

```
|mapMaybe = lambda f, xs: [y for y in (f(x) for x in xs) if y is not None]
```

4.1.6 catMaybes

keep the non-None values; mapMaybe with the identity.

```
|>>> catMaybes([1, None, 2, None])
| [1, 2]
```

Roll your own — the definition:

```
|catMaybes = partial(filter_, isJust) # mapMaybe id
```

Used before it is rolled: *id* in §16.1.1 (p. 133), *partial* in §16.1.6 (p. 135).

4.1.7 concat

flatten ONE level of nesting.

```
|>>> concat([[1, 2], [3], []])
| [1, 2, 3]
|>>> concat(["ab", "cd"])
| ['a', 'b', 'c', 'd']
```

Roll your own — the definition:

```
|concat = lambda xss: [x for xs in xss for x in xs] # flatten one level of nesting
```

4.1.8 concatMap

map each element to a LIST, and pool all the results: concat after map_, fused. The backbone of enumeration – "for every choice, produce all outcomes".

```
|>>> concatMap(lambda w: [w, w.upper()], ["a", "b"])
| ['a', 'A', 'b', 'B']
|>>> concatMap(lambda x: [x] * x, [1, 2, 3])
| [1, 2, 2, 3, 3, 3]
|>>> concatMap(lambda x: [x, -x], [1, 2]) == concat(map_(lambda x: [x, -x], [1, 2]))
| True
```

Roll your own — the definition:

```
|concatMap = lambda f, xs: [y for x in xs for y in f(x)] # map f over xs, then flatten
```

4.1.9 starmap

map a MULTI-argument function over a list of argument tuples: map_ composed with uncurry.

```
|>>> starmap(lambda a, b: a * b, [(2, 3), (4, 5)])
| [6, 20]
|>>> starmap(pow, [(2, 3), (3, 2)])
| [8, 9]
```

Roll your own — the definition:

```
|starmap = lambda f, pairs: [f(*p) for p in pairs] # map . uncurry
```

Used before it is rolled: *uncurry* in §16.1.5 (p. 135).

4.1.10 zipWith

combine two lists elementwise with a function; stops at the shorter. `zip_` is just `zipWith(make-a-pair)`.

```
>>> zipWith(add, [1, 2], [10, 20])
[11, 22]
>>> zipWith(lambda a, b: a > b, [3, 1], [2, 5, 9])
[True, False]
```

Roll your own — the definition:

```
zipWith = lambda f, a, b: [f(x, y) for x, y in zip(a, b)] # combine two lists elementwise
```

4.1.11 zipWith3

the three-list version.

```
>>> zipWith3(lambda a, b, c: a + b + c, [1], [2], [3])
[6]
```

Roll your own — the definition:

```
zipWith3 = lambda f, a, b, c: [f(x, y, z) for x, y, z in zip(a, b, c)] # combine three lists elementwise
```

4.1.12 cross

every pairing of two lists (the cartesian product). For the product of a list with ITSELF *n* times, see `replicateM`.

```
>>> cross([1, 2], "ab")
[(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
>>> len(cross(range(3), range(4)))
12
```

Roll your own — the definition:

```
cross = lambda a, b: [(x, y) for x in a for y in b] # cartesian; `product` is the fold
```

Used before it is rolled: *product* in §5.1.5 (p. 41).

4.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

4.1 Squares of the evens

drill

Given a list of integers, return the squares of just the even ones, in order. This is the filter-then-map pipeline in its smallest form.

`squares_of_evens(xs: list) -> list`

```
>>> squares_of_evens([1, 2, 3, 4])
[4, 16]
>>> squares_of_evens([1, 3, 5])
[]
>>> squares_of_evens([])
[]
```

Solution.

```
squares_of_evens = lambda xs: map_(lambda x: x * x, filter_(even, xs))
```

Note. `filter_` selects, `map_` transforms – read the pipeline inside-out, filter first. $O(n)$. `even` is the prelude predicate, ready to hand to `filter_`.

4.2 First over the line

drill

Return the first element of a list strictly greater than a threshold, or None if no element qualifies. Stop at the first hit – do not scan the whole list.

```
first_over(xs: list, t) -> value | None
```

```
>>> first_over([1, 5, 3, 9], 4)
5
>>> first_over([1, 2, 3], 9) is None
True
>>> first_over([], 0) is None
True
```

Solution.

```
first_over = lambda xs, t: find(lambda x: x > t, xs)
```

Note. find is the early exit a fold cannot make: it produces elements lazily and stops at the first match, returning None on failure. $O(k)$ where k is the winner's position.

4.3 Pass / fail split

drill

Split exam scores into (passed, failed) against a cutoff, where a score $\geq$ cutoff passes. Keep both groups in their original order and make a single pass.

```
split_pass_fail(scores: list, cutoff: int) -> tuple
```

```
>>> split_pass_fail([50, 80, 60, 90], 70)
([80, 90], [50, 60])
>>> split_pass_fail([], 70)
([], [])
>>> split_pass_fail([100, 100], 50)
([100, 100], [])
```

Solution.

```
split_pass_fail = lambda scores, cutoff: partition(lambda s: s >= cutoff, scores)
```

Note. partition returns (keepers, rest) in one pass, calling the predicate exactly once per element – cheaper and clearer than two filter_ passes. $O(n)$.

4.4 Salvage the integers

drill

Given a list of string tokens, return the integers among them, in order, dropping any token that is not a (possibly negative) integer. Empty strings are dropped.

```
clean_ints(tokens: list) -> list
```

```
>>> clean_ints(["12", "x", "-3", ""])
[12, -3]
>>> clean_ints(["0", "00"])
[0, 0]
>>> clean_ints(["nope"])
[]
```

Solution.

```
clean_ints = lambda toks: mapMaybe(
    lambda s: int(s) if s.lstrip("-").isdigit() else None, toks)
```

Note. mapMaybe fuses parse-and-filter: the parser answers a value or None, and the Nones are dropped in one pass. lstrip('-') lets a single leading minus through. $O(n)$.

4.5 Stutter

drill

Return a new list in which every element of the input appears twice in a row, in order.

```
duplicate_each(xs: list) -> list
```

```
>>> duplicate_each([1, 2])
[1, 1, 2, 2]
>>> duplicate_each([])
[]
>>> duplicate_each(["a"])
['a', 'a']
```

Solution.

```
duplicate_each = lambda xs: concatMap(lambda x: [x, x], xs)
```

Note. `concatMap` maps each element to a LITTLE LIST and flattens the results in one step – the shape for ‘each input yields several outputs’. $O(n)$.

4.6 Clean the sensor feed

apply

A sensor feed is a list of readings in which dropped samples appear as `None`. Return the pair (good, dropped): the list of surviving readings in order, and how many were `None`.

```
clean_readings(readings: list) -> tuple
```

```
>>> clean_readings([1, None, 2, None, 3])
([1, 2, 3], 2)
>>> clean_readings([None, None])
([], 2)
>>> clean_readings([4, 5])
([4, 5], 0)
```

Solution.

```
def clean_readings(readings):
    good = catMaybes(readings)
    return (good, len(readings) - len(good))
```

Note. `catMaybes` drops the Nones (it is `mapMaybe` with the identity); the drop count is the length difference. $O(n)$.

4.7 Line-item totals

apply

An order has parallel arrays of unit prices and quantities. Return the total for each line item (price times quantity), position by position. If the arrays differ in length, the shorter one governs.

```
line_totals(prices: list, quantities: list) -> list
```

```
>>> line_totals([2, 3, 5], [4, 1, 2])
[8, 3, 10]
>>> line_totals([2, 3, 5], [4])
[8]
>>> line_totals([], [])
[]
```

Solution.

```
line_totals = lambda prices, quantities: zipWith(mul, prices, quantities)
```

Note. `zipWith` combines two lists elementwise with a function, stopping at the shorter – the go-to for ‘do X to matching positions of two arrays’. $O(n)$.

4.8 Three-column totals

apply

An invoice has three parallel columns: base price, tax, and tip. Return the grand total for each row by summing the three columns position by position.

```
row_totals(base: list, tax: list, tip: list) -> list
```

```
>>> row_totals([10, 20], [1, 2], [2, 3])
[13, 25]
>>> row_totals([10], [1], [2])
[13]
>>> row_totals([], [], [])
[]
```

Solution.

```
| row_totals = lambda base, tax, tip: zipWith3(lambda b, t, p: b + t + p, base, tax, tip)
```

Note. zipWith3 is zipWith for three lists at once – no intermediate tuples. $O(n)$.

4.9 Rectangle areas

apply

Given a list of rectangles, each a (width, height) pair, return the area of each rectangle in order.

```
areas(rects: list) -> list
```

```
>>> areas([(2, 3), (4, 5)])
[6, 20]
>>> areas([(0, 9)])
[0]
>>> areas([])
[]
```

Solution.

```
| areas = lambda rects: starmap(mul, rects)
```

Note. starmap spreads each tuple across a multi-argument function – map_ composed with uncurry, so the (w, h) pairs unpack straight into the lambda. $O(n)$.

4.10 Merge the playlists

apply

Given several playlists, each a list of song titles, merge them into one playlist in order, then remove duplicate songs keeping the first appearance of each.

```
merge_playlists(playlists: list) -> list
```

```
>>> merge_playlists([["a", "b"], ["b", "c"], ["a"]])
['a', 'b', 'c']
>>> merge_playlists([])
[]
>>> merge_playlists([], ["solo"])
['solo']
```

Solution.

```
| merge_playlists = lambda playlists: nub(concat(playlists))
```

Note. concat flattens the list of lists by one level; nub then drops repeats keeping first occurrence. $O(\text{total songs})$.

4.11 Affordable outfits

challenge

A wardrobe has shirts and pants, each an (name, price) pair. Generate every shirt-and-pant outfit, keep only those whose combined price is within a budget, and return them as (shirt_name, pant_name) pairs in generation order (shirts outer, pants

```
inner).
affordable_outfits(shirts: list, pants: list, budget: int) -> list
```

```
>>> shirts = [("red", 20), ("blue", 30)]
>>> pants = [("a", 40), ("b", 10)]
>>> affordable_outfits(shirts, pants, 45)
[('red', 'b'), ('blue', 'b')]
>>> affordable_outfits(shirts, pants, 5)
[]
>>> affordable_outfits([], pants, 100)
[]
```

Solution.

```
def affordable_outfits(shirts, pants, budget):
    pairs = cross(shirts, pants)
    ok = filter_(lambda pr: pr[0][1] + pr[1][1] <= budget, pairs)
    return [(s[0], p[0]) for s, p in ok]
```

Note. cross is the Cartesian product – every (shirt, pant) combination; filter_ keeps the affordable ones; a comprehension pulls the names out. $O(\text{len}(\text{shirts}) * \text{len}(\text{pants}))$.

4.12 Inventory reconciliation

challenge

An inventory log is a list of transaction strings. Each is an item name optionally led by a sign: '+apple' or 'apple' is an addition, '-banana' is a removal. Blank strings are skipped. Parse the log, then return the pair (added, removed): the item names that were added and the item names that were removed, each in the order they appeared in the log.

```
reconcile(txns: list) -> tuple
```

```
>>> reconcile(["+apple", "-banana", "cherry", "", "-apple"])
(['apple', 'cherry'], ['banana', 'apple'])
>>> reconcile(["", " "])
([], [])
>>> reconcile(["milk", "-milk"])
(['milk'], ['milk'])
```

Solution.

```
def reconcile(txns):
    def parse(t):
        t = t.strip()
        if not t:
            return None
        if t[0] == '-':
            return (-1, t[1:])
        if t[0] == '+':
            return (1, t[1:])
        return (1, t)
    signed = mapMaybe(parse, txns)
    adds, rems = partition(lambda sr: sr[0] > 0, signed)
    return ([item for _, item in adds], [item for _, item in rems])
```

Note. mapMaybe parses each line to a (sign, item) record and drops the blanks; partition splits additions from removals in one pass; comprehensions strip the sign back off. $O(n)$. The common bug is treating a bare 'apple' as malformed rather than an addition.

Chapter 5

The Fold

The trunk of the tree. A fold is a for-loop whose state has been given a contract: an accumulator, a combiner, one pass. Everything after this chapter is a fold in costume – scans keep the fold's history, unfolds run it backwards, dict folds aim it at a mapping, memo folds a recursion into a cache. Learn the two argument disciplines cold: `foldl`'s combiner takes (`acc`, `x`), `foldr`'s takes (`x`, `acc`). The enumeration folds close the chapter with the brute-force licenses: 2^n subsets and $|xs|^n$ words, legal when you say the bound out loud.

5.1 Concepts

5.1.1 `foldl`

THE left fold, and the prelude's most important definition. It is exactly this loop: `acc = base; for x in xs: acc = f(acc, x); return acc`. The combiner takes (`ACCUMULATOR`, `element`) – accumulator first. Argument order of `foldl` itself: (`f`, `xs`, `base`), with `base` optional – omitted, the first element seeds the accumulator (and an empty list is then an error).

```
>>> foldl(lambda acc, x: acc * 10 + x, [1, 2, 3], 0)
123
>>> foldl(max, [3, 9, 2])           # no base: 3 seeds it
9
>>> foldl(add, [], 0)               # empty + base: the base comes back
0
>>> foldl(lambda acc, w: acc + len(w), ["ab", "c"], 0)
3
```

How it works: `foldl` is precisely this loop, given a name and a contract:

```
>>> def foldl_spelled_out(f, xs, base):
...     acc = base
...     for x in xs:
...         acc = f(acc, x)
...     return acc
>>> foldl_spelled_out(add, [1, 2, 3], 0) == foldl(add, [1, 2, 3], 0)
True
```

Two disciplines hide in the signature. The combiner takes (`ACCUMULATOR`, `element`) – accumulator first, always; reversing them is the classic fold bug. And the optional base is guarded by `NOTHING`, so the fold can tell "no seed – use the first element" from "the seed is `None`".

Why it earns its place: the fold is the universal list consumer. `sum`, `max`, `reverse`, "build a dict", "run a state machine over events" – each is one choice of combiner and seed. When you can say "carry THIS state, update it THIS way per element", the loop is already written.

```
>>> foldl(lambda acc, x: [x] + acc, [1, 2, 3], [])           # reverse is a fold
[3, 2, 1]
>>> foldl(lambda acc, w: acc + len(w), ["ab", "cde"], 0)
5
```

Remember it as: a fold is a for-loop whose state has been given a contract.

Roll your own: seed the accumulator – base if one was given, else the first element (the NOTHING guard is what lets None be a legitimate seed) – then a single for-loop folding $acc = f(acc, x)$. The entire contract is which side the accumulator sits on.

```
def foldl(f, xs, base=NOTHING): # THE left fold; Python buried its own in functools as reduce
    it = iter(xs)
    if base is NOTHING:
        try:
            acc = next(it)
        except StopIteration:
            raise TypeError("fold of empty sequence with no initial value")
    else:
        acc = base
    for x in it:
        acc = f(acc, x)
    return acc
```

5.1.2 foldl1

foldl seeded by the first element, spelled out. Use when the accumulator has the same type as the elements and emptiness cannot happen.

```
>>> foldl1(add, [1, 2, 3])
6
>>> foldl1(min, [3, 1, 2])
1
```

Roll your own — the definition:

```
foldl1 = foldl # foldl seeded by the first element
```

5.1.3 foldr

the RIGHT fold: nests from the right, f takes (element, ACCUMULATOR) – element first, mirror of foldl. Implemented as foldl of the flipped function over the reversed list: iterative and stack-safe, an identity that only holds because Python is strict. The parenthesis example makes the direction visible.

```
>>> foldr(lambda x, acc: '(' + x + '+' + acc + ')', "abc", "z")
'(a+(b+(c+z)))'
>>> foldl(lambda acc, x: '(' + acc + '+' + x + ')', "abc", "z")
'(((z+a)+b)+c)'
```

```
>>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], [])
[1, 2, 3]
```

How it works: the mirror image – the combiner takes (element, ACCUMULATOR) and the nesting grows from the RIGHT: $f(a, f(b, f(c, z)))$. The prelude does not write a second recursion: $foldr = foldl(flip(f), reversed(xs), base)$. Flip fixes the argument order, reversing fixes the traversal, and the existing engine runs backwards.

The honesty clause: that identity only holds because Python is STRICT. Haskell's foldr can process infinite lists – laziness lets f decide whether the rest is ever needed; ours materialises and reverses the list first, so it cannot. Same name, same finite behaviour, different superpower.

Why it earns its place: building right-to-left. Consing onto a front is natural from the right – section 13's from_list IS a foldr of ListNode, and here it is, derived by hand:

```
>>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], []) # rebuild, order kept
[1, 2, 3]
>>> to_list(foldr(ListNode, [1, 2, 3], None)) # from_list, spelled out
[1, 2, 3]
```

Remember it as: foldl eats left-to-right; foldr thinks right-to-left – and in strict Python it is foldl wearing flip and a reversal.

Roll your own — the definition:

```
foldr = lambda f, xs, base: foldl(flip(f), reversed(list(xs)), base) # THE right fold, via foldl
```

Used before it is rolled: *ListNode* in §13.1.9 (p. 109), *from_list* in §13.1.10 (p. 109), *to_list* in §13.1.11 (p. 110), *flip* in §16.1.3

(p. 133).

5.1.4 foldr1

right fold seeded by the LAST element.

```
>>> foldr1(sub, [1, 2, 3])    # 1 - (2 - 3)
2
```

Roll your own — the definition:

```
foldr1 = lambda f, xs: foldl(flip(f), reversed(list(xs))) # foldr seeded by the last element
```

Used before it is rolled: *flip* in §16.1.3 (p. 133).

5.1.5 product

multiply everything: the numeric fold, seeded with 1 so an empty product is 1.

```
>>> product([2, 3, 4])
24
>>> product([])
1
```

Roll your own — the definition:

```
product = lambda xs: foldl(mul, xs, 1) # the numeric fold
```

5.1.6 subsequences

every subset of the elements (the powerset), built by a fold in which each element **DOUBLES** the accumulator: keep every existing subset, and every existing subset extended by the newcomer. 2^n results – a brute-force license for n up to about 20, and you should say that bound out loud before using it.

```
>>> subsequences([1, 2])
[[], [1], [2], [1, 2]]
>>> subsequences("ab")
[[], ['a'], ['b'], ['a', 'b']]
>>> len(subsequences("abcd"))
16
```

How it works: watch the accumulator double. Start from `[]` – one subset, the empty one. Each element x rewrites `acc` as `acc + [s + [x] for s in acc]`: every existing subset survives (x left out), and every existing subset reappears extended by x (x taken). n elements, n doublings, 2^n subsets, in a stable order.

```
>>> acc = []
>>> acc = acc + [s + [1] for s in acc]; acc
[[], [1]]
>>> acc = acc + [s + [2] for s in acc]; acc
[[], [1], [2], [1, 2]]
```

Why it earns its place: it is a brute-force **LICENSE**. At $n \leq 20$, checking every subset is a few million cheap operations – honest, exhaustive, and unbeatable to debug. Say the bound out loud (" 2^{15} is 32768 – fine") and enumerate; cleverness can come later if the bound grows.

```
>>> best = maxOn(sum, [s for s in subsequences([3, -1, 4, -2]) if sum(s) <= 6])
>>> sum(best)
6
```

Remember it as: the powerset fold – each element doubles the world.

Roll your own — the definition:

```
subsequences = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs), [[]])
```

Used before it is rolled: *maxOn* in §9.1.3 (p. 71).

5.1.7 replicateM

every length- n word over an alphabet: cross applied n times, $|xs|^n$ results. The other brute-force license – all bitstrings, all dice rolls, all assignments of n slots.

```
>>> replicateM(2, "ab")
[['a', 'a'], ['a', 'b'], ['b', 'a'], ['b', 'b']]
>>> len(replicateM(3, [0, 1]))
8
```

Roll your own — the definition:

```
replicateM = lambda n, xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs], range(n), [[]])
```

5.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

5.1 Sum of squares

drill

Using `foldl`, compute the sum of the squares of a list of integers. The accumulator carries the running total; seed it with 0 so the empty list answers 0.

```
sum_squares(xs: list[int]) -> int
```

```
>>> sum_squares([1, 2, 3])
14
>>> sum_squares([])
0
>>> sum_squares([-2])
4
```

Solution.

```
sum_squares = lambda xs: foldl(lambda acc, x: acc + x * x, xs, 0)
```

Note. One fold, seed 0, combiner $acc + x^2$; $O(n)$. Common mistake: squaring the accumulator instead of the element.

5.2 Longest word, first on ties

drill

Using `foldl1`, return the longest word in a non-empty list. When two words tie in length, the EARLIER one must win – choose your comparison so the accumulator survives ties.

```
longest_word(ws: list[str]) -> str
```

```
>>> longest_word(["hi", "abc", "de"])
'abc'
>>> longest_word(["aa", "bb"])
'aa'
>>> longest_word(["x"])
'x'
```

Solution.

```
longest_word = lambda ws: foldl1(lambda acc, w: w if len(w) > len(acc) else acc, ws)
```

Note. `foldl1` seeds with the first element; the strict $>$ keeps the incumbent on ties. Using $\geq$ would hand ties to the newcomer – the classic tie-break slip.

5.3 Cons cells by hand

drill

Using `foldr`, convert a list into nested pairs ending in `None`, Lisp-style: `[1, 2, 3]` becomes `(1, (2, (3, None)))`. The nesting grows from the right, which is exactly what `foldr` does; note its combiner receives (element, accumulator).

```

to_pairs(xs: list) -> tuple | None
|>>> to_pairs([1, 2, 3])
(1, (2, (3, None)))
|>>> to_pairs(['a'])
('a', None)
|>>> to_pairs([]) is None
True

```

Solution.

```
| to_pairs = lambda xs: foldr(lambda x, acc: (x, acc), xs, None)
```

Note. foldr(f, xs, None) with f = make-a-pair; the seed None is the empty-list arm. Trying this with foldl builds the chain backwards.

5.4 Factorial

drill

Using product, compute n factorial. Remember product of an empty sequence is 1 – which is precisely why 0! works with no special case.

```

factorial(n: int) -> int
|>>> factorial(5)
120
|>>> factorial(0)
1
|>>> factorial(1)
1

```

Solution.

```
| factorial = lambda n: product(range(1, n + 1))
```

Note. product is the numeric fold seeded with 1, so the n = 0 case falls out for free; O(n).

5.5 Every substring team name

drill

Using subsequences, list all subsets of s's characters as strings – every string formable by deleting zero or more characters from s (keeping order): powerset_strings("ab") is ['', 'a', 'b', 'ab']. Join each subsequence back into a string; keep subsequences' own order.

```

powerset_strings(s: str) -> list[str]
|>>> powerset_strings("ab")
['', 'a', 'b', 'ab']
|>>> powerset_strings("")
['']
|>>> len(powerset_strings("abc"))
8

```

Solution.

```
| powerset_strings = lambda s: map_("", join, subsequences(s))
```

Note. subsequences gives lists of characters; one map_ of join finishes. 2^n results – say the bound before calling it on anything long.

5.6 Bank statement audit

apply

A bank account starts at 'start' (never negative) and processes a list of signed transactions in order. Return (final_balance, overdrawn) where overdrawn is True if the running balance ever dipped below zero at any point – even if later deposits recovered it. One fold: the accumulator carries both the balance and the flag.

```

balance_audit(start: int, txns: list[int]) -> tuple[int, bool]
|>>> balance_audit(100, [-150, 75])
(25, True)

```

```
>>> balance_audit(100, [])
(100, False)
>>> balance_audit(0, [-1, 1])
(0, True)
>>> balance_audit(10, [5])
(15, False)
```

Solution.

```
balance_audit = lambda start, txns: foldl(
    lambda st, x: (st[0] + x, st[1] or st[0] + x < 0), txns, (start, False))
```

Note. Tuple accumulator (balance, flag); the 'or' latches the flag once tripped. $O(n)$. Common mistake: checking only the final balance.

5.7 Ways to roll it

apply

You roll n six-sided dice. Using replicateM, count every way the ordered dice can sum to exactly target. n is small ($n \leq 6$), so enumerating all 6^n outcomes is the honest play.

```
ways_to_roll(n: int, target: int) -> int
```

```
>>> ways_to_roll(2, 7)
6
>>> ways_to_roll(1, 3)
1
>>> ways_to_roll(2, 13)
0
>>> ways_to_roll(3, 18)
1
```

Solution.

```
ways_to_roll = lambda n, target: len([r for r in replicateM(n, range(1, 7)) if sum(r) == target])
```

Note. replicateM(n , faces) is every ordered roll; filter and count. 6^n outcomes – state the bound. The DP version exists, but at $n \leq 6$ enumeration is simpler and safer.

5.8 Decompress the telemetry

apply

A sensor stream arrives run-length encoded as (char, count) pairs. Using foldl, decompress it back into the original string: $[('a', 3), ('b', 2)]$ becomes 'aaabb'.

```
rle_decode(pairs: list[tuple[str, int]]) -> str
```

```
>>> rle_decode([('a', 3), ('b', 2)])
'aaabb'
>>> rle_decode([])
''
>>> rle_decode([('x', 1)])
'x'
```

Solution.

```
rle_decode = lambda pairs: foldl(lambda acc, p: acc + p[0] * p[1], pairs, '')
```

Note. A fold whose accumulator is the output string; $\text{char} * \text{count}$ expands each run. The inverse of the g5 rle_encode problem – round-trip them as a self-check.

5.9 Config templating

apply

A deployment file is produced by applying find-and-replace edits IN ORDER: each edit is a (find, replace) pair, and later edits see the result of earlier ones. Using foldl with the text as the accumulator, apply them all.

```
apply_edits(text: str, edits: list[tuple[str, str]]) -> str
```

```
>>> apply_edits("hello world", [("world", "there"), ("hello", "hi")])
```

```
'hi there'
>>> apply_edits("aa", [("a", "b")])
'bb'
>>> apply_edits("cfg:$X", [("X", "1")])
'cfg:1'
>>> apply_edits("keep", [])
'keep'
```

Solution.

```
apply_edits = lambda text, edits: foldl(lambda t, e: t.replace(e[0], e[1]), edits, text)
```

Note. The accumulator is the evolving TEXT and the list folded over is the edits – a fold where state is a document, not a number. Order sensitivity is the point.

5.10 Exact change

apply

You hold a small pouch of coins, each usable at most once ($n \leq 20$). Using subsequences, decide whether there is any way to pick a subset of them summing to exactly target. The empty subset makes 0, so target 0 is always reachable.

```
exact_change(coins: list[int], target: int) -> bool
```

```
>>> exact_change([3, 7, 2], 9)
True
>>> exact_change([5, 10], 7)
False
>>> exact_change([], 0)
True
>>> exact_change([4], 0)
True
```

Solution.

```
exact_change = lambda coins, target: any(sum(s) == target for s in subsequences(coins))
```

Note. The powerset as a brute-force license: 2^n subsets at $n \leq 20$ is about a million cheap checks. `any()` stops at the first witness.

5.11 Minimal hiring committee

challenge

A project needs a set of skills. Each candidate brings a list of skills. Hiring is expensive, so find the SIZE of the smallest group of candidates that together covers every needed skill, or -1 if no group can. Candidate count is small ($n \leq 12$): enumerate groups with subsequences, keep the covering ones, minimise the size. An empty requirement needs zero hires.

```
min_team(required: list[str], candidates: list[list[str]]) -> int
```

```
>>> min_team(["py", "sql"], [["py"], ["sql"], ["py", "sql"]])
1
>>> min_team(["a", "b"], [["a"], ["b"]])
2
>>> min_team(["go"], [["py"]])
-1
>>> min_team([], [["x"]])
0
```

Solution.

```
def min_team(required, candidates):
    covers = [s for s in subsequences(candidates) if set(required) <= set(concat(s))]
    return min(map_len, covers), default=-1)
```

Note. Set cover is NP-hard in general; $n \leq 12$ licenses the 2^n sweep. `concat` flattens a group's skills; set inclusion checks coverage; `min` with default handles impossibility.

5.12 Fair loot split

challenge

Two players split a pile of treasure items (values, $n \leq 20$) into two groups; every item goes to one side. Return the smallest possible absolute difference between the two groups' totals. Observe that choosing one side's subset s fixes the difference to $\text{abs}(\text{total} - 2 * \text{sum}(s))$, then let subsequences try every side.

```
min_partition_diff(xs: list[int]) -> int
```

```
>>> min_partition_diff([1, 6, 11, 5])
1
>>> min_partition_diff([3, 1])
2
>>> min_partition_diff([2, 2])
0
>>> min_partition_diff([])
0
>>> min_partition_diff([7])
7
```

Solution.

```
def min_partition_diff(xs):
    total = sum(xs)
    return min(abs(total - 2 * sum(s)) for s in subsequences(xs))
```

Note. The classic balanced-partition interview problem, honestly brute-forced: one algebraic observation turns it into a powerset minimum. 2^n at $n \leq 20$; the DP version is the follow-up conversation.

5.13 Next greater reading

challenge

Daily sensor readings arrive as a list. For each position report the NEXT GREATER reading to its right – the first later value strictly larger – or -1 when no later reading beats it. A double loop is $O(n^2)$; a list used as a stack of still-unanswered positions, driven by one left-to-right fold, answers everything in $O(n)$.

```
next_greater(xs: list[int]) -> list[int]
```

```
>>> next_greater([2, 1, 3])
[3, 3, -1]
>>> next_greater([1, 2, 3, 4])
[2, 3, 4, -1]
>>> next_greater([4, 3, 2, 1])
[-1, -1, -1, -1]
>>> next_greater([2, 2])
[-1, -1]
>>> next_greater([])
[]
```

Solution.

```
def next_greater(xs):
    res = [-1] * len(xs)
    def step(stack, i):
        while stack and xs[stack[-1]] < xs[i]:
            res[stack.pop()] = xs[i]
        stack.append(i)
        return stack
    foldl(step, range(len(xs)), [])
    return res
```

Note. The monotonic stack holds indices still waiting for their answer; each index is pushed once and popped at most once, so the fold is $O(n)$. 'Next greater' is the recognition ladder's textbook STACK trigger – same skeleton as matching pairs of brackets.

Chapter 6

Scans

A scan is a fold that keeps its history – every intermediate accumulator, not just the last. That one change turns “the state at every step” from n folds into one pass, makes prefix sums (and with them $O(1)$ range queries) a single line, and reveals celebrated one-pass algorithms – Kadane’s most famously – as scans in a trench coat. When a statement says “running”, “so far” or “at each step”, it is dictating this chapter.

6.1 Concepts

6.1.1 `accumulate`

the generator behind the scans: yields the running combination of everything seen so far. With no function it sums; ‘initial’ seeds it. This is Python’s `itertools` name with Haskell’s scan semantics.

```
>>> list(accumulate([1, 2, 3]))
[1, 3, 6]
>>> list(accumulate([1, 2], initial=10))
[10, 11, 13]
>>> list(accumulate([3, 1, 2], min))
[3, 1, 1]
```

Roll your own: the same skeleton as `foldl`, but a GENERATOR – yield the accumulator once before the loop, then yield again after every combine. The early bare return on an empty, seedless input is what makes an empty scan come back empty instead of raising.

```
def accumulate(xs, f=None, initial=NOTHING): # scanl / scanl1
    if f is None:
        f = add
    it = iter(xs)
    if initial is NOTHING:
        try:
            acc = next(it)
        except StopIteration:
            return
    else:
        acc = initial
    yield acc
    for x in it:
        acc = f(acc, x)
        yield acc
```

Used before it is rolled: `scanl` in §6.1.2 (p. 47), `scanl1` in §6.1.3 (p. 48).

6.1.2 `scanl`

a fold that KEEPS ITS HISTORY: returns every intermediate accumulator, starting with the seed, so the result has $\text{len}(xs)+1$ elements and its last element equals the foldl. Argument order (f, z, xs) – seed in the middle, Haskell’s order, unlike `foldl`’s

trailing base. Prefix sums are the canonical scan.

```
>>> scanl(add, 0, [3, 1, 4])
[0, 3, 4, 8]
>>> last(scanl(add, 0, [1, 2, 3])) == foldl(add, [1, 2, 3], 0)
True
```

The DP secret: many celebrated one-pass algorithms are scans in a trench coat, because a scan IS a dynamic-programming table – entry *i* holds the answer for the prefix ending at *i*. Kadane’s famous maximum-subarray "trick" is one `scanl` and a `max`:

```
>>> xs = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
>>> max(scanl(lambda best, x: max(x, best + x), xs))
6
```

Prefix sums, the other superpower: scan once with (+) and every range sum becomes two reads, `pref[j] - pref[i]` – $O(n)$ window questions collapse to $O(1)$.

```
>>> pref = scanl(add, 0, [3, 1, 4, 1, 5])
>>> pref[4] - pref[1] # sum of elements 1..3
6
```

Remember it as: when you need the state at EVERY step, one scan replaces *n* folds.

Roll your own — the definition:

```
scanl = lambda f, z, xs: list(accumulate(xs, f, initial=z)) # foldl, keeping every accumulator
```

Used before it is rolled: `scanl1` in §6.1.3 (p. 48).

6.1.3 `scanl1`

the history-keeping fold seeded by the first element; `len(xs)` results. Running maxima and minima live here.

```
>>> scanl1(min, [3, 1, 2])
[3, 1, 1]
>>> scanl1(max, [1, 3, 2, 5])
[1, 3, 3, 5]
```

Roll your own — the definition:

```
scanl1 = lambda f, xs: list(accumulate(xs, f)) # scanl with no explicit base
```

6.1.4 `scanr`

the suffix scan: folds from the RIGHT, showing every suffix’s result; the first element is the full fold, the last is the seed. Suffix sums for `product_except_self`-style problems.

```
>>> scanr(add, 0, [1, 2, 3])
[6, 5, 3, 0]
```

Roll your own — the definition:

```
scanr = lambda f, z, xs: list(reversed(scanl(flip(f), z, list(reversed(list(xs))))) # foldr, keeping every acc
```

Used before it is rolled: `flip` in §16.1.3 (p. 133).

6.1.5 `scanr1`

suffix scan seeded by the last element.

```
>>> scanr1(add, [1, 2, 3])
[6, 5, 3]
```

Roll your own — the definition:

```
scanr1 = lambda f, xs: list(reversed(scanl1(flip(f), list(reversed(list(xs))))) # scanr with no base
```

Used before it is rolled: `flip` in §16.1.3 (p. 133).

6.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

6.1 Running maximum

drill

Using `scanl1`, produce the running maximum of a list: element `i` of the answer is the largest value seen among the first `i+1` inputs. An empty list yields an empty scan.

```
running_max(xs: list[int]) -> list[int]
```

```
>>> running_max([3, 1, 4, 1, 5])
[3, 3, 4, 4, 5]
>>> running_max([7])
[7]
>>> running_max([])
[]
```

Solution.

```
running_max = lambda xs: scanl1(max, xs)
```

Note. `scanl1` seeds with the first element and keeps every intermediate; `max` is the combiner. $O(n)$. This scan IS the best-so-far column of many DP tables.

6.2 Account history

drill

An account opens at balance 'start' and applies signed transactions in order. Using `scanl`, return the FULL balance history – the balance at each step, opening balance first – `len(txns) + 1` entries.

```
account_history(start: int, txns: list[int]) -> list[int]
```

```
>>> account_history(100, [-20, 50])
[100, 80, 130]
>>> account_history(0, [])
[0]
>>> account_history(5, [-10])
[5, -5]
```

Solution.

```
account_history = lambda start, txns: scanl(add, start, txns)
```

Note. `scanl`'s seed appears as the first output, which models the opening balance exactly; the `n+1` length is the contract, not a bug.

6.3 Suffix sums

drill

Using `scanr`, compute suffix sums: element `i` is the sum of `xs[i:]`, and the final element is the seed 0 (the empty suffix). `[1, 2, 3]` becomes `[6, 5, 3, 0]`.

```
suffix_sums(xs: list[int]) -> list[int]
```

```
>>> suffix_sums([1, 2, 3])
[6, 5, 3, 0]
>>> suffix_sums([])
[0]
>>> suffix_sums([5])
[5, 0]
```

Solution.

```
suffix_sums = lambda xs: scanr(add, 0, xs)
```

Note. `scanr` folds from the right and keeps every suffix's result; the whole-list answer lands first. Mirror image of prefix sums.

6.4 Best price still ahead

drill

Shopping day: `prices[i]` is today's price at shop `i`, and you may only buy at the CURRENT shop or a later one. Using `scanr1`, compute for each position the cheapest price from there to the end of the street.

```
best_remaining(prices: list[int]) -> list[int]
```

```
>>> best_remaining([5, 3, 8, 2, 4])
[2, 2, 2, 2, 4]
>>> best_remaining([7])
[7]
>>> best_remaining([4, 6])
[4, 6]
```

Solution.

```
best_remaining = lambda prices: scanr1(min, prices)
```

Note. A right-to-left running minimum: `scanr1(min)`. Pairing it with a left scan is the shape of many buy/sell problems.

6.5 Running product

drill

Using `accumulate` with a custom combiner, produce the running product of a list: `[2, 3, 4]` becomes `[2, 6, 24]`. Remember `accumulate` is a generator – deliver a list.

```
running_product(xs: list[int]) -> list[int]
```

```
>>> running_product([2, 3, 4])
[2, 6, 24]
>>> running_product([])
[]
>>> running_product([5])
[5]
```

Solution.

```
running_product = lambda xs: list(accumulate(xs, mul))
```

Note. `accumulate` defaults to addition; hand it `*` for products. It yields lazily, so `list()` materialises – forgetting that is the usual slip.

6.6 Record-breaking days

apply

A weather station logs one temperature per day. A day sets a RECORD if it is strictly hotter than every previous day; the first day always counts. Count the record days. Hint: on the running-maximum scan, records are exactly where consecutive entries strictly rise – pairwise makes those visible.

```
new_records(temps: list[int]) -> int
```

```
>>> new_records([3, 1, 4, 4, 5])
3
>>> new_records([2, 2, 2])
1
>>> new_records([1, 2, 3])
3
>>> new_records([])
0
```

Solution.

```
def new_records(temps):
    if temps == []:
        return 0
    return 1 + len([1 for a, b in pairwise(scanl1(max, temps)) if b > a])
```

Note. `scanl1(max)` turns history into a staircase; each strict step up is a record, plus one for day zero. Watch the tie case: equalling the record is not breaking it.

6.7 Range-sum service

apply

A dashboard must answer many sum queries over the same fixed list: each query (i, j) asks for the sum of $xs[i:j]$. Precompute prefix sums ONCE with `scanl`, then answer every query in $O(1)$ as $pre[j] - pre[i]$.

```
range_sums(xs: list[int], queries: list[tuple[int, int]]) -> list[int]
```

```
>>> range_sums([3, 1, 4, 1], [(0, 2), (1, 3)])
[4, 5]
>>> range_sums([2, 2, 2], [(0, 3)])
[6]
>>> range_sums([1, 2], [])
[]
>>> range_sums([5], [(0, 0)])
[0]
```

Solution.

```
def range_sums(xs, queries):
    pre = scanl(add, 0, xs)
    return map_(lambda q: pre[q[1]] - pre[q[0]], queries)
```

Note. One $O(n)$ scan converts every window question into two array reads. The seed 0 makes $pre[i]$ the sum of the first i elements, so half-open (i, j) needs no off-by-one fixups.

6.8 Fuel gauge report

apply

A rover starts with 'start' units of fuel and applies signed fuel deltas (burns and top-ups) in order. Report (lowest_level_ever, mission_ok) where mission_ok means the level – including the starting level – never went negative. Scan the history once, then read both answers off it.

```
fuel_report(start: int, deltas: list[int]) -> tuple[int, bool]
```

```
>>> fuel_report(10, [-4, -5, 3, -6])
(-2, False)
>>> fuel_report(5, [-1])
(4, True)
>>> fuel_report(0, [])
(0, True)
```

Solution.

```
def fuel_report(start, deltas):
    history = scanl(add, start, deltas)
    return (min(history), min(history) >= 0)
```

Note. The scan is the whole flight recorder; min answers both questions. A bare fold would need a compound accumulator to remember the low-water mark.

6.9 Maximum drawdown

apply

Risk teams measure a stock's worst fall from a prior peak: the maximum of `peak_so_far` - price over all days. Compute it by zipping the running maximum against the prices themselves; a never-falling series has drawdown 0.

```
drawdown(prices: list[int]) -> int
```

```
>>> drawdown([5, 3, 8, 4, 7])
4
>>> drawdown([1, 2, 3])
0
>>> drawdown([10])
0
>>> drawdown([])
0
```

Solution.

```
def drawdown(prices):
```

```

    if prices == []:
        return 0
    return max(zipWith(sub, scanl1(max, prices), prices))

```

Note. `scanl1(max)` is the peak line; `zipWith` subtracts the actual price under it. The mirror of best-time-to-sell: there you chase rises off the running MIN.

6.10 Peak concurrency

apply

Sessions occupy overlapping intervals of time; the log records them as +1 for each opening and -1 for each closing, in time order. Report the highest number of simultaneously open sessions; with no events the answer is 0. Scan the deltas from 0 and take the highest point.

```
peak_concurrency(events: list[int]) -> int
```

```

>>> peak_concurrency([1, 1, -1, 1, -1])
2
>>> peak_concurrency([1, -1, 1, -1])
1
>>> peak_concurrency([])
0

```

Solution.

```
peak_concurrency = lambda events: max(scanl(add, 0, events))
```

Note. The seed 0 doubles as the empty-log answer since `scanl` always emits it. This is the sweep-line pattern in miniature: deltas, running level, extremum.

6.11 Trapped rainwater

challenge

After a storm, water sits on a skyline of bar heights: above each bar, water fills up to the LOWER of the tallest bar to its left and the tallest to its right (itself included), minus the bar's own height. Total the trapped water. Two scans – a running max from the left and one from the right – plus one `zipWith3` finish it in $O(n)$. The classic interview problem, solved the scan way.

```
trapped(heights: list[int]) -> int
```

```

>>> trapped([0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1])
6
>>> trapped([2, 0, 2])
2
>>> trapped([1, 2, 3])
0
>>> trapped([])
0

```

Solution.

```

def trapped(heights):
    if heights == []:
        return 0
    lmax, rmax = scanl1(max, heights), scanr1(max, heights)
    return sum(zipWith3(lambda l, r, h: min(l, r) - h, lmax, rmax, heights))

```

Note. LeetCode 42 without a single index: left-max scan, right-max scan, pointwise $\min(l, r) - h$. Each term is ≥ 0 because each scan includes the bar itself.

6.12 Best sightseeing pair

challenge

A road trip visits spots with scores. Choosing spots $i < j$ earns $\text{score}[i] + \text{score}[j] + i - j$ (distance decays the fun). Maximise over all pairs, in $O(n)$: note the formula splits into $(\text{score}[i] + i) + (\text{score}[j] - j)$, so a running maximum of the first part, one step behind, meets each j . At least two spots are guaranteed.

```
best_pair(scores: list[int]) -> int
```

```
>>> best_pair([8, 1, 5, 2, 6])
```

```
11
>>> best_pair([1, 2])
2
>>> best_pair([1, 1, 1])
1
```

Solution.

```
def best_pair(scores):
    gain = scanl1(max, map_(lambda p: p[1] + p[0], enum(scores)))
    drop = map_(lambda p: p[1] - p[0], enum(scores))
    return max(zipWith(add, init(gain), tail(drop)))
```

Note. LeetCode 1014: split the pair formula, scan the left part's running max, zip it one position behind the right part. `init/tail` do the $i < j$ offset with no index fiddling.

Chapter 7

Streams and Unfolds

Production instead of consumption. Generators let a sequence be infinite – `count`, `cycle`, `iterate` – because nothing is computed until a consumer asks; `unfoldr` is the finite twin, growing a list from a seed until the step function says stop. Anything that PEELS (digits, chunks, trajectories, parent chains) is an unfold, and recognising that retires a whole family of while-loops with threaded state.

7.1 Concepts

7.1.1 `count`

the infinite arithmetic sequence `start, start+step, ...` Never consume it whole: pair it with `take`, `takewhile` or `find`, which stop.

```
>>> take(3, count(10))
[10, 11, 12]
>>> take(3, count(0, 5))
[0, 5, 10]
```

Roll your own — the definition:

```
def count(start=0, step=1): # [start, start+step ..]
    n = start
    while True:
        yield n
        n += step
```

Used before it is rolled: `take` in §8.1.2 (p. 63).

7.1.2 `repeat`

one value forever, or exactly `n` times with the second argument.

```
>>> take(2, repeat('x'))
['x', 'x']
>>> list(repeat('x', 3))
['x', 'x', 'x']
>>> list(repeat('x', 0))
[]
```

Roll your own — the definition:

```
def repeat(x, n=None): # yield x forever, or n times if given
    if n is None:
        while True:
            yield x
    else:
        for _ in range(n):
            yield x
```

Used before it is rolled: *take* in §8.1.2 (p. 63).

7.1.3 cycle

a finite list looped forever; an EMPTY list raises immediately (Haskell errors too – the alternative is a silent infinite hang).

```
>>> take(5, cycle([1, 2]))
[1, 2, 1, 2, 1]
```

Roll your own — the definition:

```
def cycle(xs): # repeat xs forever
    xs = list(xs)
    if not xs:
        raise ValueError("cycle: empty list") # Haskell errors too; the alternative is a silent hang
    while True:
        yield from xs
```

Used before it is rolled: *take* in §8.1.2 (p. 63).

7.1.4 iterate

$x, f(x), f(f(x)), \dots$ forever: repeated application as a stream. Any deterministic state machine becomes iterate on its step function – Fibonacci is iterate on a pair.

```
>>> take(4, iterate(lambda x: 2 * x, 1))
[1, 2, 4, 8]
>>> step = lambda p: (snd(p), fst(p) + snd(p))
>>> map_(fst, take(6, iterate(step, (0, 1))))
[0, 1, 1, 2, 3, 5]
```

Roll your own — the definition:

```
def iterate(f, x): # iterate f x = [x, f x, f (f x), ...]
    while True:
        yield x
        x = f(x)
```

Used before it is rolled: *take* in §8.1.2 (p. 63).

7.1.5 unfoldr

iterate's FINITE twin, and the cure for threading (state, acc) tuples through loops. You supply $f(\text{seed})$ returning either (value, seed') to emit a value and continue, or None to stop; unfoldr collects the values. Anything that "peels" a structure – digits, chunks, parent chains – is an unfoldr.

```
>>> unfoldr(lambda m: None if m == 0 else (m % 10, m // 10), 407)
[7, 0, 4]
>>> unfoldr(lambda n: None if n == 0 else (n, n - 1), 5)
[5, 4, 3, 2, 1]
>>> unfoldr(const(None), "seed") # stop immediately: empty
[]
```

How it works: the step function carries a two-outcome contract – $f(\text{seed})$ returns (value, next_seed) to emit and continue, or None to stop – and unfoldr collects what was emitted. It is the fold REVERSED: a fold consumes a list into a summary; an unfold grows a list out of a seed. (Category theory says anamorphism; interviewers say "generate the sequence".)

Why it earns its place: it retires the (state, accumulator) tuple you would otherwise thread through a while loop. Anything that PEELS – digits off a number, chunks off a list, parents up a tree, a Collatz trajectory – states its entire logic in the step function.

```
>>> unfoldr(lambda n: None if n == 1 else (n, n // 2 if even(n) else 3 * n + 1), 6)
[6, 3, 10, 5, 16, 8, 4, 2]
```

Versus iterate: iterate is the infinite cousin – an unfold whose step never says None, consumed with take. Reach for iterate when the stream is conceptually endless, unfoldr when the seed itself knows where the end is.

Remember it as: fold tears down; unfold builds up; the seed carries the future.

More unfolds, same two-outcome contract – peeling chunks off a list, bits off a number, walking a parent chain to its root:

```
>>> unfoldr(lambda xs: None if xs == [] else (xs[:2], xs[2:]), [1, 2, 3, 4, 5])
[[1, 2], [3, 4], [5]]
>>> unfoldr(lambda m: None if m == 0 else (m % 2, m // 2), 13)          # bits, LSB first
[1, 0, 1, 1]
>>> parents = {'c': 'b', 'b': 'a', 'a': None}
>>> unfoldr(lambda n: None if n is None else (n, parents[n]), 'c')      # ancestry
['c', 'b', 'a']
```

And the seed can carry several facts at once – here (current, next, remaining) generates Fibonacci with its own countdown built in:

```
>>> unfoldr(lambda s: None if s[2] == 0 else (s[0], (s[1], s[0] + s[1], s[2] - 1)), (0, 1, 8))
[0, 1, 1, 2, 3, 5, 8, 13]
```

Roll your own: prime the pump with `step = f(seed)`, then loop while `step` is not `None` – append the value, feed the new seed back to `f`. The two-outcome contract lives entirely in `f`; `unfoldr` is just the loop that honours it, plus the list that collects what was emitted.

```
def unfoldr(f, seed): # Data.List unfoldr: f(seed) -> None | (value, seed')
    out = [] # iterate's finite twin: grows a list until f says stop --
    step = f(seed) # no more threading (state, acc) tuples through until
    while step is not None:
        val, seed = step
        out.append(val)
        step = f(seed)
    return out
```

Used before it is rolled: *until* in §15.1.2 (p. 126), *const* in §16.1.2 (p. 133).

7.1.6 chain

concatenate iterables lazily, one after another: a generator expression, so the tail is never touched until asked for (`chain over count()` is fine). `concat` is the strict, list-building cousin.

```
>>> list(chain([1], [2, 3]))
[1, 2, 3]
>>> take(4, chain([1, 2], count(10)))
[1, 2, 10, 11]
```

Roll your own — the definition:

```
chain = lambda *iterables: (x for it in iterables for x in it) # concat, lazily
```

Used before it is rolled: *take* in §8.1.2 (p. 63).

7.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

7.1 First multiples

drill

Return the first `n` positive multiples of `k`, in order. Build the infinite stream of multiples with `count` and consume it with `take` – no arithmetic on indices.

```
multiples(k: int, n: int) -> list[int]
```

```
>>> multiples(3, 4)
[3, 6, 9, 12]
>>> multiples(5, 1)
[5]
>>> multiples(2, 0)
```

```
| []
```

Solution.

```
| multiples = lambda k, n: take(n, count(k, k))
```

Note. `count(k, k)` is the infinite stream `k, 2k, 3k, ...`; `take` stops it. $O(n)$. Common mistake: reaching for range arithmetic when the stream says it directly.

Rolled later: `take` in §8.1.2 (p. 63).

7.2 Banner line

drill

Build a banner string of `n` copies of a single character using `repeat` (the bounded, second-argument form) and `join`.

```
banner(ch: str, n: int) -> str
```

```
>>> banner('*', 3)
'***'
>>> banner('-', 0)
''
>>> banner('=', 5)
'====='
```

Solution.

```
| banner = lambda ch, n: "".join(repeat(ch, n))
```

Note. `repeat(x, n)` is the finite form; `join` materialises it. $O(n)$. Same result as `ch * n` – the point is knowing `repeat`’s two modes.

7.3 Powers table

drill

Return the first `n` powers of `b` starting at `b**0`, using `iterate` – repeated multiplication as a stream, no `**` operator.

```
powers(b: int, n: int) -> list[int]
```

```
>>> powers(2, 5)
[1, 2, 4, 8, 16]
>>> powers(10, 3)
[1, 10, 100]
>>> powers(3, 0)
[]
```

Solution.

```
| powers = lambda b, n: take(n, iterate(lambda x: x * b, 1))
```

Note. `iterate(f, 1)` is `1, b, b^2, ...` forever; `take` cuts it. $O(n)$ multiplications.

Rolled later: `take` in §8.1.2 (p. 63).

7.4 Countdown

drill

Return `[n, n-1, ..., 1]` with `unfoldr`: the seed is the current number, the `unfold` stops at zero. `countdown(0)` is empty.

```
countdown(n: int) -> list[int]
```

```
>>> countdown(4)
[4, 3, 2, 1]
>>> countdown(1)
[1]
>>> countdown(0)
[]
```

Solution.

```
| countdown = lambda n: unfoldr(lambda m: None if m == 0 else (m, m - 1), n)
```

Note. The two-outcome contract in miniature: emit `(m, m-1)` or stop with `None`. $O(n)$.

7.5 Splice three feeds

drill

Concatenate three iterables into one list using chain – lists, strings, anything iterable, in the order given.

`splice(a, b, c) -> list`

```
>>> splice([1], [2, 3], [])
[1, 2, 3]
>>> splice('ab', 'cd', 'e')
['a', 'b', 'c', 'd', 'e']
>>> splice([], [], [])
[]
```

Solution.

```
splice = lambda a, b, c: list(chain(a, b, c))
```

Note. chain is lazy concatenation; list() drains it. O(total length).

7.6 Round-robin card deal

apply

A dealer hands cards to players in strict rotation: first card to the first player, second to the second, wrapping around until the cards run out. Return the (player, card) pairs in deal order. At least one player is guaranteed.

`deal(players: list[str], cards: list) -> list[tuple]`

```
>>> deal(['ann', 'bo'], [1, 2, 3, 4, 5])
[('ann', 1), ('bo', 2), ('ann', 3), ('bo', 4), ('ann', 5)]
>>> deal(['solo'], ['x', 'y'])
[('solo', 'x'), ('solo', 'y')]
>>> deal(['a', 'b'], [])
[]
```

Solution.

```
deal = lambda players, cards: zip_(cycle(players), cards)
```

Note. cycle makes the rotation infinite; zip_ stops at the shorter side – the cards. O(cards). Common mistake: index arithmetic with % when cycle already says it.

7.7 Colony doubling

apply

A bacteria colony starts at 'start' cells and doubles every day. Return how many days pass before the colony first reaches at least 'goal' cells (0 if it already has). Model the growth as an iterate stream and count the days spent below goal.

`days_to_reach(start: int, goal: int) -> int`

```
>>> days_to_reach(1, 8)
3
>>> days_to_reach(5, 5)
0
>>> days_to_reach(3, 100)
6
```

Solution.

```
days_to_reach = lambda start, goal: len(
    takeWhile(lambda x: x < goal, iterate(lambda x: 2 * x, start)))
```

Note. The stream is start, 2*start, ...; the answer is how many terms stay under goal. O(log(goal/start)). Watch the >= boundary: day counting is off by one if you use <=.

Rolled later: takeWhile in §8.1.6 (p. 64).

7.8 LED bit pattern

apply

An LED row shows a number in binary, most significant bit first. Return the bit list for n >= 1: peel bits least-significant-first with unfoldr, then reverse.

```
bits(n: int) -> list[int]
```

```
>>> bits(13)
[1, 1, 0, 1]
>>> bits(1)
[1]
>>> bits(8)
[1, 0, 0, 0]
```

Solution.

```
bits = lambda n: list(reversed(unfoldr(lambda m: None if m == 0 else (m % 2, m // 2), n)))
```

Note. unfoldr naturally emits LSB first ($m \% 2$ peels the low bit); one reversal fixes the display order. $O(\log n)$.

7.9 Chain of command

apply

An org chart maps each employee to their manager, with the top boss mapped to None. Return the chain of command from a given employee up to and including the top: an unfold whose seed is the current person and whose next seed is their manager.

```
chain_of_command(boss: dict, name: str) -> list[str]
```

```
>>> boss = {'amy': 'raj', 'raj': 'kim', 'kim': None}
>>> chain_of_command(boss, 'amy')
['amy', 'raj', 'kim']
>>> chain_of_command(boss, 'kim')
['kim']
>>> chain_of_command({'x': None}, 'x')
['x']
```

Solution.

```
chain_of_command = lambda boss, name: unfoldr(
    lambda n: None if n is None else (n, boss[n]), name)
```

Note. A parent-chain walk is an unfold: emit the person, seed becomes the manager, None stops. $O(\text{depth})$. The same shape reconstructs paths in Dijkstra.

7.10 When the buses meet

apply

Bus A stops at minute a , $a+da$, $a+2*da$, ...; bus B at b , $b+db$, Return the first minute at or after both start times when the two schedules coincide. A shared minute is guaranteed to exist. Search A's infinite schedule with find.

```
sync(a: int, da: int, b: int, db: int) -> int
```

```
>>> sync(3, 4, 1, 6)
7
>>> sync(10, 1, 4, 7)
11
>>> sync(0, 5, 0, 3)
0
```

Solution.

```
sync = lambda a, da, b, db: find(lambda x: x >= b and (x - b) % db == 0, count(a, da))
```

Note. find is the only consumer that can search an infinite count stream: it stops at the first hit. The membership test for B's schedule is arithmetic, not a second stream.

7.11 Look-and-say

challenge

The look-and-say sequence starts '1'; each term reads the previous aloud: '1' is one 1 -> '11'; '11' is two 1s -> '21'; '21' -> '1211'; and so on. Return the n th term (1-indexed). Peel each term into runs of equal digits with unfoldr, encode each run as count then digit, and drive the whole sequence with iterate.

```
look_and_say(n: int) -> str
```

```
>>> look_and_say(1)
```

```
'1'
>>> look_and_say(2)
'11'
>>> look_and_say(3)
'21'
>>> look_and_say(5)
'111221'
```

Solution.

```
def look_and_say(n):
    def peel(u):
        if u == "":
            return None
        i = fromMaybe(len(u), find(lambda j: u[j] != u[0], range(1, len(u))))
        return (u[:i], u[i:])
    step = lambda s: "".join(map_(lambda r: str(len(r)) + r[0], unfoldr(peel, s)))
    return take(n, iterate(step, "1"))[-1]
```

Note. Two unfolds stacked: unfoldr peels one term into runs (find locates where the run ends, fromMaybe handles the final run), and iterate unfolds the sequence of terms. $O(n * \text{term length})$. Common mistake: forgetting the last run when no differing digit exists.

Rolled later: take in §8.1.2 (p. 63).

7.12 Hailstone champion

challenge

The hailstone (Collatz) trajectory of n repeatedly halves even numbers and maps odd m to $3*m + 1$, ending at 1. Among all starts $1..limit$, return the start with the LONGEST trajectory (counting every term including the final 1); ties go to the smaller start. Express one trajectory as an unfoldr and measure it.

`hail_champion(limit: int) -> int`

```
>>> hail_champion(1)
1
>>> hail_champion(10)
9
>>> hail_champion(30)
27
```

Solution.

```
def hail_champion(limit):
    step = lambda m: None if m == 1 else (m, m // 2 if even(m) else 3 * m + 1)
    length = lambda n: len(unfoldr(step, n)) + 1
    return max(range(1, limit + 1), key=length)
```

Note. unfoldr emits every term before 1, so trajectory length is $\text{len} + 1$. max with key= keeps the FIRST winner on ties, and range ascends, so ties resolve to the smaller start for free. Interview follow-up: memoize lengths across starts with memo.

Chapter 8

Spans and Views

The consumers and finite views that make streams usable: **take** and friends stop lazily; **span/break_** cut a sequence at a condition – a lexer in one call; **inits/tails** enumerate every prefix and suffix, which is where substring problems begin; and **chunksOf** pages anything. Small tools, constant use.

8.1 Concepts

8.1.1 **islice**

at most the first *n* items of ANY iterable, lazily; the safe window onto an infinite stream. Runs short without complaint.

```
>>> list(islice(count(0), 3))
[0, 1, 2]
>>> list(islice([1, 2], 5))
[1, 2]
```

Roll your own — the definition:

```
def islice(iterable, stop): # take
    it = iter(iterable)
    for _ in range(stop):
        try:
            yield next(it)
        except StopIteration:
            return
```

*Used before it is rolled: **take** in §8.1.2 (p. 63).*

8.1.2 **take**

the first *n*, as a real list. **take/drop** are the positional pair; **takeWhile/dropWhile** are the conditional pair.

```
>>> take(3, count(1))
[1, 2, 3]
>>> take(2, [7, 8, 9])
[7, 8]
```

Roll your own — the definition:

```
take = lambda n, xs: list(islice(iter(xs), n)) # works on infinite streams
```

*Used before it is rolled: **on** in §16.1.7 (p. 135).*

8.1.3 **takewhile**

yield elements WHILE the predicate holds, then stop at the first failure (a generator). Contrast **filter_**: **filter_** scans everything; **takewhile** stops – on [2, 4, 5, 6], **filter_** keeps the 6, **takewhile** does not.

```
>>> list(takewhile(even, [2, 4, 5, 6]))
[2, 4]
>>> filter_(even, [2, 4, 5, 6])
[2, 4, 6]
```

Roll your own — the definition:

```
def takewhile(cond, xs): # generator: yield the leading run where cond holds
    for x in xs:
        if not cond(x):
            return
        yield x
```

8.1.4 takeuntil

like `takewhile(not p)`, but **INCLUDES** the first element where `p` holds – the do-while flavour: "read up to and including the terminator".

```
>>> list(takeuntil(lambda x: x == 3, [1, 2, 3, 4]))
[1, 2, 3]
>>> list(takeuntil(lambda x: x > 0, [5, 6]))
[5]
```

Roll your own — the definition:

```
def takeuntil(cond, xs): # like takewhile(not . cond), but INCLUDES the first hit
    for x in xs:
        yield x
        if cond(x):
            return
```

8.1.5 dropwhile

discard while the predicate holds, then yield **EVERYTHING** after the first failure, no more testing. `takewhile`'s complement.

```
>>> list(dropwhile(lambda x: x < 3, [1, 2, 3, 1]))
[3, 1]
```

Roll your own — the definition:

```
def dropwhile(cond, xs): # generator: drop the leading run where cond holds
    it = iter(xs)
    for x in it:
        if not cond(x):
            yield x
            break
    yield from it
```

8.1.6 takeWhile

`takewhile`, delivered as a list (capital W = list version).

```
>>> takeWhile(lambda x: x < 3, [1, 2, 3, 1])
[1, 2]
```

Roll your own — the definition:

```
takeWhile = lambda cond, xs: list(takewhile(cond, xs)) # take the leading run where cond holds, as a list
```

8.1.7 dropWhile

`dropwhile`, delivered as a list.

```
>>> dropWhile(lambda x: x < 3, [1, 2, 3, 1])
[3, 1]
```

Roll your own — the definition:

```
dropWhile = lambda cond, xs: list(dropwhile(cond, xs)) # drop the leading run where cond holds, as a list
```

8.1.8 span

split at the first failure: (longest satisfying prefix, the rest), computed in ONE pass so it is safe on one-shot iterators (the old two-scan version silently broke on generators – this is the fix). A lexer in one call.

```
>>> span(lambda x: x < 3, [1, 2, 3, 4, 1])
([1, 2], [3, 4, 1])
>>> span(str.isdigit, "123abc")
(['1', '2', '3'], ['a', 'b', 'c'])
```

Roll your own: materialise xs once (that is the one-pass guarantee), locate the first failing index with next() over enumerate – defaulting to len(xs) when the predicate never fails – and slice twice. The previous two-scan version ran takewhile and dropwhile separately and silently broke on one-shot generators; this is the repair.

```
def span(cond, xs): # split at first failure, ONE pass; safe on one-shot iters
    xs = list(xs)
    i = next((i for i, x in enumerate(xs) if not cond(x)), len(xs))
    return (xs[:i], xs[i:])
```

Used before it is rolled: *on* in §16.1.7 (p. 135).

8.1.9 break_

span with the predicate negated: split where the condition first HOLDS.

```
>>> break_(lambda x: x > 2, [1, 2, 3, 1])
([1, 2], [3, 1])
```

Roll your own — the definition:

```
break_ = lambda cond, xs: span(compose(not_, cond), xs) # split at first hit (opposite of span)
```

Used before it is rolled: *compose* in §16.1.8 (p. 136).

8.1.10 inits

every prefix, from empty to the whole thing; len(xs)+1 of them.

```
>>> inits([1, 2])
[[], [1], [1, 2]]
>>> inits("ab")
['', 'a', 'ab']
```

Roll your own — the definition:

```
inits = lambda xs: [xs[:i] for i in range(len(xs) + 1)] # Data.List: every prefix, [] first
```

8.1.11 tails

every suffix, whole thing first, down to empty. Substring problems start here: every substring is a prefix of some suffix.

```
>>> tails("abc")
['abc', 'bc', 'c', '']
>>> tails([1, 2])
[[1, 2], [2], []]
```

Roll your own — the definition:

```
tails = lambda xs: [xs[i:] for i in range(len(xs) + 1)] # every suffix; substrings start here
```

8.1.12 chunksOf

consecutive pieces of length n ; the last may be short. Works on lists and strings alike since it is pure slicing.

```
>>> chunksOf(2, [1, 2, 3, 4, 5])
[[1, 2], [3, 4], [5]]
>>> chunksOf(3, "ABCDEFG")
['abc', 'def', 'g']
```

Roll your own — the definition:

```
chunksOf = lambda n, xs: [xs[i:i + n] for i in range(0, len(xs), n)] # Data.List.Split; short last ok
```

8.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

8.1 Leading run

drill

Return the longest prefix of `xs` whose elements are all strictly positive, using `span`. The rest of the list is discarded.

```
leading_positive(xs: list) -> list
```

```
>>> leading_positive([2, 5, 1, -1, 9])
[2, 5, 1]
>>> leading_positive([-1, 2])
[]
>>> leading_positive([4, 4])
[4, 4]
```

Solution.

```
leading_positive = lambda xs: span(lambda x: x > 0, xs)[0]
```

Note. `span` returns (prefix, rest) in one pass; take the first component. $O(\text{prefix length})$.

8.2 Peek first k

drill

Return the first k elements of a possibly-infinite stream with `take`. If the stream is shorter than k , return all of it — `take` never complains.

```
peek(k: int, xs) -> list
```

```
>>> peek(3, count(0))
[0, 1, 2]
>>> peek(5, [1, 2])
[1, 2]
>>> peek(0, count(9))
[]
```

Solution.

```
peek = lambda k, xs: take(k, xs)
```

Note. `take` is the safe window onto an infinite stream; `islice` underlies it. $O(k)$.

8.3 Skip the header

drill

A list begins with comment lines starting `'#'`. Drop that leading block and return the remaining lines, using `dropWhile`. Once a non-comment line appears, keep everything.

```
skip_header(lines: list) -> list
```

```
>>> skip_header(['# a', '# b', 'x', '# c'])
['x', '# c']
```

```
>>> skip_header(['data'])
['data']
>>> skip_header(['# only'])
[]
```

Solution.

```
skip_header = lambda lines: dropWhile(lambda s: s.startswith('#'), lines)
```

Note. dropWhile discards while true, then keeps the rest untested – note '#' survives. $O(n)$.

Rolled later: *lines* in §9.1.11 (p. 74).

8.4 Every prefix

drill

Return every prefix of *xs* from empty to the whole list, using *inits*.

```
prefixes(xs: list) -> list[list]
```

```
>>> prefixes([1, 2, 3])
[[], [1], [1, 2], [1, 2, 3]]
>>> prefixes([])
[[]]
>>> prefixes(['a'])
[[], ['a']]
```

Solution.

```
prefixes = lambda xs: inits(xs)
```

Note. *inits* gives $\text{len}(xs)+1$ prefixes, empty first. $O(n^2)$ total size.

8.5 Deal into pages

drill

Split a list into consecutive pages of size *k*; the final page may be short. Use *chunksOf*.

```
paginate(k: int, xs: list) -> list[list]
```

```
>>> paginate(2, [1, 2, 3, 4, 5])
[[1, 2], [3, 4], [5]]
>>> paginate(3, [1, 2, 3])
[[1, 2, 3]]
>>> paginate(2, [])
[]
```

Solution.

```
paginate = lambda k, xs: chunksOf(k, xs)
```

Note. *chunksOf* is pure slicing, so it works on lists and strings alike. $O(n)$.

8.6 Read the leading integer

apply

A token string begins with an optional run of digits, then arbitrary characters. Return the leading integer, or 0 if no digit starts it. Use *span* to split the digit run from the tail, then interpret it.

```
read_int(s: str) -> int
```

```
>>> read_int('123abc')
123
>>> read_int('7')
7
>>> read_int('abc')
0
>>> read_int('')
0
```

Solution.

```
| read_int = lambda s: int("".join(span(str.isdigit, s)[0]) or "0")
```

Note. `span(str.isdigit, s)[0]` is the digit prefix as a char list; join and int it, with 'or 0' for the empty case. This is exactly how a lexer peels one token. $O(\text{len})$.

8.7 Split on the first blank

apply

An email is a header block followed by a blank line (") and then the body. Return (headers, body) where headers excludes the blank separator and body is the lines after it. Use `break_` to cut at the first blank line. If there is no blank line, the body is empty.

```
split_email(lines: list) -> tuple
|>>> split_email(['To: a', 'Re: b', '', 'hello', 'bye'])
|(['To: a', 'Re: b'], ['hello', 'bye'])
|>>> split_email(['To: a', 'Re: b'])
|(['To: a', 'Re: b'], [])
|>>> split_email(['', 'body'])
|([], ['body'])
```

Solution.

```
| def split_email(lines):
|     head, rest = break_(lambda s: s == '', lines)
|     return (head, rest[1:])
```

Note. `break_` splits at the first line where the predicate holds – the blank – putting it at the front of rest, so `rest[1:]` drops the separator. $O(n)$.

Rolled later: lines in §9.1.11 (p. 74).

8.8 Read up to the sentinel

apply

A sensor emits readings terminated by a sentinel value -1. Return the readings up to AND including the first -1 (the do-while flavour: the terminator is part of the record). Use `takeuntil`. If no -1 appears, return the whole list.

```
read_record(xs: list) -> list
|>>> read_record([4, 7, -1, 9])
|[4, 7, -1]
|>>> read_record([1, 2, 3])
|[1, 2, 3]
|>>> read_record([-1, 5])
|[-1]
```

Solution.

```
| read_record = lambda xs: list(takeuntil(lambda x: x == -1, xs))
```

Note. `takeuntil` INCLUDES the first hit, unlike `takewhile` which would stop just before -1. $O(\text{record length})$. Common mistake: using `takewhile` and losing the sentinel.

8.9 Longest good prefix length

apply

Given a list of daily temperatures, a warm streak from day 0 runs while each day is at least 'floor'. Return how many opening days form that streak, using `span` to isolate the prefix and measuring it. A first day below floor gives 0.

```
warm_streak(temps: list, floor: int) -> int
|>>> warm_streak([70, 72, 68, 90], 70)
|2
|>>> warm_streak([60, 80], 70)
|0
|>>> warm_streak([75, 75, 75], 70)
|3
```

Solution.

```
warm_streak = lambda temps, floor: len(span(lambda t: t >= floor, temps)[0])
```

Note. span isolates the leading run, len measures it – one pass, no manual counter. O(streak).

8.10 All substrings

apply

Return every non-empty contiguous substring of s, in the order: by start position, then by length. The tool is tails – every substring is a non-empty prefix of some suffix. So take each suffix's inits, drop the empty one, and concatenate.

```
substrings(s: str) -> list[str]
```

```
>>> substrings('ab')
['a', 'ab', 'b']
>>> substrings('a')
['a']
>>> substrings('')
[]
>>> substrings('xyz')
['x', 'xy', 'xyz', 'y', 'yz', 'z']
```

Solution.

```
substrings = lambda s: concatMap(
    lambda suf: map_(lambda p: "".join(p), inits(list(suf))[1:]),
    takeWhile(lambda suf: suf != "", tails(s)))
```

Note. tails gives every suffix (dropping the empty tail); each suffix's inits[1:] are its non-empty prefixes – together, every substring. O(n²) substrings, the true size.

8.11 Run-length split

challenge

Split a list into maximal runs of equal consecutive elements, WITHOUT using group or groupBy – do it directly with span. Peel the leading run (all elements equal to the head) off the front repeatedly until the list is empty.

```
runs(xs: list) -> list[list]
```

```
>>> runs([1, 1, 2, 3, 3, 3])
[[1, 1], [2], [3, 3, 3]]
>>> runs([])
[]
>>> runs(['a'])
[['a']]
>>> runs([5, 5, 5])
[[5, 5, 5]]
```

Solution.

```
def runs(xs):
    peel = lambda ys: None if ys == [] else span(lambda x: x == ys[0], ys)
    return unfoldr(peel, list(xs))
```

Note. span(equal-to-head) is exactly one run; unfoldr repeats the peel until empty. This is how group is built. O(n). Common mistake: comparing to the previous element instead of the run's head.

8.12 Justify to width

challenge

Greedily wrap words into lines no longer than 'width' characters, words joined by single spaces; each line packs as many words as fit. Every word is guaranteed to fit alone. Peel one line at a time: take the longest prefix of the remaining words whose joined length stays within width, emit it, continue with the rest.

```
justify(words: list, width: int) -> list[str]
```

```
>>> justify(['a', 'bb', 'ccc'], 6)
['a bb', 'ccc']
>>> justify(['one', 'two', 'three'], 7)
['one two', 'three']
```

```
>>> justify([], 5)
[]
>>> justify(['hello'], 5)
['hello']
```

Solution.

```
def justify(words, width):
    def peel(ws):
        if ws == []:
            return None
        fits = lambda k: len(unwords(ws[:k])) <= width
        k = last(takeWhile(fits, range(1, len(ws) + 1)))
        return (unwords(ws[:k]), ws[k:])
    return unfoldr(peel, list(words))
```

Note. unfoldr peels one line per step; takeWhile(fits, count(1)) finds the largest word count that fits, and last takes it. O(total characters). Every word fitting alone guarantees $k \geq 1$, so the unfold always makes progress.

Rolled later: *words* in §9.1.9 (p. 73), *unwords* in §9.1.10 (p. 74).

Chapter 9

Order and Text

“Does order unlock it?” is the recognition ladder’s first question because sorting linearises problems: tuple keys buy multi-level orderings in one call, `minOn/maxOn` answer best-by-key in $O(n)$, and the `bisect` pair reads a sorted list’s True/False boundary three different ways – membership, insertion point, counting. The string quartet turns text pipelines into list pipelines and back.

9.1 Concepts

9.1.1 `sortOn`

sort by a key function, evaluated once per element. TUPLE KEYS are the power move: (primary, secondary) sorts two levels at once, and negating a numeric component flips just that level – (-count, word) is "count descending, ties alphabetical", which `reverse=True` cannot express.

```
>>> sortOn(len, ["bbb", "a", "cc"])
['a', 'cc', 'bbb']
>>> sortOn(lambda kv: (-kv[1], kv[0]), [('b', 2), ('a', 2), ('c', 3)])
[('c', 3), ('a', 2), ('b', 2)]
```

Roll your own — the definition:

```
sortOn = lambda f, xs: sorted(xs, key=f) # sortOn (schwartzian, f called once per element)
```

9.1.2 `minOn`

the element with the smallest key, in $O(n)$. `sortOn + head` gives the same answer for $O(n \log n)$ – pay the sort only when you need ALL of them ordered.

```
>>> minOn(abs, [-3, 2, 5])
2
>>> minOn(snd, [('a', 9), ('b', 1)])
('b', 1)
```

Roll your own — the definition:

```
minOn = lambda f, xs: min(xs, key=f) # minimumBy (comparing f): sortOn + head without the sort
```

9.1.3 `maxOn`

the element with the largest key, $O(n)$; the FIRST winner on ties, so pre-sorting the input controls the tie-break.

```
>>> maxOn(len, ["aa", "b", "cc"])
'aa'
```

Roll your own — the definition:

```
maxOn = lambda f, xs: max(xs, key=f) # maximumBy (comparing f) in  $O(n)$ ; first wins ties
```

9.1.4 cmp_to_key

adapts an old-style comparator (returning negative, zero or positive) into the key= protocol. Reach for it only when a rule genuinely compares two elements against each other and no per-element key exists.

```
>>> sorted(["bb", "a"], key=cmp_to_key(lambda a, b: len(a) - len(b)))
['a', 'bb']
```

Roll your own: a tiny wrapper class implementing ONE dunder – `__lt__` defers to `cmp(a, b) < 0`. Python's sort only ever asks "is a less than b?", so that single method is enough to smuggle any comparator through the key= interface.

```
def cmp_to_key(cmp): # resurrect Python 2's cmp for sortBy
    class K:
        def __init__(self, x): self.x = x
        def __lt__(self, other): return cmp(self.x, other.x) < 0
    return K
```

Used before it is rolled: *sortBy* in §9.1.5 (p. 72).

9.1.5 sortBy

comparator sort, Python 2 style, built on `cmp_to_key`.

```
>>> sortBy(flip(sub), [1, 3, 2])
[3, 2, 1]
```

Roll your own — the definition:

```
sortBy = lambda cmp, xs: sorted(xs, key=cmp_to_key(cmp)) # comparator sort, Python 2 style
```

Used before it is rolled: *flip* in §16.1.3 (p. 133).

9.1.6 merge

join two ALREADY-SORTED lists into one sorted list, stably, in $O(n+m)$: the two-pointer loop at the heart of merge sort.

```
>>> merge([1, 4, 6], [2, 3, 7])
[1, 2, 3, 4, 6, 7]
>>> merge([], [1, 2])
[1, 2]
```

Roll your own: two cursors, always copying the smaller head forward; the `<=` (rather than `<`) is what makes the merge STABLE, preferring the left list on ties. When either side runs dry, glue both tails on – one of them is empty, so the `+` is harmless.

```
def merge(a, b): # merge two sorted lists, stable; the heart of merge sort
    out, i, j = [], 0, 0 # (section 13's merge_lists is this same loop on ListNodes)
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out + a[i:] + b[j:]
```

Used before it is rolled: *merge_lists* in §13.1.15 (p. 111), on in §16.1.7 (p. 135).

9.1.7 bisect_left

binary search over a SORTED list: the first index whose element is $\geq x$; equivalently, where x would insert to the left of its equals. If `a[i] != x` there, x is absent – that check is binary search.

```
>>> bisect_left([10, 20, 20, 30], 20)
1
>>> bisect_left([10, 20, 30], 25)      # insertion point for a missing value
2
```

How it works: on a sorted list the predicate "element < x" reads True for a prefix and False for the rest – TTTTFFFF. Binary search does not hunt for x; it finds that BOUNDARY, halving the interval per probe. `bisect_left` returns the first index $\geq x$, `bisect_right` the first $> x$, and from those two facts three idioms fall out.

```
>>> xs = [10, 20, 20, 30]
>>> bisect_left(xs, 20), bisect_right(xs, 20)    # the run of 20s occupies [1, 3)
(1, 3)
```

Idiom one, membership: x is present iff the slot `bisect_left` names actually holds it. Idiom two, insertion point: that same index is where x belongs to keep the list sorted. Idiom three, counting: right minus left counts occurrences – and with two different probes, the values inside any range.

```
>>> i = bisect_left(xs, 25)
>>> i, (i < len(xs) and xs[i] == 25)            # 25 would insert at 3; absent
(3, False)
```

Why it earns its place: every threshold ladder is secretly a sorted list. Grading bands, tax brackets, rate tiers – one `bisect` replaces the if/elif staircase:

```
>>> grade = lambda score: "FDCBA"[bisect_right([60, 70, 80, 90], score)]
>>> grade(59), grade(60), grade(95)
('F', 'D', 'A')
```

Remember it as: `bisect` finds the boundary in a sea of Trues and Falses; everything else is reading that boundary three ways.

Roll your own: the lo/hi halving loop over an invariant – everything left of lo is $< x$, everything from hi rightwards is $\geq x$. One comparison, `a[mid] < x`, decides which half survives; when lo meets hi, that meeting point IS the boundary. `bisect_right` is the identical loop with $\leq$ in place of $<$ – one character moves the boundary to the far side of the equals.

```
def bisect_left(a, x): # first index where a[i] >= x
    lo, hi = 0, len(a)
    while lo < hi:
        mid = halve(lo + hi)
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

Used before it is rolled: `bisect_right` in §9.1.8 (p. 73).

9.1.8 bisect_right

the first index whose element is $> x$: where x would insert to the RIGHT of its equals. right minus left counts the occurrences.

```
>>> bisect_right([10, 20, 20, 30], 20)
3
>>> xs = [10, 20, 20, 30]
>>> bisect_right(xs, 20) - bisect_left(xs, 20)
2
```

Roll your own — the definition:

```
def bisect_right(a, x): # first index where a[i] > x
    lo, hi = 0, len(a)
    while lo < hi:
        mid = halve(lo + hi)
        if a[mid] <= x:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

9.1.9 words

split on any whitespace; runs collapse, edges vanish. The inverse direction is `unwords`, and the round trip normalises spacing.

```
>>> words("  such  spacing ")
['such', 'spacing']
>>> unwords(words("  a  b "))
'a b'
```

Roll your own — the definition:

```
words = str.split # split on whitespace
```

Used before it is rolled: *unwords* in §9.1.10 (p. 74), on in §16.1.7 (p. 135).

9.1.10 unwords

join words with single spaces.

```
>>> unwords(['a', 'b', 'c'])
'a b c'
```

Roll your own — the definition:

```
unwords = " ".join # join words with single spaces
```

9.1.11 lines

split a string into its lines (no trailing newlines kept).

```
>>> lines("one\ntwo")
['one', 'two']
```

Roll your own — the definition:

```
lines = str.splitlines # split on newlines
```

Used before it is rolled: on in §16.1.7 (p. 135).

9.1.12 unlines

join lines, TERMINATING each with a newline (not separating – note the trailing one).

```
>>> unlines(['a', 'b'])
'a\nb\n'
```

Roll your own — the definition:

```
unlines = lambda ls: "".join(l + "\n" for l in ls) # join lines back with trailing newlines
```

9.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

9.1 Sort by size, then spelling

drill

Sort a list of words shortest-first; words of equal length appear in alphabetical order. One `sortOn` with a tuple key does both levels at once.

```
sort_by_size(ws: list[str]) -> list[str]

>>> sort_by_size(["bb", "a", "ccc", "aa"])
['a', 'aa', 'bb', 'ccc']
>>> sort_by_size(["same", "size", "abcd"])
['abcd', 'same', 'size']
>>> sort_by_size([])
[]
```

Solution.

```
| sort_by_size = lambda ws: sortOn(lambda w: (len(w), w), ws)
```

Note. Tuple keys sort level by level; $O(n \log n)$. Common mistake: two separate sorts, which works only because Python's sort is stable but hides the intent.

9.2 Closest to zero*drill*

Return the element of a non-empty list closest to zero. If two elements tie in distance (like -2 and 2), the one appearing FIRST in the list wins.

```
closest_to_zero(xs: list[int]) -> int
```

```
>>> closest_to_zero([3, -1, 4])
-1
>>> closest_to_zero([-2, 2])
-2
>>> closest_to_zero([7])
7
```

Solution.

```
| closest_to_zero = lambda xs: minOn(abs, xs)
```

Note. minOn is $O(n)$ and keeps the first winner on ties – no sort needed. Sorting first would cost $O(n \log n)$ and lose the arrival tie-break.

9.3 Count occurrences in sorted data*drill*

Given an ALREADY-SORTED list and a value, count how many times the value occurs – in $O(\log n)$, using the two bisections. No scanning allowed.

```
count_val(sorted_xs: list[int], x: int) -> int
```

```
>>> count_val([1, 2, 2, 2, 3], 2)
3
>>> count_val([1, 3], 2)
0
>>> count_val([], 5)
0
```

Solution.

```
| count_val = lambda xs, x: bisect_right(xs, x) - bisect_left(xs, x)
```

Note. bisect_left finds the first slot $\geq x$, bisect_right the first $> x$; the gap between them IS the run of equals. Two $O(\log n)$ probes, zero scanning.

9.4 Normalize spacing*drill*

Collapse any runs of whitespace in a sentence to single spaces and strip the ends, using the words/unwords round trip.

```
normalize_spaces(s: str) -> str
```

```
>>> normalize_spaces("  such  spacing  ")
'such spacing'
>>> normalize_spaces("one")
'one'
>>> normalize_spaces("")
''
```

Solution.

```
| normalize_spaces = lambda s: unwords(words(s))
```

Note. words splits on any whitespace runs; unwords rejoins with single spaces. The round trip IS the normalizer – no regex required.

9.5 Three-way merge

drill

Merge three already-sorted lists into one sorted list by chaining the two-list merge. Stability must hold: on equal values, earlier-argument elements come first.

```
merge3(a: list, b: list, c: list) -> list
```

```
>>> merge3([1, 4], [2, 5], [3, 6])
[1, 2, 3, 4, 5, 6]
>>> merge3([], [7], [])
[7]
>>> merge3([1, 1], [1], [0])
[0, 1, 1, 1]
```

Solution.

```
merge3 = lambda a, b, c: merge(merge(a, b), c)
```

Note. merge is associative, so nesting two calls gives a three-way merge in $O(n)$ total. This chaining is the seed of k-way merging.

9.6 Tournament leaderboard

apply

A tournament reports (name, score) pairs. Produce the leaderboard: names only, highest score first, and players with equal scores listed alphabetically. One key, two levels.

```
leaderboard(entries: list[tuple[str, int]]) -> list[str]
```

```
>>> leaderboard([("ann", 50), ("bob", 70), ("cy", 50)])
['bob', 'ann', 'cy']
>>> leaderboard([("zed", 10)])
['zed']
>>> leaderboard([])
[]
```

Solution.

```
leaderboard = lambda es: map(_fst, sortOn(lambda e: (-e[1], e[0]), es))
```

Note. The (-score, name) tuple key gets descending score and ascending name in one sort; reverse=True could not, since it would flip the names too.

9.7 Shipping classes

apply

Parcels are classed by weight with cutoffs at 1, 5 and 20 kilograms: under 1 is an 'envelope', then 'small', then 'medium', and 20 or over is 'freight' – a parcel exactly at a cutoff belongs to the higher class. Replace the if/elif ladder with one bisect over the cutoffs.

```
shipping_class(w: float) -> str
```

```
>>> shipping_class(0.5)
'envelope'
>>> shipping_class(1)
'small'
>>> shipping_class(4.9)
'small'
>>> shipping_class(20)
'freight'
```

Solution.

```
shipping_class = lambda w: ["envelope", "small", "medium", "freight"][bisect_right([1, 5, 20], w)]
```

Note. Every threshold ladder is a sorted list in disguise; bisect_right sends boundary values upward. Mistake to know: bisect_left would send them down.

9.8 Version sort

apply

Sort dotted version strings like '1.10' and '1.2' in true numeric order (so '1.2' precedes '1.10'). Shorter versions that are a prefix of longer ones come first.

```
sort_versions(vs: list[str]) -> list[str]

>>> sort_versions(["1.10", "1.2", "2.0"])
['1.2', '1.10', '2.0']
>>> sort_versions(["0.9", "0.10.1", "0.10"])
['0.9', '0.10', '0.10.1']
>>> sort_versions(["3"])
['3']
```

Solution.

```
| sort_versions = lambda vs: sortOn(lambda v: [int(p) for p in v.split(".")], vs)
```

Note. The key turns each version into a list of ints, and Python compares lists elementwise with shorter-prefix-first – exactly version semantics. Sorting the raw strings is the classic bug ('1.10' < '1.2' lexically).

9.9 Sort the score report

apply

A report arrives as newline-separated lines of 'name score'. Return the same report with lines ordered by score descending, ties broken by name ascending – rebuilt with unlines, so the result ends with a newline.

```
sort_report(text: str) -> str

>>> sort_report("ann 50\nbob 70")
'bob 70\nann 50\n'
>>> sort_report("cy 70\nbob 70\nann 50")
'bob 70\ncy 70\nann 50\n'
>>> sort_report("solo 1")
'solo 1\n'
```

Solution.

```
| sort_report = lambda text: unlines(
|     sortOn(lambda l: (-int(words(l)[1]), words(l)[0]), lines(text)))
```

Note. lines/unlines bracket the pipeline; the tuple key does both orderings. unlines terminates every line, hence the trailing newline in the contract.

9.10 Combined guest list

apply

Two venues export their guest ids as sorted lists which may share entries. Produce one sorted list with each id appearing once. Merge first, then deduplicate – both passes linear.

```
merge_unique(a: list[int], b: list[int]) -> list[int]

>>> merge_unique([1, 3, 5], [1, 2, 5])
[1, 2, 3, 5]
>>> merge_unique([], [4, 4])
[4]
>>> merge_unique([2], [])
[2]
```

Solution.

```
| merge_unique = lambda a, b: nub(merge(a, b))
```

Note. merge keeps the result sorted in $O(n+m)$; nub then removes adjacent (and any) duplicates keeping first occurrences. Sorting a concatenation would also work but pays $O(n \log n)$ for order the inputs already had.

9.11 Median of two sorted lists

challenge

Two sensors deliver their readings as sorted lists. Return the median of ALL readings as a float: the middle element of the combined order, or the mean of the two middles when the total count is even. At least one reading is guaranteed. A linear merge is accepted here; name the $O(\log(n+m))$ partition approach in your complexity remarks.

`median_sorted(a: list[int], b: list[int]) -> float`

```
>>> median_sorted([1, 3], [2])
2.0
>>> median_sorted([1, 2], [3, 4])
2.5
>>> median_sorted([], [5])
5.0
>>> median_sorted([1, 1], [1])
1.0
```

Solution.

```
def median_sorted(a, b):
    m = merge(a, b)
    n = len(m)
    return float(m[n // 2]) if n % 2 else (m[n // 2 - 1] + m[n // 2]) / 2
```

Note. merge gives the combined order in $O(n+m)$; the median reads off the middle. The famous interview escalation is the $O(\log)$ binary-partition version – know that it exists and say so.

9.12 Longest common prefix

challenge

Return the longest prefix shared by EVERY string in a list (empty string when the list is empty or shares nothing). Insight to find: after sorting, only the FIRST and LAST strings need comparing – anything they share, every string between them shares too.

`common_prefix(ws: list[str]) -> str`

```
>>> common_prefix(["flower", "flow", "flight"])
'fl'
>>> common_prefix(["dog", "cat"])
''
>>> common_prefix([])
''
>>> common_prefix(["same"])
'same'
```

Solution.

```
def common_prefix(ws):
    if not ws:
        return ""
    ss = sorted(ws)
    keep = takeWhile(lambda p: p[0] == p[1], zip_(ss[0], ss[-1]))
    return "".join(map_(fst, keep))
```

Note. Sorting makes the lexicographic extremes the only witnesses needed; `zip_ + takeWhile` walks the agreement. $O(S \log n)$ for total char count S . Common mistake: comparing only adjacent pairs and merging results.

Chapter 10

Dict Folds and Bags

Aggregation by key – the fold aimed at a mapping. One family, one idea: `insertWith` resolves a single collision, `fromListWith` a list of them, `unionWith` two dicts’ worth; and CHOOSING the combiner is choosing a monoid – addition counts, `old + new` groups, `max` is bag union. Mind the Haskell order: the combiner hears the NEW value first. `group/groupBy` handle the positional cousin: runs, not piles.

10.1 Concepts

10.1.1 defaultdict

a dict whose missing keys create themselves via a factory: touch `d[k]` and `factory()` is installed there. This "autovivification" makes grouping and counting one-liners. Gotcha: READING a missing key also plants it – use `'in'` or `.get` when you only want to look.

```
>>> d = defaultdict(list)
>>> d['x'].append(1)
>>> d
{'x': [1]}
>>> d2 = defaultdict(int)
>>> d2['ghost']                                # a mere read...
0
>>> 'ghost' in d2                               # ...planted the key
True
```

How it works: one dunder does everything. When `d[k]` misses, Python calls `d.__missing__(k)`; this subclass responds by calling the stored factory, installing the result, and returning it. The factory runs only on a miss – and it takes NO arguments, which is why the leaf recipes in `Tree/ITree` are zero-argument lambdas.

Why it earns its place: it deletes the check-then-create dance from every grouping and counting loop. The write path becomes a bare `append` or `+=`, and the structure assembles itself on touch – autovivification, which section 12’s trie tools then apply recursively.

The discipline it demands: a mere READ also plants the key, so split your paths – write with `d[k]`, but ask questions with `'in'` or `.get`. The prelude’s bag operations read `Counters` with `.get` for exactly this reason.

```
>>> d = defaultdict(int)
>>> d['hits'] += 1                               # write path: indexing is the point
>>> d.get('misses', 0), 'misses' in d           # read path: no ghost key planted
(0, False)
```

Remember it as: index to build, `.get` to ask.

Roll your own: subclass `dict` and implement one dunder – `__missing__`, which Python calls when `[k]` fails. Call the factory, install the result, hand it back; iteration, repr and everything else is inherited. Note the factory takes no arguments, which is why leaf recipes are zero-argument lambdas.

```
class defaultdict(dict): # dict that manufactures a value via factory() on first miss
    def __init__(self, factory=None, *args, **kw):
        super().__init__(*args, **kw)
```

```

        self.factory = factory
    def __missing__(self, key):
        if self.factory is None:
            raise KeyError(key)
        self[key] = self.factory()
        return self[key]

```

Used before it is rolled: on in §16.1.7 (p. 135).

10.1.2 Counter

count occurrences: a `defaultdict(int)` fold over the input. Missing keys read as 0 (mind the autovivification gotcha above); rank the `.items()` with `sortOn`.

```

>>> c = Counter("abracadabra")
>>> c['a'], c['z']
(5, 0)
>>> sortOn(lambda kv: -kv[1], list(Counter("aab").items()))
[('a', 2), ('b', 1)]

```

Roll your own: a `defaultdict(int)` plus one fold – `d[x] += 1` per element. The `+=` on a missing key is autovivification doing all the work.

```

def Counter(xs): # count-by-value; a defaultdict(int) fold
    d = defaultdict(int)
    for x in xs:
        d[x] += 1
    return d

```

10.1.3 insertWith

`Data.Map`'s `insertWith f k v m`: insert `(k, v)`, combining with `f(new, old)` on collision – Haskell's argument order, new value first. `PERSISTENT`, also like Haskell: it returns a fresh dict and leaves the argument untouched.

```

>>> insertWith(add, 'x', 4, {'x': 1})
{'x': 5}
>>> d = {'y': 2}
>>> insertWith(add, 'x', 4, d)
{'y': 2, 'x': 4}
>>> d                                     # the original survives
{'y': 2}

```

Roll your own: one dict display – `**d, k: ...`: the splat copies `d` (that IS the persistence), and the new key's value is `f(v, d[k])` on collision (new value first, Haskell's order) or plain `v`. Compare `fromListWith`, which applies the same rule but mutates, because there the dict is its own private accumulator.

```

insertWith = lambda f, k, v, d: {'**d, k: f(v, d[k]) if k in d else v} # Data.Map insertWith: PERSISTENT
# (fresh dict); f(new, old)

```

10.1.4 fromListWith

THE dict-building fold: pour `(key, value)` pairs into a dict, combining collisions with `f(new, old)` – Haskell's order, like `insertWith`. Keys keep first-seen order.

```

>>> fromListWith(add, [(c, 1) for c in "aab"])
{'a': 2, 'b': 1}
>>> fromListWith(flip(add), [('a', [1]), ('b', [2]), ('a', [3])])
{'a': [1, 3], 'b': [2]}

```

The order gotcha, faithfully Haskell's: because `f` receives `(new, old)`, grouping with a bare `concat` REVERSES each group – the classic `Data.Map` surprise. Combine as `flip(add)` (`old + new`) to keep arrival order, or use plain `add` where order cannot matter (counts).

```
>>> fromListWith(add, [(len(w), [w]) for w in ["hi", "ox", "sun"]])
{2: ['ox', 'hi'], 3: ['sun']}
>>> fromListWith(flip(add), [(len(w), [w]) for w in ["hi", "ox", "sun"]])
{2: ['hi', 'ox'], 3: ['sun']}
```

Why it earns its place – one function, three monoids: choosing f is choosing what the dict MEANS. Addition over $(k, 1)$ pairs counts; old $+$ new over $(k, [v])$ pairs groups; max merges bags. That is the whole family: `insertWith` is one collision, `fromListWith` a list of them, `unionWith` two dicts’ worth (where f receives (a’s value, b’s value)).

The recognition cue: whenever a loop builds a dict with an if-else on key existence, you are hand-rolling this fold. Name the pairs, name the monoid, and the loop disappears.

Remember it as: grouping, counting, and merging are one fold – whose combiner hears the NEW value first.

Roll your own: an empty dict and one loop applying the collision rule per pair – $d[k] = f(v, d[k])$ if the key exists, else v . It is `insertWith` inlined and IN-PLACE: mutation is fine here because the dict being built is the fold’s own accumulator, invisible until returned.

```
def fromListWith(f, pairs): # Data.Map fromListWith: THE dict-building fold; f(new, old)
    d = {} # like insertWith -- so grouping with concat REVERSES each
    for k, v in pairs: # group (the classic Haskell gotcha); use add for counts
        d[k] = f(v, d[k]) if k in d else v
    return d # Counter == fromListWith(add) over (x, 1);
    # grouping == fromListWith(++) over (k, [v])
```

Used before it is rolled: *group* in §10.1.7 (p. 82), *flip* in §16.1.3 (p. 133).

10.1.5 unionWith

merge two dicts, combining SHARED keys with f ; a’s keys keep their order, b’s new keys follow. Choosing f is choosing a monoid: max gives bag union, $+$ gives a summing merge.

```
>>> unionWith(max, {'a': 1, 'b': 5}, {'b': 2, 'c': 3})
{'a': 1, 'b': 5, 'c': 3}
>>> unionWith(add, {'a': 1, 'b': 2}, {'b': 3, 'c': 4})
{'a': 1, 'b': 5, 'c': 4}
```

Roll your own: two splats – `**a, **...`: a copied whole, then overlaid with b’s items after each has been through the collision rule, $f(a[k], v)$ when k is shared, plain v when it is new. The asymmetry is deliberate: a’s keys keep their positions, and only keys new to b append at the end.

```
unionWith = lambda f, a, b: {**a, **{k: f(a[k], v) if k in a else v for k, v in b.items()}}
# Data.Map unionWith: f(a's value, b's value) on shared keys;
# bag_union == unionWith(max), merge-sum == unionWith(add)
```

Used before it is rolled: *bag_union* in §10.1.8 (p. 82), *on* in §16.1.7 (p. 135).

10.1.6 groupBy

runs by an equivalence predicate; each newcomer is compared with eq against the run’s FIRST element (span-based, exactly Haskell’s `groupBy`), not against its neighbour – a subtle and deliberate fidelity.

```
>>> groupBy(lambda a, b: a == b, [1, 1, 2])
[[1, 1], [2]]
>>> groupBy(lambda a, b: b - a <= 1, [1, 2, 3, 10, 11])
[[1, 2], [3], [10, 11]]
```

The word that matters is CONSECUTIVE: a new run starts whenever eq against the run’s first element fails, so ‘abab’ yields four runs. Not a weakness – a different question: runs, streaks and compression care about position, and these are the only tools asking about it.

```
>>> [(run[0], len(run)) for run in group("aaabbbbcc")] # run lengths
[('a', 3), ('b', 4), ('c', 2)]
```

For CATEGORIES – position irrelevant – either sort first, so equals become adjacent, or use `fromListWith`, which never cared about position at all. Choosing between them is stating what your key means.

```
>>> [(run[0], len(run)) for run in group(sorted("abab"))]
[('a', 2), ('b', 2)]
```

Remember it as: `group` answers "how long are the runs?"; `fromListWith` answers "what are the piles?".

Roll your own: one pass in which each element either joins the current run or starts a fresh one. The test is `eq(out[-1][0], x)` – against the run's FIRST element, not its last – which is exactly how Haskell's span-based `groupBy` behaves, and the detail that makes non-transitive predicates (like "within 1 of") agree with it.

```
def groupBy(eq, xs): # Data.List groupBy: runs of CONSECUTIVE elements; each new
    out = [] # element is compared via eq against the run's FIRST element,
    for x in xs: # exactly like Haskell (span-based) -- not its neighbor
        if out and eq(out[-1][0], x):
            out[-1].append(x)
        else:
            out.append([x])
    return out
```

Used before it is rolled: *group* in §10.1.7 (p. 82).

10.1.7 group

runs of CONSECUTIVE equal elements, as bare lists (`[[a]]`, no keys) – `Data.List`'s `group`.

```
>>> group("aaabbc")
[['a', 'a', 'a'], ['b', 'b'], ['c']]
>>> group("abab")
[['a'], ['b'], ['a'], ['b']]
```

Roll your own — the definition:

```
group = partial(groupBy, lambda a, b: a == b) # Data.List group: runs of equals, [[a]] -- no keys
```

Used before it is rolled: *partial* in §16.1.6 (p. 135).

10.1.8 bag_union

multisets (bags) are Counters; `union` takes the LARGER count per key. Key order follows set union, so compare with `==` rather than printing.

```
>>> bag_union(Counter("aab"), Counter("abb")) == {'a': 2, 'b': 2}
True
```

Roll your own — the definition:

```
bag_union = lambda a, b: {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() | b.keys()}
```

10.1.9 bag_inter

intersection: the SMALLER count per shared key, zero counts dropped. Note the implementation reads with `.get`, never `[]` – indexing a Counter would autovivify zeros into it.

```
>>> bag_inter(Counter("aab"), Counter("ab")) == {'a': 1, 'b': 1}
True
>>> bag_inter(Counter("aa"), Counter("bb")) == {}
True
```

Roll your own — the definition:

```
bag_inter = lambda a, b: {k: v for k in a.keys() & b.keys() if (v := min(a.get(k, 0), b.get(k, 0)))} # min count
    ↪ per shared key
```

10.1.10 bag_diff

difference, clipped at zero: what remains of `a` after removing `b`.

```
>>> bag_diff(Counter("aab"), Counter("ab")) == {'a': 1}
True
```

Roll your own — the definition:

```
bag_diff = lambda a, b: {k: a[k] - b[k] for k in a if a[k] > b.get(k, 0)} # clipped at 0
```

10.1.11 bag_sub

inclusion: is bag a contained in bag b, multiplicities included? The ransom-note test.

```
>>> bag_sub(Counter("ab"), Counter("aabb"))
True
>>> bag_sub(Counter("aab"), Counter("ab"))
False
```

Roll your own — the definition:

```
bag_sub = lambda a, b: all(b.get(k, 0) >= n for k, n in a.items()) # a <= b (inclusion)
```

10.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

10.1 Vowel census

drill

Count the frequency of each vowel (a, e, i, o, u) in a string, case-insensitive. Return the counts as (vowel, count) pairs sorted alphabetically; vowels that never occur are omitted.

vowel_census(s: str) -> list[tuple[str, int]]

```
>>> vowel_census("Banana")
[('a', 3)]
>>> vowel_census("queue")
[('e', 2), ('u', 2)]
>>> vowel_census("xyz")
[]
```

Solution.

```
vowel_census = lambda s: sorted(
    Counter(filter_(lambda ch: ch in "aeiou", s.lower())).items())
```

Note. `filter_` narrows to vowels, `Counter` folds the counts, `sorted` makes the output deterministic. Reading `Counter` with `[]` would autovivify zeros – returning `.items()` sidesteps that entirely.

10.2 Order totals

drill

An order log is a list of (product, quantity) pairs; products repeat. Total the quantity per product with one `fromListWith` fold. Keys keep first-seen order.

totals(pairs: list[tuple[str, int]]) -> dict[str, int]

```
>>> totals([("apple", 2), ("pear", 1), ("apple", 3)])
{'apple': 5, 'pear': 1}
>>> totals([])
{}
>>> totals([("kiwi", 4)])
{'kiwi': 4}
```

Solution.

```
totals = lambda pairs: fromListWith(add, pairs)
```

Note. Addition is commutative, so the `f(new, old)` order cannot bite here. This is THE dict-building fold at its simplest: pairs in, totals out, $O(n)$.

10.3 Longest run

drill

Return the length of the longest run of consecutive equal characters in a string (0 for the empty string). `group` hands you the runs; you measure them.

```
longest_run(s: str) -> int
```

```
>>> longest_run("aaabb")
3
>>> longest_run("abab")
1
>>> longest_run("")
0
```

Solution.

```
| longest_run = lambda s: max(map(len, group(s)), default=0)
```

Note. `group` returns bare runs (`[a]`), so `map len` measures them directly; `default=0` covers the empty string. One pass, $O(n)$.

10.4 Restock, persistently

drill

Given a warehouse inventory dict and one delivery (item, quantity), return a NEW inventory with the delivery added to that item's count – and leave the original dict untouched, which is exactly `insertWith`'s contract. New items simply appear.

```
restock(inv: dict[str, int], item: str, qty: int) -> dict[str, int]
```

```
>>> inv = {'nail': 10}
>>> restock(inv, 'nail', 5)
{'nail': 15}
>>> inv
{'nail': 10}
>>> restock(inv, 'screw', 3)
{'nail': 10, 'screw': 3}
```

Solution.

```
| restock = lambda inv, item, qty: insertWith(add, item, qty, inv)
```

Note. `insertWith` copies the dict first – that copy IS the persistence – then combines `f(new, old)` on collision. Mutating `inv` directly is the mistake this drill exists to catch.

10.5 Tile check

drill

Scrabble-style: can a word be assembled from a hand of letter tiles, respecting multiplicity (two o's in the word need two o tiles)? This is multiset inclusion – one `bag_sub`.

```
can_build(word: str, tiles: str) -> bool
```

```
>>> can_build("cat", "tacos")
True
>>> can_build("book", "bok")
False
>>> can_build("", "xyz")
True
```

Solution.

```
| can_build = lambda word, tiles: bag_sub(Counter(word), Counter(tiles))
```

Note. Two `Counter` folds and an inclusion test; $O(n + m)$. A set-based check is the classic wrong answer – it ignores multiplicity.

10.6 High-water marks

apply

Two flood sensors each report per-zone water maxima as a dict. Combine them into one report keeping the HIGHER reading for zones both saw; zones only one sensor saw pass through unchanged. Choose the right combining function for `unionWith` – the monoid is the design decision.

```
high_water(a: dict[str, int], b: dict[str, int]) -> dict[str, int]

>>> high_water({'north': 3, 'south': 7}, {'south': 9, 'east': 2})
{'north': 3, 'south': 9, 'east': 2}
>>> high_water({}, {'west': 1})
{'west': 1}
>>> high_water({'north': 4}, {})
{'north': 4}
```

Solution.

```
high_water = lambda a, b: unionWith(max, a, b)
```

Note. `unionWith(max)` is exactly `bag_union` generalized to any dict; a's keys keep their positions, b's new keys append. Swapping in addition would instead give a summing merge – same shape, different monoid.

10.7 Click sessions

apply

A user's click timestamps arrive sorted. Clicks belong to one session while they fall within 30 seconds OF THE SESSION'S FIRST CLICK; a later click starts a new session. Split the timestamps into sessions – note this is `groupBy`'s compare-against-the-run's-FIRST semantics, verbatim, not a neighbor-gap rule.

```
sessions(ts: list[int]) -> list[list[int]]

>>> sessions([0, 10, 25, 50, 60, 100])
[[0, 10, 25], [50, 60], [100]]
>>> sessions([])
[]
>>> sessions([5])
[[5]]
```

Solution.

```
sessions = lambda ts: groupBy(lambda start, t: t - start <= 30, ts)
```

Note. `groupBy` hands eq the run's FIRST element and each newcomer – precisely the anchored-window rule. A neighbor-gap rule would need pairwise instead; confusing the two is the point of this exercise.

10.8 Department roster

apply

HR exports (department, employee) rows in hiring order. Build a dict mapping each department to its employees IN HIRING ORDER. Mind from `listWith`'s `f(new, old)` argument order: a bare `concat` would reverse each roster.

```
roster(rows: list[tuple[str, str]]) -> dict[str, list[str]]

>>> roster([("eng", "ann"), ("ops", "bob"), ("eng", "cy")])
{'eng': ['ann', 'cy'], 'ops': ['bob']}
>>> roster([])
{}
>>> roster([("qa", "dee")])
{'qa': ['dee']}
```

Solution.

```
roster = lambda rows: fromListWith(flip(add), [(d, [e]) for d, e in rows])
```

Note. Wrap each employee as a one-element list, then combine `old + new` to keep arrival order – `f` receives the NEW value first (Haskell's order), so a plain `(+)` reverses every roster. That reversal is the classic gotcha.

Rolled later: flip in §16.1.3 (p. 133).

10.9 Shopping list

apply

A recipe lists its ingredients with repeats ('egg' twice means two eggs); your pantry is a list of what you have. Produce the shopping list – what is still missing and how many of each – as (item, count) pairs sorted alphabetically. Multiset difference, clipped at zero.

```
shopping_list(need: list[str], have: list[str]) -> list[tuple[str, int]]
```

```
>>> shopping_list(["egg", "egg", "milk"], ["egg"])
[('egg', 1), ('milk', 1)]
>>> shopping_list(["jam"], ["jam", "jam"])
[]
>>> shopping_list([], ["salt"])
[]
```

Solution.

```
shopping_list = lambda need, have: sorted(
    bag_diff(Counter(need), Counter(have)).items())
```

Note. `bag_diff` subtracts with clipping at zero, so surplus pantry items vanish rather than going negative; `sorted` makes the output order deterministic.

10.10 Traffic rollup

apply

Each server in a fleet reports a dict of per-page visit counts. Roll the whole fleet up into one dict of grand totals. This is a FOLD whose combining step is itself a `unionWith` – dict-merging as a monoid.

```
rollup(reports: list[dict[str, int]]) -> dict[str, int]
```

```
>>> rollup([{'home': 1}, {'home': 2, 'faq': 5}, {'faq': 1}])
{'home': 3, 'faq': 6}
>>> rollup([])
{}
>>> rollup([{'a': 1}])
{'a': 1}
```

Solution.

```
rollup = lambda reports: foldl(lambda acc, d: unionWith(add, acc, d), reports, {})
```

Note. `unionWith(+)` merges two reports; `foldl` extends that to any number, seeded with the empty dict (the monoid's identity). Key order: first server to mention a page owns its position.

10.11 Majority element

challenge

An election log is a list of votes. The most common candidate wins only with a STRICT majority (more than half of all votes): return that candidate, or `None` when nobody qualifies. Ties for the top count must not invent a majority. Count with a bag; interrogate it deterministically.

```
majority(xs: list) -> value | None
```

```
>>> majority([1, 2, 1, 1])
1
>>> majority([1, 2]) is None
True
>>> majority([]) is None
True
>>> majority([5])
5
```

Solution.

```
def majority(xs):
    if not xs:
        return None
    c = Counter(xs)
```

```
best = maxOn(lambda k: c[k], sorted(c))
return best if c[best] * 2 > len(xs) else None
```

Note. Counter tallies in $O(n)$; maxOn over the sorted keys picks a deterministic top; the strict $> \text{len}/2$ test rejects mere pluralities. Interview escalation: Boyer-Moore voting does it in $O(1)$ space – name it.

10.12 Top spender per city

challenge

A payment feed yields (city, customer, amount) rows; customers repeat. For every city, find the customer with the highest TOTAL spend there; break ties by taking the alphabetically first name. Return city: customer with cities in first-appearance order. Two stacked dict folds: totals per (city, customer), then a per-city best.

```
top_spender(rows: list[tuple[str, str, int]]) -> dict[str, str]

>>> top_spender([("nyc", "ann", 50), ("nyc", "bob", 70), ("la", "cy", 20),
...              ("nyc", "ann", 30)])
{'nyc': 'ann', 'la': 'cy'}
>>> top_spender([("sf", "a", 10), ("sf", "b", 10)])
{'sf': 'a'}
>>> top_spender([])
{}
```

Solution.

```
def top_spender(rows):
    totals = fromListWith(add, [(c, p), a] for c, p, a in rows)
    per_city = fromListWith(flip(add),
                           [(c, [(p, t)]) for (c, p), t in totals.items()])
    return {c: minOn(lambda pt: (-pt[1], pt[0]), ps)[0] for c, ps in per_city.items()}
```

Note. Fold one: totals keyed by the (city, customer) PAIR. Fold two: regroup by city, keeping arrival order with old + new. minOn over (-total, name) gets highest-spend-then-alphabetical in one key. $O(n)$ folds plus a per-city scan.

Rolled later: flip in §16.1.3 (p. 133).

Chapter 11

Trees and Tries

Hierarchy as nested dicts that build themselves. `ITree` autovivifies a branch the moment you touch it – a trie in one line – and `Tree(depth, leaf)` bottoms the same idea out at fixed depth. Writing is indexing or `setpath`; asking is ALWAYS `getpath` (reads plant ghost branches otherwise); and `paths` folds the whole tree back into rows, which is where the value gets harvested.

11.1 Concepts

11.1.1 Tree

nested defaultdicts to a FIXED depth, with a leaf factory at the bottom. `Tree(1, list)` is the grouping dict; `Tree(2, int)` is a sparse 2-D counting table. Indexing autovivifies the whole path.

```
>>> t = Tree(1, list)
>>> t['evens'].append(2)
>>> t
{'evens': [2]}
>>> m = Tree(2, int)
>>> m['a']['b'] += 1
>>> m
{'a': {'b': 1}}
```

Roll your own — the definition:

```
Tree = lambda depth, leaf: (defaultdict(leaf) if depth == 1 # fixed-depth nested defaultdict;
                             else defaultdict(lambda: Tree(depth - 1, leaf))) # leaves default to leaf()
```

Used before it is rolled: *leaves* in §11.1.4 (p. 90).

11.1.2 ITree

the INFINITE tree: every branch is another `ITree`, so any depth autovivifies on touch. This is the trie: index letter by letter and the branch builds itself.

```
>>> t = ITree()
>>> t['x']['y']['z'] = 9
>>> t
{'x': {'y': {'z': 9}}}
```

How it works: the definition is a knot – `ITree = lambda: defaultdict(ITree)`. Every missing key manufactures another `ITree`, which will do the same, without end. Indexing therefore GROWS the tree: touching `t['c']['a']['t']` conjures the whole branch. A trie in one line, built from nothing but `__missing__`.

```
>>> t = ITree()
>>> for w in ["cat", "car"]: setpath(t, list(w) + ['$'], w)
>>> sorted([leaf for p, leaf in paths(t) if p[-1] == '$'])
['car', 'cat']
```

Why it earns its place: hierarchies – tries, directory trees, nested groupings – stop needing setup code. Writing is indexing (or setpath, which also plants a leaf); harvesting is paths. The one edged rule: reads autovivify too, so QUESTIONS go through getpath, never bare indexing – getpath’s entry demonstrates the ghost branch this avoids.

Versus Tree(depth, leaf): Tree is the finite cousin – fixed depth, with a real leaf factory (list, int) at the bottom instead of more tree. Tree(1, list) is the grouping dict; ITree is for depth you cannot know in advance, like words.

Remember it as: a dict that dreams up more of itself on demand.

Roll your own: ITree = lambda: defaultdict(ITree) mentions itself – legal, because the lambda body only runs on a MISS, by which time the name is bound. Tree bottoms the same recursion out at a fixed depth by handing the last level a real leaf factory instead of more tree.

```
| ITree = lambda: defaultdict(ITree) # infinitely-nestable defaultdict: a tree that grows on access
```

Used before it is rolled: paths in §11.1.3 (p. 90), setpath in §11.1.5 (p. 91), on in §16.1.7 (p. 135).

11.1.3 paths

fold a whole Tree/ITree into a flat list of (keypath, leaf) pairs, in insertion order. The read- everything companion to setpath’s write-one-path.

```
>>> t = ITree()
>>> setpath(t, ['a', 'b'], 1); setpath(t, ['a', 'c'], 2)
>>> paths(t)
[(['a', 'b'], 1), (['a', 'c'], 2)]
```

How it works: structural recursion with two arms, exactly the case-arms-as-fold story. A non-dict (or empty node) is a LEAF: report the single row ([], leaf). A dict is a BRANCH: recurse into every subtree and prepend the branching key to each keypath that comes back. The whole tree flattens into (keypath, leaf) rows, insertion-ordered, losslessly.

Why it earns its place: building a trie is the easy half; the value is HARVESTING it. autocomplete filters rows under a prefix, deepest_directory takes maxOn of keypath length, leaves reads the second column. One cata, many harvests – you never write tree-walking code again.

```
>>> t = ITree()
>>> setpath(t, ['a', 'b'], 1); setpath(t, ['c'], 2)
>>> paths(t)
[(['a', 'b'], 1), (['c'], 2)]
>>> maxOn(lambda row: len(row[0]), paths(t))
(['a', 'b'], 1)
```

Remember it as: paths turns a tree back into rows; setpath turns rows into a tree.

Roll your own: recursion on one question – is this still a dict? A leaf reports a single row with an empty path; a branch recurses into every child and conses its key onto every path that comes back. Depth equals tree height, comfortably inside Python’s limit for sane tries.

```
def paths(t): # cata for Tree/ITree: [(keypath, leaf)]
    # recursion depth = tree height (word length / len(nums)) -- well under the ~1000 limit
    if not isinstance(t, dict) or not t: # leaf = non-dict value, or empty node
        return [([], t)]
    return [( [k] + p, leaf) for k, sub in t.items() for p, leaf in paths(sub)]
```

Used before it is rolled: setpath in §11.1.5 (p. 91).

11.1.4 leaves

just the leaf values of paths.

```
>>> t = ITree()
>>> setpath(t, ['a'], 1); setpath(t, ['b'], 2)
>>> leaves(t)
[1, 2]
```

Roll your own — the definition:

```
| leaves = compose(partial(map_, snd), paths) # every leaf value in a Tree/ITree, in path order
```

Used before it is rolled: *setpath* in §11.1.5 (p. 91), *partial* in §16.1.6 (p. 135), *compose* in §16.1.8 (p. 136).

11.1.5 setpath

write a leaf value at the end of a key path, autovivifying the branch on the way. Caution: writing at an interior key CLOBBERS the subtree below it.

```
>>> t = ITree()
>>> setpath(t, ['a', 'b'], 7)
>>> t
{'a': {'b': 7}}
>>> setpath(t, ['a'], 0)          # clobbers {'b': 7}
>>> t
{'a': 0}
```

Roll your own: walk all but the last key by PLAIN INDEXING – on an ITree that autovivifies the branch as you go – then assign at the final key, the single write. *getpath* is the mirror image with the safety catch on: *isinstance* and *'in'* checks instead of indexing, so reading plants nothing.

```
def setpath(t, ks, v): # write leaf v at key path ks (autovivifies interior)
    for k in ks[:-1]:
        t = t[k]
    t[ks[-1]] = v # NB: clobbers any subtree already at ks
```

11.1.6 getpath

read along a key path WITHOUT autovivifying, returning a default when the path is absent. This is why it exists: reading a trie with [] would plant ghost branches.

```
>>> t = ITree()
>>> setpath(t, ['a', 'b'], 7)
>>> getpath(t, ['a', 'b']), getpath(t, ['a', 'z'], default=0)
(7, 0)
>>> 'z' in t['a']          # no ghost branch was planted
False
```

Roll your own — the definition:

```
def getpath(t, ks, default=None): # read along ks WITHOUT autovivifying
    for k in ks:
        if not isinstance(t, dict) or k not in t:
            return default
        t = t[k]
    return t
```

11.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

11.1 Plant a settings tree

drill

Write *set_all*(pairs), where each pair is (key_path, value) with key_path a list of keys. Plant every value in a fresh ITree with *setpath* and return the tree. Later pairs overwrite earlier ones at the same path.

set_all(pairs: list[tuple[list, object]]) -> dict

```
>>> set_all([(['a', 'b'], 1), (['c'], 2)])
{'a': {'b': 1}, 'c': 2}
>>> set_all([(['x'], 1), (['x'], 9)])
{'x': 9}
>>> set_all([])
```

```
| {}
```

Solution.

```
def set_all(pairs):
    t = ITree()
    for ks, v in pairs:
        setpath(t, ks, v)
    return t
```

Note. One setpath per pair; autovivification builds every interior node. $O(\text{total path length})$. Common mistake: forgetting that setpath at an interior key clobbers the subtree below it – here that is the specified overwrite behaviour.

11.2 Dotted config lookup

drill

Write `get_setting(cfg, dotted, default)`: read a nested dict with a dotted key string like `'server.tls.on'`. Missing keys – or a path that runs THROUGH a non-dict leaf – answer the default, and the read must never plant ghost keys.

`get_setting(cfg: dict, dotted: str, default) -> object`

```
>>> cfg = {'server': {'port': 8080, 'tls': {'on': True}}}
>>> get_setting(cfg, 'server.port', 0)
8080
>>> get_setting(cfg, 'server.tls.on', False)
True
>>> get_setting(cfg, 'server.host', 'localhost')
'localhost'
>>> get_setting(cfg, 'server.port.max', -1)
-1
```

Solution.

```
| get_setting = lambda cfg, dotted, default: getpath(cfg, dotted.split('.'), default)
```

Note. `split('.')` turns the dotted string into a key path; `getpath` does the safe descent. Common mistake: chained `cfg[a][b]` indexing, which raises on misses and autovivifies on defaultdicts.

Rolled later: on in §16.1.7 (p. 135).

11.3 Every leaf value

drill

Write `leaf_values(t)`: all leaf values of a nested dict, in insertion order. Recognise the prelude tool before writing any recursion.

`leaf_values(t: dict) -> list`

```
>>> leaf_values({'a': {'b': 1, 'c': 2}, 'd': 3})
[1, 2, 3]
>>> leaf_values({'x': 7})
[7]
>>> leaf_values({'a': {'b': {'c': 'deep'}}})
['deep']
```

Solution.

```
| leaf_values = leaves
```

Note. This IS the prelude's leaves – the second column of paths. Like `flatten == concat`, recognising an existing tool is the whole exercise.

11.4 Two-level tally

drill

Write `sales_table(pairs)`: count (city, product) sale events into a two-level table – `table[city][product]` is the number of sales. Use `Tree(2, int)` so no key ever needs checking.

`sales_table(pairs: list[tuple[str, str]]) -> dict`

```
>>> sales_table([('nyc', 'ai'), ('nyc', 'ai'), ('la', 'uav')])
{'nyc': {'ai': 2}, 'la': {'uav': 1}}
```

```
>>> sales_table({})
{}
>>> sales_table([('x', 'y')])['x']['y']
1
```

Solution.

```
def sales_table(pairs):
    t = Tree(2, int)
    for city, product in pairs:
        t[city][product] += 1
    return t
```

Note. `Tree(2, int)` autovivifies the city level and hands the product level an int leaf factory, so the whole body is one `+=`. $O(n)$.

11.5 Slash-joined key paths

drill

Write `key_paths(t)`: the key path of every leaf in a nested dict (string keys), joined with `'/'`, in insertion order.

`key_paths(t: dict) -> list[str]`

```
>>> key_paths({'a': {'b': 1}, 'c': 2})
['a/b', 'c']
>>> key_paths({'solo': 9})
['solo']
>>> key_paths({'a': {'b': {'c': 0}}})
['a/b/c']
```

Solution.

```
key_paths = lambda t: ['/'.join(p) for p, _ in paths(t)]
```

Note. `paths` hands back (keypath, leaf) rows; keep the first column and join. $O(\text{total keys})$.

11.6 Phone prefix counter

apply

A call-routing box holds a set of DISTINCT phone numbers (digit strings). Write `count_with_prefix(numbers, prefix)`: how many numbers start with the given prefix. Build a digit trie once (`'$'`-terminated so a full number survives being a prefix of another), descend to the prefix subtree with `getpath`, and count the `'$'` leaves below it.

`count_with_prefix(numbers: list[str], prefix: str) -> int`

```
>>> nums = ['5551234', '5555678', '5551299']
>>> count_with_prefix(nums, '555')
3
>>> count_with_prefix(nums, '5551')
2
>>> count_with_prefix(nums, '9')
0
>>> count_with_prefix(['12', '123'], '12')
2
```

Solution.

```
def count_with_prefix(numbers, prefix):
    t = ITree()
    for n in numbers:
        setpath(t, list(n) + ['$'], True)
    sub = getpath(t, list(prefix), {})
    return len([1 for p, _ in paths(sub) if p and p[-1] == '$'])
```

Note. The `'$'` terminator is what lets `'12'` be counted under prefix `'12'` even though `'123'` continues past it. `getpath` (never indexing) keeps the miss case from planting a ghost branch. $O(\text{total digits})$ to build, $O(\text{subtree})$ to count.

11.7 Deep config merge

apply

Deployment configs are nested dicts. Write `deep_merge(a, b)`: `b`'s settings override `a`'s, except where BOTH sides hold dicts – then merge recursively. Return a new dict; neither argument may be mutated at any level you touch. Key order: `a`'s keys first, then `b`'s new ones.

```
deep_merge(a: dict, b: dict) -> dict
```

```
>>> deep_merge({'a': {'x': 1}, 'b': 2}, {'a': {'y': 9}, 'c': 3})
{'a': {'x': 1, 'y': 9}, 'b': 2, 'c': 3}
>>> deep_merge({'a': {'x': 1}}, {'a': 5})
{'a': 5}
>>> deep_merge({}, {'k': 1})
{'k': 1}
>>> orig = {'a': {'x': 1}}
>>> _ = deep_merge(orig, {'a': {'y': 2}})
>>> orig
{'a': {'x': 1}}
```

Solution.

```
def deep_merge(a, b):
    out = dict(a)
    for k, v in b.items():
        if k in out and isinstance(out[k], dict) and isinstance(v, dict):
            out[k] = deep_merge(out[k], v)
        else:
            out[k] = v
    return out
```

Note. `unionWith`'s shape, recursing when both sides are dicts – `dict(a)` at every level is the persistence (compare `insertWith`). Common mistake: `out[k].update(v)`, which mutates `a`'s subtree in place.

11.8 Folder rollup

apply

Given `(path, size)` pairs like `('a/b/f.txt', 10)`, write `folder_size(files, folder)`: the total size under a folder such as `'a/b'` (or the size of a single file if the path names one). Unknown paths answer 0. Build the tree with `setpath`, descend with `getpath`, sum the leaves.

```
folder_size(files: list[tuple[str, int]], folder: str) -> int
```

```
>>> fs = [('a/b/f.txt', 10), ('a/b/g.txt', 5), ('a/c/h.txt', 7), ('d.txt', 1)]
>>> folder_size(fs, 'a/b')
15
>>> folder_size(fs, 'a')
22
>>> folder_size(fs, 'zzz')
0
>>> folder_size(fs, 'a/b/f.txt')
10
```

Solution.

```
def folder_size(files, folder):
    t = ITree()
    for path, size in files:
        setpath(t, path.split('/'), size)
    sub = getpath(t, folder.split('/'), {})
    return 0 if sub == {} else sum(leaves(sub))
```

Note. A file path resolves to an int, and `paths()` on a non-dict yields one `([], leaf)` row, so `sum(leaves(...))` covers files and folders alike. The `==` guard matters: `paths` treats an empty dict as a LEAF, so summing the miss default would try `0 + .`

11.9 Word index, two keys deep

apply

Write `word_index(words)`: index words first by initial letter, then by length – `index[letter][length]` is the list of matching words in arrival order. Words are non-empty and lowercase.

```
word_index(words: list[str]) -> dict
>>> word_index(['ant', 'ape', 'bee', 'at'])
{'a': {3: ['ant', 'ape'], 2: ['at']}, 'b': {3: ['bee']}}
>>> word_index([])
{}
>>> word_index(['x'])['x'][1]
['x']
```

Solution.

```
def word_index(words):
    t = Tree(2, list)
    for w in words:
        t[w[0]][len(w)].append(w)
    return t
```

Note. `Tree(2, list)`: two autovivified levels ending in list leaves, so grouping is a bare `append`. The `fromListWith` alternative needs nested dict surgery – fixed-depth `Tree` is the cleaner fit here.

11.10 Flatten to dotted lines

apply

The inverse of a config tree: write `dotted(t)` producing sorted `'path.to.key=value'` strings for every leaf (`str()` the value). An empty tree yields `[]`. Keys are strings without dots.

```
dotted(t: dict) -> list[str]
>>> dotted({'srv': {'port': 8080, 'tls': {'on': True}}, 'debug': False})
['debug=False', 'srv.port=8080', 'srv.tls.on=True']
>>> dotted({})
[]
>>> dotted({'a': 1})
['a=1']
```

Solution.

```
def dotted(t):
    if not t:
        return []
    return sorted('.'.join(p) + '=' + str(leaf) for p, leaf in paths(t))
```

Note. `paths` flattens, `join+str` renders, `sorted` makes the output deterministic. The empty guard exists because `paths()` reports the empty dict itself as a leaf row – a corner of the paths contract worth remembering.

Rolled later: on in §16.1.7 (p. 135).

11.11 Longest common prefix via trie

challenge

Write `common_prefix(words)`: the longest string that every word starts with. Build a '\$'-terminated trie, then walk from the root while the current node has EXACTLY one child and that child is not the terminator. `[]` answers ". Classic interview question – the trie walk is the $O(\text{total chars})$ answer that scales to millions of words.

```
common_prefix(words: list[str]) -> str
>>> common_prefix(['flower', 'flow', 'flight'])
'fl'
>>> common_prefix(['car', 'card'])
'car'
>>> common_prefix(['abc'])
'abc'
>>> common_prefix(['dog', 'race'])
''
>>> common_prefix([])
```

```
| ''
```

Solution.

```
def common_prefix(words):
    t = ITree()
    for w in words:
        setpath(t, list(w) + ['$'], True)
    out, node = '', t
    while len(node) == 1 and '$' not in node:
        k = next(iter(node))
        out, node = out + k, node[k]
    return out
```

Note. The walk stops at the first branch point OR the first '\$' – the terminator is what stops 'car'/'card' at 'car'. Membership ('\$' in node) is safe on an ITree: 'in' never autovivifies, only indexing does.

11.12 Render the directory listing

challenge

Given directory paths like 'a/b' (parents may or may not be listed separately), write `tree_lines(dirs)`: the indented listing – each name on its own line, two spaces per depth, children in first-mention order. Build by folding indexing over each path's parts (walking IS building on an ITree), then emit with your own recursion over items().

`tree_lines(dirs: list[str]) -> list[str]`

```
>>> tree_lines(['a/b', 'a/c', 'd'])
['a', '  b', '  c', 'd']
>>> tree_lines(['x/y/z'])
['x', '  y', '    z']
>>> tree_lines([])
[]
>>> tree_lines(['m', 'm/n'])
['m', '  n']
```

Solution.

```
def tree_lines(dirs):
    t = ITree()
    for d in dirs:
        foldl(lambda node, k: node[k], d.split('/'), t)
    def emit(node, depth):
        out = []
        for k, sub in node.items():
            out.append('  ' * depth + k)
            out.extend(emit(sub, depth + 1))
        return out
    return emit(t, 0)
```

Note. The build needs no `setpath` because there is no leaf value – indexing autovivifies the whole branch (the `deepest_directory` move). `paths()` cannot render interior nodes, so the emitter is a hand recursion over `items()`; that is the intended lesson.

Chapter 12

Machines: Deque, DSU, Heap

Three data structures rebuilt from nothing, each carrying one great argument: the two-stack `deque`'s amortised $O(1)$ ends (each element migrates once); union-find's path halving making connectivity effectively constant – with `union` returning `False` AS the cycle detector; and the heap's single invariant (parent beats children) keeping the minimum readable at `h[0]` forever. Owning these means never being helpless without imports.

12.1 Concepts

12.1.1 deque

a double-ended queue built from two stacks, all four end operations amortised $O(1)$ (elements migrate between stacks only when one runs dry). This is the BFS frontier; a plain list's `pop(0)` is $O(n)$.

```
>>> d = deque([1, 2, 3])
>>> d.appendleft(0); d.append(4)
>>> d.popleft(), d.pop()
(0, 4)
>>> len(d), list(d)
(3, [1, 2, 3])
```

How it works: two plain lists, nose to nose – `_out` holds the front (reversed), `_in` holds the back. `append` pushes onto `_in`, `appendleft` onto `_out`, both $O(1)$. The trick is popping an EMPTY side: the other list is reversed across in one $O(n)$ migration, and popping is $O(1)$ again.

Why that still counts as $O(1)$: amortization. Each element migrates at most once between being pushed and popped, so n operations cost $O(n)$ in total – constant per operation on average, which is what the workload's big-O cares about. A bare Python list cannot say this: `pop(0)` shifts every element, every single time.

```
>>> q = deque()
>>> for x in [1, 2, 3]: q.append(x)
>>> q.popleft(), q.popleft()          # first pop pays the migration; the next is free
(1, 2)
```

Why it earns its place: BFS. The frontier must yield its OLDEST member (`popleft`) while newcomers join at the back (`append`) – `levelorder` and `grid_path` are this class doing its one job.

Remember it as: two stacks facing each other make a queue; the reversal bill is paid once per element.

Roll your own: two plain lists nose to nose, each end appending and popping its own list at $O(1)$. All the cleverness is in the pop guard: when your side is empty, reverse the OTHER list across in one go. Each element makes that crossing at most once between entering and leaving, so the occasional $O(n)$ reversal amortises to $O(1)$ per operation.

```
class deque: # two-stack; all four ends amortized O(1)
    def __init__(self, xs=()):
        self._in, self._out = [], list(reversed(list(xs)))
    def append(self, x): self._in.append(x)
    def appendleft(self, x): self._out.append(x)
    def pop(self):
        if not self._in:
```

```

        self._in, self._out = list(reversed(self._out)), []
    return self._in.pop()
def popleft(self):
    if not self._out:
        self._out, self._in = list(reversed(self._in)), []
    return self._out.pop()
def __len__(self): return len(self._in) + len(self._out)
def __iter__(self): return iter(self._out[::-1] + self._in)

```

12.1.2 dsu

disjoint-set union (union-find) over n elements, with path halving. `find` gives a set's representative; union merges and returns `False` when the two were ALREADY connected – which is exactly a cycle detector (Kruskal's test).

```

>>> find, union = dsu(4)
>>> union(0, 1), union(1, 2)
(True, True)
>>> union(0, 2)                                # already connected: a cycle
False
>>> find(0) == find(2), find(0) == find(3)
(True, False)

```

How it works: `parent[]` links every element toward its set's ROOT, and `find` follows the chain up. The subtle line is `parent[x] = parent[parent[x]]` – path HALVING: every `find` re-points each visited node at its grandparent, so chains shrink as a side effect of being walked. That one line makes operations effectively constant time (inverse Ackermann, for the curious).

Why it earns its place: dynamic connectivity. "Are these two in the same group yet?" under a stream of merges is awkward for every other structure and trivial here. And union returning `False` – already connected – IS cycle detection: Kruskal's spanning tree is built by skipping every `False`.

```

>>> find, union = dsu(5)
>>> [union(a, b) for a, b in [(0, 1), (1, 2), (3, 4), (0, 2)]]
[True, True, True, False]
>>> len({find(i) for i in range(5)})           # two components remain
2

```

Remember it as: `find` walks to the root, halving as it goes; a `False` union is a cycle caught.

Roll your own: `parent[]` as the forest and two closures instead of a class. `find`'s loop carries the entire trick in one line – `parent[x] = parent[parent[x]]` re-points every visited node at its grandparent (path halving), flattening chains as a side effect of walking them. `union` is `find` twice plus one root re-point, answering `False` when the roots already agree.

```

def dsu(n): # union-find, path halving; union -> False if already joined
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return False
        parent[ra] = rb
        return True
    return find, union

```

12.1.3 heappush

push onto a min-heap kept in a plain list (sift-up). The smallest element is always `h[0]`. For a MAX-heap, push negated values.

```

>>> h = []
>>> for x in [5, 1, 8]: heappush(h, x)
>>> h[0]
1

```

| 1

How it works: the flat list IS a binary tree – $h[i]$'s children live at $2i+1$ and $2i+2$ – kept under one invariant: every parent $\leq$ its children. `heappush` appends at the first free leaf and sifts UP, swapping with its parent while smaller; `heappop` moves the last leaf to the root and sifts DOWN toward the smaller child. Each walks one root-to-leaf path: $O(\log n)$.

Why the invariant is enough: nothing is fully sorted – siblings sit in any order – yet $h[0]$ is ALWAYS the minimum, because the root beats its children, who beat theirs. You pay $\log n$ per operation for a permanently readable minimum; draining the heap is heapsort; pushing negatives turns min into max.

```
>>> h = []
>>> for x in [7, 2, 9, 4]: heappush(h, x)
>>> h[0]
2
>>> [heappop(h) for _ in range(4)]
[2, 4, 7, 9]
```

Why it earns its place: "repeatedly take the best next thing" – Dijkstra's frontier, `running_median`'s two halves, any streaming top-k. When next-best is needed many times while the data keeps changing, the heap beats re-sorting every time.

Remember it as: a flat list, a parent-beats-children promise, and log-n repairs.

Roll your own: append at the first free leaf, then sift UP – while the parent at $(i - 1) // 2$ is larger, swap and climb. The index arithmetic IS the tree; watch the array after four pushes:

```
>>> h = []
>>> for x in [7, 2, 9, 4]: heappush(h, x)
>>> h
[2, 4, 9, 7]
# root 2; children 4 and 9; 7 pushed down under 4
```

```
def heappush(h, x): # min-heap on a plain list: sift-up
    h.append(x)
    i = len(h) - 1
    while i and h[halve(i - 1)] > h[i]:
        h[halve(i - 1)], h[i] = h[i], h[halve(i - 1)]
        i = halve(i - 1)
```

Used before it is rolled: `heappop` in §12.1.4 (p. 99), on in §16.1.7 (p. 135).

12.1.4 heappop

remove and return the minimum (sift-down). Popping repeatedly drains in sorted order – that is heapsort.

```
>>> h = []
>>> for x in [5, 1, 8]: heappush(h, x)
>>> [heappop(h) for _ in range(3)]
[1, 5, 8]
>>> g = []
>>> for x in [3, 9, 5]: heappush(g, -x)
>>> -heappop(g)
9
# max-heap via negation
```

Roll your own: swap the root with the last leaf, pop it off the end, then sift DOWN – repeatedly swap with the SMALLER of the two children (choosing the smaller is the classic bug site) until neither child is smaller. One root-to-leaf path each way: $O(\log n)$.

```
def heappop(h): # min-heap on a plain list: sift-down
    h[0], h[-1] = h[-1], h[0]
    top = h.pop()
    i, n = 0, len(h)
    while True:
        s, l, r = i, 2*i + 1, 2*i + 2
        if l < n and h[l] < h[s]: s = l
        if r < n and h[r] < h[s]: s = r
        if s == i:
            return top
        h[i], h[s] = h[s], h[i]
```

```
i = s
```

Used before it is rolled: on in §16.1.7 (p. 135).

12.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

12.1 Rotate left with a deque

drill

Write `rotate_left(xs, k)`: rotate a list `k` positions left using a deque – `popleft` feeds `append`, `k` times. `k` may be 0, huge, or applied to an empty list.

```
rotate_left(xs: list, k: int) -> list
```

```
>>> rotate_left([1, 2, 3, 4], 1)
[2, 3, 4, 1]
>>> rotate_left([1, 2, 3, 4], 6)
[3, 4, 1, 2]
>>> rotate_left([], 3)
[]
>>> rotate_left([7], 99)
[7]
```

Solution.

```
def rotate_left(xs, k):
    d = deque(xs)
    for _ in range(k % len(xs) if xs else 0):
        d.append(d.popleft())
    return list(d)
```

Note. Each rotation is `popleft` + `append`, both $O(1)$; `k % len` caps the work. Common mistake: forgetting the empty-list guard before the modulo.

12.2 Keep only the last `k`

drill

A dashboard shows only the `k` most recent readings. Write `last_k(xs, k)`: feed the stream through a deque, evicting from the front whenever the buffer exceeds `k`, and return the final buffer as a list.

```
last_k(xs: list, k: int) -> list
```

```
>>> last_k([1, 2, 3, 4, 5], 2)
[4, 5]
>>> last_k([1, 2], 5)
[1, 2]
>>> last_k([], 3)
[]
```

Solution.

```
def last_k(xs, k):
    d = deque()
    for x in xs:
        d.append(x)
        if len(d) > k:
            d.popleft()
    return list(d)
```

Note. Append at the back, evict from the front: the bounded-buffer idiom, $O(1)$ per event. A plain list doing `pop(0)` here would be $O(n)$ per eviction.

12.3 Same group yet?

drill

Write `same_group(n, pairs, queries)` for elements $0..n-1$: apply every merge in pairs, then answer each query (a, b) with whether a and b ended up connected. Compare find results – never raw parent values, which are arbitrary.

`same_group(n: int, pairs: list[tuple], queries: list[tuple]) -> list[bool]`

```
>>> same_group(5, [(0, 1), (1, 2)], [(0, 2), (0, 3), (3, 3)])
[True, False, True]
>>> same_group(3, [], [(0, 1)])
[False]
>>> same_group(2, [(0, 1)], [(1, 0)])
[True]
```

Solution.

```
def same_group(n, pairs, queries):
    f, union = dsu(n)
    for a, b in pairs:
        union(a, b)
    return [f(a) == f(b) for a, b in queries]
```

Note. Union everything, then each query is two finds. Near $O(1)$ per operation with path halving. Naming the local 'f' avoids shadowing the prelude's find.

12.4 Heapsort by draining

drill

Write `heap_sorted(xs)`: push every element onto a min-heap, then pop until empty. The drain order IS sorted order – that is heapsort.

`heap_sorted(xs: list) -> list`

```
>>> heap_sorted([5, 1, 4, 1, 3])
[1, 1, 3, 4, 5]
>>> heap_sorted([])
[]
>>> heap_sorted([2])
[2]
```

Solution.

```
def heap_sorted(xs):
    h = []
    for x in xs:
        heappush(h, x)
    return [heappop(h) for _ in range(len(h))]
```

Note. n pushes + n pops, each $O(\log n)$: $O(n \log n)$ total. Duplicates survive – a heap is a bag, not a set.

12.5 The k smallest

drill

Write `k_smallest(xs, k)`: the k smallest values in ascending order (all of them if k exceeds the length). Push everything, pop k times.

`k_smallest(xs: list, k: int) -> list`

```
>>> k_smallest([9, 4, 7, 1, 0], 3)
[0, 1, 4]
>>> k_smallest([5, 5, 5], 2)
[5, 5]
>>> k_smallest([1, 2], 10)
[1, 2]
```

Solution.

```
def k_smallest(xs, k):
    h = []
    for x in xs:
```

```

    heappush(h, x)
    return [heappop(h) for _ in range(min(k, len(h)))]

```

Note. $O(n \log n)$ build, $O(k \log n)$ drain – beats fully sorting when k is small. $\min(k, \text{len})$ is the guard the last doctest checks.

12.6 Hot potato

apply

Players stand in a circle and count around: every m -th player is eliminated until one remains (the Josephus game). Write `survivor(names, m)`: rotate the deque $m-1$ times (popleft feeding append), eliminate with a popleft, repeat. Return the last name standing.

```
survivor(names: list[str], m: int) -> str
```

```

>>> survivor(['a', 'b', 'c', 'd', 'e'], 2)
'c'
>>> survivor(['solo'], 7)
'solo'
>>> survivor(['a', 'b', 'c'], 1)
'c'
>>> survivor(['a', 'b'], 3)
'b'

```

Solution.

```

def survivor(names, m):
    d = deque(names)
    while len(d) > 1:
        for _ in range(m - 1):
            d.append(d.popleft())
        d.popleft()
    return d.popleft()

```

Note. The deque IS the circle: rotation moves the count, popleft eliminates. $O(n * m)$. Common mistake: rotating m times instead of $m - 1$.

12.7 Merge k sorted feeds

apply

k sensors each emit an already-sorted list of readings. Write `merge_all(lists)`: one sorted stream, heap-powered: seed the heap with each list's head as (value, list_index, position), then repeatedly pop the smallest and push that list's next reading. The index in the tuple breaks value ties so comparison never reaches unorderable data.

```
merge_all(lists: list[list[int]]) -> list[int]
```

```

>>> merge_all([[1, 4, 6], [2, 3], [5]])
[1, 2, 3, 4, 5, 6]
>>> merge_all([], [1], [])
[1]
>>> merge_all([])
[]
>>> merge_all([[1, 1], [1]])
[1, 1, 1]

```

Solution.

```

def merge_all(lists):
    h, out = [], []
    for li, xs in enumerate(lists):
        if xs:
            heappush(h, (xs[0], li, 0))
    while h:
        val, li, i = heappop(h)
        out.append(val)
        if i + 1 < len(lists[li]):
            heappush(h, (lists[li][i + 1], li, i + 1))
    return out

```

Note. The heap holds at most one entry per list, so each of the N total readings costs $O(\log k)$: $O(N \log k)$, the multiway-merge bound. This is the prelude's two-list merge generalised by a frontier.

12.8 How many friend circles?

apply

n people, a list of friendships. Friendship is transitive through the merges. Write `components(n, pairs)`: how many separate circles remain – union every pair, then count DISTINCT roots via `find` over everyone.

`components(n: int, pairs: list[tuple]) -> int`

```
>>> components(5, [(0, 1), (3, 4)])
3
>>> components(4, [])
4
>>> components(3, [(0, 1), (1, 2)])
1
>>> components(0, [])
0
```

Solution.

```
def components(n, pairs):
    f, union = dsu(n)
    for a, b in pairs:
        union(a, b)
    return len({f(i) for i in range(n)})
```

Note. Count roots with a set comprehension over `find` – roots are arbitrary but distinct per component. The final full pass of `finds` also flattens every chain.

12.9 The edge that closes a loop

apply

Network links arrive one at a time. Write `first_cycle_edge(n, edges)`: the FIRST link that connects two already-connected nodes (union answering `False`), or `None` if every link merges something new. This is exactly how Kruskal's algorithm rejects edges.

`first_cycle_edge(n: int, edges: list[tuple]) -> tuple | None`

```
>>> first_cycle_edge(5, [(0, 1), (1, 2), (0, 2), (3, 4)])
(0, 2)
>>> first_cycle_edge(3, [(0, 1), (1, 2)]) is None
True
>>> first_cycle_edge(2, [(0, 0)])
(0, 0)
```

Solution.

```
def first_cycle_edge(n, edges):
    f, union = dsu(n)
    return find(lambda e: not union(e[0], e[1]), edges)
```

Note. `find`'s laziness meets `union`'s side effect: edges are merged one at a time until one reports `False`, and nothing after it is touched. Keep the `dsu` locals named `f/union` so the prelude's `find` stays visible.

12.10 k-th largest, live

apply

A leaderboard shows the k -th largest score seen so far, updating per submission. Write `kth_largest_stream(xs, k)`: after each element, report the k -th largest so far, or `None` while fewer than k have arrived. Keep a min-heap of the k best – its root IS the answer.

`kth_largest_stream(xs: list[int], k: int) -> list`

```
>>> kth_largest_stream([3, 1, 5, 12, 2, 11], 3)
[None, None, 1, 3, 3, 5]
>>> kth_largest_stream([4, 4], 1)
[4, 4]
>>> kth_largest_stream([], 2)
```

| []

Solution.

```
def kth_largest_stream(xs, k):
    h, out = [], []
    for x in xs:
        heappush(h, x)
        if len(h) > k:
            heappop(h)
        out.append(h[0] if len(h) == k else None)
    return out
```

Note. The bounded min-heap holds the k largest; evicting the MIN keeps exactly them, and $h[0]$ reads the k -th largest in $O(1)$. $O(n \log k)$. Common mistake: a max-heap, which cannot evict its smallest.

12.11 Sliding window maximum

challenge

Write `window_max(xs, k)`: the maximum of every length- k window, in order (classic hard interview question – $O(n)$ required, so no per-window `max()`). Keep a deque of INDICES with values decreasing: each newcomer pops smaller values off the back; the front is the window's max, dropped when it slides out of range. The prelude deque has no peek: peek by pop-then-push-back on the side you need.

`window_max(xs: list[int], k: int) -> list[int]`

```
>>> window_max([1, 3, -1, -3, 5, 3, 6, 7], 3)
[3, 3, 5, 5, 6, 7]
>>> window_max([2, 1], 1)
[2, 1]
>>> window_max([4, 2, 3], 3)
[4]
>>> window_max([], 2)
[]
```

Solution.

```
def window_max(xs, k):
    d, out = deque(), []
    for i, x in enumerate(xs):
        while len(d):
            j = d.pop()
            if xs[j] >= x:
                d.append(j)
                break
        d.append(i)
        f = d.popleft()
        if f > i - k:
            d.appendleft(f)
        if i >= k - 1:
            f = d.popleft()
            out.append(xs[f])
            d.appendleft(f)
    return out
```

Note. The monotonic deque: every index enters once and leaves once (back-pop when beaten, front-pop when expired), so the nested while is still $O(n)$ amortised. The pop/push- back peeks are the standard workaround for a peekless deque.

12.12 Cheapest network (Kruskal)

challenge

Weighted links (w, u, v) can connect n offices. Write `kruskal(n, edges)`: the minimum total cost to connect ALL offices, or None if impossible. Sort edges by weight, take every edge whose union succeeds, stop counting when $n-1$ merges have landed. `sortOn` + `dsu` is the whole algorithm.

`kruskal(n: int, edges: list[tuple]) -> int | None`

```

>>> kruskal(4, [(1, 0, 1), (2, 1, 2), (10, 0, 2), (3, 2, 3)])
6
>>> kruskal(3, [(5, 0, 1)]) is None
True
>>> kruskal(1, [])
0
>>> kruskal(4, [(4, 0, 1), (1, 2, 3), (2, 1, 2), (7, 0, 3)])
7

```

Solution.

```

def kruskal(n, edges):
    f, union = dsu(n)
    total = merges = 0
    for w, u, v in sort0n(fst, edges):
        if union(u, v):
            total += w
            merges += 1
    return total if merges == n - 1 else None

```

Note. Greedy-by-weight is optimal because a cheapest edge crossing any cut belongs to some minimum spanning tree; union==False is the cycle filter. $O(E \log E)$ for the sort; the dsu work is effectively linear. merges == n-1 doubles as the connectivity test.

Chapter 13

Nodes: Linked Lists and Binary Trees

The LeetCode gives. For trees, one catamorphism – `tfold` – makes every traversal and measure a one-liner, because `z` is the empty-tree arm and `f` the node arm; `levelorder` adds the deque for breadth-first. For linked lists, the pointer idioms worth knowing cold: three-pointer reversal, fast/slow middles, Floyd’s cycle detection, dummy-head merging. Convert node results with `to_list` so everything stays testable.

13.1 Concepts

13.1.1 `TreeNode`

the LeetCode binary tree: `val`, `left`, `right`, with `None` for absent children.

```
>>> t = TreeNode(2, TreeNode(1), TreeNode(3))
>>> t.left.val, t.val, t.right.val
(1, 2, 3)
```

Roll your own — the definition:

```
class TreeNode: # the LeetCode binary tree
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
```

13.1.2 `tfold`

the tree catamorphism: recursively combines each node’s value with the already-folded left and right results, `z` standing in for empty subtrees. EVERY structural tree computation is one choice of `f` and `z` – the five traversal/measure functions below are all `tfold` one-liners.

```
>>> t = TreeNode(2, TreeNode(1), TreeNode(3))
>>> tfold(lambda v, l, r: v + l + r, 0, t)      # sum
6
>>> tfold(lambda v, l, r: max(v, l, r), 0, t)   # max
3
```

How it works: the tree catamorphism – structural recursion with a contract. Two arms, exactly like a case expression: the empty tree answers `z`; a node answers `f(value, left_result, right_result)`, both results computed by the same recursion. Hand it `(f, z)` and any question about a tree becomes one line – the five traversals and both measures in section 13 are all `tfold` with different arms.

Why it earns its place: it separates WALKING from DECIDING. The recursion is written once, correct forever; you only ever author the arms. Even tree TRANSFORMATION fits – mirror a tree by swapping the child results while rebuilding:

```
>>> t = TreeNode(2, TreeNode(1), TreeNode(3))
>>> mirror = tfold(lambda v, l, r: TreeNode(v, r, l), None, t)
>>> inorder(mirror)
[3, 2, 1]
```

The one caveat: recursion depth equals tree height – fine for balanced and interview-sized trees; say a sentence out loud about adversarially skewed ones.

Remember it as: *z* is the empty-tree arm, *f* is the node arm – the fold walks, you decide.

Roll your own: one line of structural recursion – *z* when the tree is *None*, else *f* over the node’s value and the two recursive results.

Roll your own: one line of structural recursion – *z* when the tree is *None*, else *f* over the node’s value and the two recursive results. Every traversal and measure in this section is that line with a different *f* plugged in.

```
| tfold = lambda f, z, t: z if t is None else f(t.val, tfold(f, z, t.left), tfold(f, z, t.right))
```

*Used before it is rolled: **inorder** in §13.1.3 (p. 108).*

13.1.3 inorder

left, node, right; on a binary SEARCH tree this is sorted order.

```
| >>> inorder(TreeNode(2, TreeNode(1), TreeNode(3)))
| [1, 2, 3]
```

Roll your own — the definition:

```
| inorder = partial(tfold, lambda v, l, r: l + [v] + r, []) # left, root, right
```

*Used before it is rolled: **partial** in §16.1.6 (p. 135).*

13.1.4 preorder

node first, then children: the copy/serialise order.

```
| >>> preorder(TreeNode(2, TreeNode(1), TreeNode(3)))
| [2, 1, 3]
```

Roll your own — the definition:

```
| preorder = partial(tfold, lambda v, l, r: [v] + l + r, []) # root, left, right
```

*Used before it is rolled: **partial** in §16.1.6 (p. 135).*

13.1.5 postorder

children first, then node: the delete/evaluate order.

```
| >>> postorder(TreeNode(2, TreeNode(1), TreeNode(3)))
| [1, 3, 2]
```

Roll your own — the definition:

```
| postorder = partial(tfold, lambda v, l, r: l + r + [v], []) # left, right, root
```

*Used before it is rolled: **partial** in §16.1.6 (p. 135).*

13.1.6 tdepth

the height of the tree.

```
| >>> tdepth(TreeNode(2, TreeNode(1, TreeNode(0)), TreeNode(3)))
| 3
```

Roll your own — the definition:

```
| tdepth = partial(tfold, lambda _, l, r: 1 + max(l, r), 0) # height of the tree
```

*Used before it is rolled: **partial** in §16.1.6 (p. 135).*

13.1.7 tsize

the number of nodes.

```
>>> tsize(TreeNode(2, TreeNode(1), TreeNode(3)))
3
```

Roll your own — the definition:

```
tsize = partial(tfold, lambda _, l, r: 1 + l + r, 0) # number of nodes in the tree
```

Used before it is rolled: *partial* in §16.1.6 (p. 135).

13.1.8 levelorder

breadth-first values, level by level: the deque earning its keep. Compare preorder on the same tree to see DFS vs BFS.

```
>>> t = TreeNode(1, TreeNode(2, TreeNode(4)), TreeNode(3))
>>> levelorder(t)
[1, 2, 3, 4]
>>> preorder(t)
[1, 2, 4, 3]
```

Roll your own: seed a deque with the root, then loop – popleft, record the value, append the children that exist. FIFO order IS breadth-first; swap the deque for a list used as a stack and the identical loop turns into depth-first.

```
def levelorder(t): # BFS: the deque earning its keep
    out, q = [], deque([t] if t else [])
    while len(q):
        n = q.popleft()
        out.append(n.val)
        if n.left: q.append(n.left)
        if n.right: q.append(n.right)
    return out
```

13.1.9 ListNode

the LeetCode singly-linked list: val and next.

```
>>> n = ListNode(1, ListNode(2))
>>> n.val, n.next.val
(1, 2)
```

Roll your own — the definition:

```
class ListNode: # the LeetCode linked list
    def __init__(self, val=0, nxt=None):
        self.val, self.next = val, nxt
```

13.1.10 from_list

build a linked list from a Python list: literally foldr(ListNode, xs, None). foldr's f takes (element, accumulator), and ListNode(val, nxt) has exactly that shape, so the last element is wrapped first and each earlier one points at the list built so far.

```
>>> to_list(from_list([1, 2, 3]))
[1, 2, 3]
```

Roll your own — the definition:

```
from_list = lambda xs: foldr(ListNode, xs, None) # build; foldr of ListNode
```

Used before it is rolled: *to_list* in §13.1.11 (p. 110).

13.1.11 to_list

walk a linked list back into a Python list; the round-trip partner of `from_list`, and the way to make node results printable. An unfoldr: the seed is the node, each step yields `(node.val, node.next)`, and `None` ends it.

```
>>> to_list(from_list("ab"))
['a', 'b']
>>> to_list(None)
[]
```

Roll your own — the definition:

```
to_list = compose(list, partial(unfoldr, lambda n: None if n is None else (n.val, n.next))) # ListNode chain ->
    ↪ list
```

Used before it is rolled: *partial* in §16.1.6 (p. 135), *compose* in §16.1.8 (p. 136).

13.1.12 reverse_list

reverse in place with the three-pointer walk; `prev` is a fold accumulator wearing pointers.

```
>>> to_list(reverse_list(from_list([1, 2, 3])))
[3, 2, 1]
```

Roll your own: the three-pointer walk compressed into one simultaneous assignment – `node.next, prev, node = prev, node, node.next`. The right-hand side is evaluated in full before any assignment lands, which is what saves the pointer you are about to overwrite.

```
def reverse_list(node): # three-pointer; prev is a fold accumulator
    prev = None
    while node:
        node.next, prev, node = prev, node, node.next
    return prev
```

13.1.13 middle

the middle node by fast/slow pointers (fast moves twice per step); on even lengths, the SECOND middle.

```
>>> middle(from_list([1, 2, 3])).val
2
>>> middle(from_list([1, 2, 3, 4])).val
3
```

Roll your own: fast advances two hops for slow's one, so when fast falls off the end, slow stands at the middle. The loop condition (`fast and fast.next`) is precisely what selects the SECOND middle on even lengths.

```
def middle(node): # fast/slow: second middle for even lengths
    slow = fast = node
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    return slow
```

13.1.14 has_cycle

Floyd's tortoise and hare: the fast pointer laps the slow one if and only if a cycle exists; $O(1)$ space.

```
>>> has_cycle(from_list([1, 2, 3]))
False
>>> n = ListNode(1); n.next = n # a self-loop
>>> has_cycle(n)
True
```

Roll your own: the same fast/slow walk, but watching for the pointers to MEET – on a cycle the fast runner laps the slow one; on a straight list it simply falls off. $O(1)$ space, no visited set.

```
def has_cycle(node): # Floyd: fast laps slow iff a cycle exists
    slow = fast = node
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            return True
    return False
```

13.1.15 merge_lists

merge two SORTED linked lists, using the dummy-head idiom to avoid special-casing the first node.

```
>>> to_list(merge_lists(from_list([1, 3]), from_list([2, 4])))
[1, 2, 3, 4]
```

Roll your own: the dummy-head idiom – a throwaway node to hang the result from, so appending the first real node needs no special case; walk both lists copying the smaller head, glue the survivor, return dummy.next.

```
def merge_lists(a, b): # merge two sorted lists; dummy-head idiom
    dummy = tail_ = ListNode()
    while a and b:
        if a.val <= b.val:
            tail_.next, a = a, a.next
        else:
            tail_.next, b = b, b.next
        tail_ = tail_.next
    tail_.next = a or b
    return dummy.next
```

13.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

13.1 Sum of the tree

drill

Return the sum of every value in a binary tree. The empty tree sums to 0. Use tfold: pick the combining arm and the empty-tree seed.

```
tree_sum(t: TreeNode | None) -> int
>>> tree_sum(TreeNode(2, TreeNode(1), TreeNode(3)))
6
>>> tree_sum(TreeNode(5, TreeNode(3, TreeNode(1)), TreeNode(2)))
11
>>> tree_sum(None)
0
```

Solution.

```
tree_sum = lambda t: tfold(lambda v, l, r: v + l + r, 0, t)
```

Note. The catamorphism with $f = \text{plus}$ and $z = 0$. $O(n)$. Common slip: forgetting z covers BOTH missing children and the empty tree.

13.2 Largest value in the tree

drill

Return the largest value in a NON-EMPTY binary tree of integers, again as one tfold. Think about what seed is safe when every real value must be able to beat it.

```
tree_max(t: TreeNode) -> int # t is non-empty
>>> tree_max(TreeNode(2, TreeNode(9), TreeNode(3)))
```

```

9
>>> tree_max(TreeNode(-5, TreeNode(-2, TreeNode(-8)), None))
-2
>>> tree_max(TreeNode(7))
7

```

Solution.

```
tree_max = lambda t: tfold(lambda v, l, r: max(v, l, r), float('-inf'), t)
```

Note. f = three-way max, z = -infinity so absent children never win. $O(n)$. The non-empty contract is what keeps -inf from leaking out.

13.3 Count the leaves

drill

Count the LEAVES (nodes with no children) of a binary tree. One tfold suffices – notice that a node is a leaf exactly when both child results are zero.

```

count_leaves(t:  TreeNode | None) -> int
>>> count_leaves(TreeNode(2, TreeNode(1), TreeNode(3)))
2
>>> count_leaves(TreeNode(1, TreeNode(2, TreeNode(3))))
1
>>> count_leaves(TreeNode(7))
1
>>> count_leaves(None)
0

```

Solution.

```
count_leaves = lambda t: tfold(lambda v, l, r: (l + r) or 1, 0, t)
```

Note. $(l + r)$ or 1: internal nodes pass the children's total through; a leaf's 0 total falls back to 1. $O(n)$. The 'or' trick replaces an explicit is-leaf test tfold cannot make.

13.4 Linked length

drill

Return how many nodes a linked list holds. Convert once and measure – `to_list` is the bridge between pointer-land and list-land.

```

llen(node:  ListNode | None) -> int
>>> llen(from_list([1, 2, 3]))
3
>>> llen(from_list([]))
0
>>> llen(ListNode(9))
1

```

Solution.

```
llen = lambda node: len(to_list(node))
```

Note. `to_list` then `len`: $O(n)$ either way, and the list form is printable and testable. `from_list([])` is None – the empty list IS the null pointer.

13.5 The traversal triple

drill

Return a tree's (inorder, preorder, postorder) value lists as one tuple. All three are tfold one-liners already in the prelude; this drill is about KNOWING their orders cold.

```

traversals(t:  TreeNode | None) -> tuple[list, list, list]
>>> traversals(TreeNode(2, TreeNode(1), TreeNode(3)))
([1, 2, 3], [2, 1, 3], [1, 3, 2])
>>> traversals(TreeNode(3, TreeNode(2, TreeNode(1))))
([1, 2, 3], [3, 2, 1], [1, 2, 3])

```

```
>>> traversals(None)
([], [], [])
```

Solution.

```
traversals = lambda t: (inorder(t), preorder(t), postorder(t))
```

Note. `inorder` = left,node,right; `preorder` = node first (serialise/copy order); `postorder` = node last (delete/evaluate order). A left chain makes pre and post read as reverses.

13.6 Kth newest audit entry

apply

A server keeps an append-only audit trail as a linked list, oldest entry first. Ops asks for the k-th NEWEST entry ($k = 1$ is the last one). $1 \leq k \leq \text{length}$ is guaranteed.

```
kth_from_end(node: ListNode, k: int) -> value
```

```
>>> kth_from_end(from_list([10, 20, 30, 40]), 1)
40
>>> kth_from_end(from_list([10, 20, 30, 40]), 4)
10
>>> kth_from_end(from_list([7]), 1)
7
```

Solution.

```
kth_from_end = lambda node, k: to_list(node)[-k]
```

Note. Materialise once, index from the end: $O(n)$ time either way. The two-pointer gap trick saves memory, not time – say that trade aloud if asked.

13.7 Palindrome ticket tape

apply

A ticket printer spools IDs onto a tape held as a linked list. QA wants to know whether the tape reads the same in both directions. The empty tape counts as a palindrome.

```
is_palindrome_list(node: ListNode | None) -> bool
```

```
>>> is_palindrome_list(from_list([1, 2, 1]))
True
>>> is_palindrome_list(from_list([1, 2]))
False
>>> is_palindrome_list(from_list([]))
True
>>> is_palindrome_list(from_list([7]))
True
```

Solution.

```
def is_palindrome_list(node):
    xs = to_list(node)
    return xs == xs[::-1]
```

Note. Convert, compare with the reversed slice: $O(n)$ time, $O(n)$ space. The $O(1)$ -space version (`reverse_list` on the back half from middle) is the follow-up interviewers like.

13.8 Deduplicate a sorted list

apply

Subscription events arrive as a SORTED linked list of customer ids, possibly with repeats. Return a linked list with each id once, order preserved.

```
dedup(node: ListNode | None) -> ListNode | None
```

```
>>> to_list(dedup(from_list([1, 1, 2, 3, 3])))
[1, 2, 3]
>>> to_list(dedup(from_list([4, 4, 4])))
[4]
```

```
>>> to_list(dedup(from_list([])))
[]
>>> to_list(dedup(from_list([1, 2])))
[1, 2]
```

Solution.

```
dedup = lambda node: from_list(nub(to_list(node)))
```

Note. Round-trip through list-land: `to_list`, `nub` (first occurrence wins – on sorted input that equals adjacent-dedup), `from_list`. $O(n)$. Pointer surgery is the $O(1)$ -extra-space alternative.

13.9 Is it a search tree?

apply

A binary tree claims to be a binary SEARCH tree with strictly increasing values. Verify the claim. Key insight: a BST is exactly a tree whose inorder walk is strictly increasing – so check the walk, not the tree.

```
is_bst(t: TreeNode | None) -> bool
```

```
>>> is_bst(TreeNode(2, TreeNode(1), TreeNode(3)))
True
>>> is_bst(TreeNode(2, TreeNode(3), TreeNode(1)))
False
>>> is_bst(TreeNode(2, TreeNode(2)))
False
>>> is_bst(None)
True
```

Solution.

```
def is_bst(t):
    xs = inorder(t)
    return all(a < b for a, b in pairwise(xs))
```

Note. `inorder` + `pairwise` + `all`: three prelude names replace the classic min/max-bounds recursion. Strict `<` also rejects duplicates. $O(n)$. The bounds recursion is the $O(1)$ -extra-space follow-up.

13.10 Symmetric tree

apply

An org chart is 'symmetric' if the whole tree equals its own mirror image – values AND structure. Check it by folding the tree into a nested-tuple shape twice: once as-is, once with the child results swapped, and comparing.

```
is_symmetric(t: TreeNode | None) -> bool
```

```
>>> is_symmetric(TreeNode(1, TreeNode(2), TreeNode(2)))
True
>>> is_symmetric(TreeNode(1, TreeNode(2), TreeNode(3)))
False
>>> is_symmetric(TreeNode(1, TreeNode(2, TreeNode(3)), TreeNode(2, TreeNode(3))))
False
>>> is_symmetric(TreeNode(1, TreeNode(2, None, TreeNode(3)), TreeNode(2, TreeNode(3))))
True
>>> is_symmetric(None)
True
```

Solution.

```
def is_symmetric(t):
    shape = tfold(lambda v, l, r: (v, l, r), None, t)
    mirror = tfold(lambda v, l, r: (v, r, l), None, t)
    return shape == mirror
```

Note. `tfold` with `(v, l, r)` serialises the exact structure as nested tuples; swapping to `(v, r, l)` builds the mirror's serialisation. Equal tuples == symmetric tree. $O(n)$. Plain inorder comparison is NOT enough – it loses structure.

13.11 Merge k sorted feeds

challenge

k sensor feeds each deliver readings as a sorted linked list. Produce one sorted linked list containing everything. An empty collection of feeds yields the empty list. Hint: you already own a two-list merge; what fold turns two-way into k-way?

```
merge_k(feeds: list[ListNode | None]) -> ListNode | None

>>> to_list(merge_k([from_list([1, 4]), from_list([2, 3]), from_list([5])]))
[1, 2, 3, 4, 5]
>>> to_list(merge_k([from_list([]), from_list([2]), from_list([1, 3])]))
[1, 2, 3]
>>> to_list(merge_k([]))
[]
>>> to_list(merge_k([from_list([9])]))
[9]
```

Solution.

```
merge_k = lambda feeds: foldl(merge_lists, feeds, None)
```

Note. foldl of the two-way merge, seeded with the empty list. $O(k*N)$ total; the heap-of-heads version is $O(N \log k)$ – name it, then justify the simpler fold at interview scale.

13.12 Height-balanced?

challenge

A tree is height-balanced when EVERY node's two subtrees differ in height by at most 1. Decide it in one pass. Hint: tfold can carry a PAIR upward – (height, balanced_so_far) – so each node judges itself from its children's pairs.

```
is_balanced(t: TreeNode | None) -> bool

>>> is_balanced(TreeNode(1, TreeNode(2), TreeNode(3)))
True
>>> is_balanced(TreeNode(1, TreeNode(2, TreeNode(3))))
False
>>> is_balanced(None)
True
>>> deep = TreeNode(2, TreeNode(3, TreeNode(4)))
>>> is_balanced(TreeNode(1, deep, TreeNode(9)))
False
```

Solution.

```
def is_balanced(t):
    def f(v, l, r):
        return (1 + max(l[0], r[0]),
                l[1] and r[1] and abs(l[0] - r[0]) <= 1)
    return tfold(f, (0, True), t)[1]
```

Note. The compound-accumulator trick: fold up (height, ok) pairs so balance is checked at every node in one $O(n)$ pass. The classic mistake is checking only the root's height difference – the fourth test catches exactly that.

Chapter 14

Grids and Windows

Two small tools with outsized reach. The neighbour generators put the bounds check in ONE place so every flood-fill and BFS body stays clean. `longest_window` inverts control for sliding windows: the skeleton owns the two pointers, you own the state as three closures – `valid`, `push`, `shed` – and designing them is just answering “what makes a window illegal?”.

14.1 Concepts

14.1.1 `neighbors4`

the in-bounds orthogonal neighbours of cell (r, c) in an R-by-C grid. Bounds checking lives HERE, so BFS bodies stay clean.

```
>>> sorted(neighbors4(0, 0, 2, 2))      # corner: two neighbours
[(0, 1), (1, 0)]
>>> sorted(neighbors4(1, 1, 3, 3))      # centre: four
[(0, 1), (1, 0), (1, 2), (2, 1)]
```

Roll your own: yield each in-bounds delta of (r, c) – the bounds check lives HERE so every caller’s BFS body stays clean. `neighbors8` is the same generator over a 3x3 delta grid, with (dr or dc) excluding the centre cell.

```
def neighbors4(r, c, R, C): # in-bounds orthogonal neighbors
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        if 0 <= r + dr < R and 0 <= c + dc < C:
            yield r + dr, c + dc
```

14.1.2 `neighbors8`

the same plus diagonals.

```
>>> len(list(neighbors8(1, 1, 3, 3)))
8
>>> sorted(neighbors8(0, 0, 2, 2))
[(0, 1), (1, 0), (1, 1)]
```

Roll your own — the definition:

```
def neighbors8(r, c, R, C): # ... plus diagonals
    for dr in (-1, 0, 1):
        for dc in (-1, 0, 1):
            if (dr or dc) and 0 <= r + dr < R and 0 <= c + dc < C:
                yield r + dr, c + dc
```

14.1.3 `longest_window`

the sliding-window skeleton: it grows the right edge one element at a time (calling `push`), shrinks the left edge while your `valid()` says the window is illegal (calling `shed`), and tracks the best length. YOU supply the state as closures; the skeleton does the two-pointer bookkeeping. Longest- substring-without-repeats and longest-run-under-a- budget are both three closures away.

```
>>> seen = []
>>> longest_window("abcabb", lambda: len(seen) == len(set(seen)),
...               seen.append, lambda c: seen.pop(0))
3
>>> w = []
>>> longest_window([2, 1, 3, 4], lambda: sum(w) <= 6,
...               w.append, lambda x: w.pop(0))
3
```

How it works: inversion of control. The skeleton owns the two pointers – it grows hi one element per step (calling push), then shrinks lo while your valid() reports the window broken (calling shed per evicted element) – and it tracks the best length ever seen legal. You own only the STATE, held in closures over variables sitting next to the call.

Designing the closures is answering one question: "what makes a window ILLEGAL, and what must I track to know?" Distinct characters -> a duplicate count. A budget -> a running sum. At most k of something -> a counter. push and shed are that state's increment and decrement; valid is the legality test.

```
>>> w = []
>>> longest_window("aabcab", lambda: len(w) == len(set(w)), w.append, lambda c: w.pop(0))
3
```

Why closures and not a class: the state lives exactly as long as the call, sits in scope where you can read it, and needs no ceremony. Functions carrying state – the same muscle as curry's captured argument, here working a two-pointer job.

Remember it as: the skeleton slides the window; your three closures say what legal means.

Roll your own: hi sweeps forward exactly once, add() greeting each newcomer; an inner while shrinks lo, shed() by shed(), until valid() approves again. Every element is added once and removed at most once – the entire two-pointer O(n) argument, visible in two loops.

```
def longest_window(xs, valid, push, shed): # sliding-window skeleton; state lives in the closures
    lo = best = 0
    for hi in range(len(xs)):
        push(xs[hi])
        while not valid(): # shrink until legal again
            shed(xs[lo])
            lo += 1
        best = max(best, hi - lo + 1)
    return best
```

Used before it is rolled: *until* in §15.1.2 (p. 126).

14.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

14.1 How many neighbours?

drill

Return how many in-bounds orthogonal neighbours cell (r, c) has in an R-by-C grid. neighbors4 already does the bounds checking – you just count what it yields.

degree(r: int, c: int, R: int, C: int) -> int

```
>>> degree(0, 0, 2, 2)
2
>>> degree(0, 1, 3, 3)
3
>>> degree(1, 1, 3, 3)
4
>>> degree(0, 0, 1, 1)
0
```

Solution.

```
degree = lambda r, c, R, C: len(list(neighbors4(r, c, R, C)))
```

Note. neighbors4 is a generator, so list() it before len(). Corners have 2, edges 3, interior 4 – and a 1x1 grid has none.

14.2 Diagonal neighbours only

drill

Return the DIAGONAL in-bounds neighbours of (r, c), sorted. You own two neighbour generators; the diagonals are exactly what one yields and the other does not.

```
diag_neighbors(r: int, c: int, R: int, C: int) -> list[tuple[int, int]]
```

```
>>> diag_neighbors(0, 0, 3, 3)
[(1, 1)]
>>> diag_neighbors(1, 1, 3, 3)
[(0, 0), (0, 2), (2, 0), (2, 2)]
>>> diag_neighbors(0, 1, 1, 3)
[]
```

Solution.

```
diag_neighbors = lambda r, c, R, C: sorted(
    set(neighbors8(r, c, R, C)) - set(neighbors4(r, c, R, C)))
```

Note. Set difference of the 8-neighbourhood and the 4-neighbourhood, sorted for a deterministic answer. A 1-row grid has no diagonals at all.

14.3 Live neighbours

drill

A Game-of-Life board is a list of '0'/'1' strings. Count the live cells among the 8-neighbourhood of (r, c).

```
live_around(grid: list[str], r: int, c: int) -> int
```

```
>>> board = ['111', '101', '111']
>>> live_around(board, 1, 1)
8
>>> live_around(board, 0, 0)
2
>>> live_around(['1'], 0, 0)
0
```

Solution.

```
live_around = lambda grid, r, c: sum(
    grid[nr][nc] == '1' for nr, nc in neighbors8(r, c, len(grid), len(grid[0])))
```

Note. sum over a generator of booleans counts the Trues; bounds live inside neighbors8, so no edge special-cases. This is the inner loop of every Life step.

14.4 Longest positive stretch

drill

Given daily profit/loss numbers, find the length of the longest run of strictly positive days. Use longest_window: the state you must track is how many non-positive values sit inside the current window.

```
longest_positive(xs: list[int]) -> int
```

```
>>> longest_positive([1, 2, -1, 3, 4, 5])
3
>>> longest_positive([-1, -2])
0
>>> longest_positive([])
0
>>> longest_positive([5, 6])
2
```

Solution.

```
def longest_positive(xs):
    bad = [0]
    def add(x): bad[0] += x <= 0
```


Note. Outer scan + inner flood: every cell enters the queue at most once, $O(R \cdot C)$. Mark seen when ENQUEUEING, not when popping, or the queue balloons with duplicates.

14.7 At most k distinct products

apply

A shelf camera streams the product code seen each second, as a string of letters. Find the longest stretch showing at most k DISTINCT products. Window state: per-code counts plus how many codes are currently present.

`longest_at_most_k(s: str, k: int) -> int`

```
>>> longest_at_most_k('eceba', 2)
3
>>> longest_at_most_k('aa', 1)
2
>>> longest_at_most_k('aabbcc', 1)
2
>>> longest_at_most_k('abc', 0)
0
```

Solution.

```
def longest_at_most_k(s, k):
    counts, distinct = Tree(1, int), [0]
    def add(c):
        counts[c] += 1
        distinct[0] += counts[c] == 1
    def rem(c):
        counts[c] -= 1
        distinct[0] -= counts[c] == 0
    return longest_window(s, lambda: distinct[0] <= k, add, rem)
```

Note. distinct moves only when a count crosses $0 \leftrightarrow 1$ – that boundary bookkeeping is the whole problem. $k = 0$ forces the window empty forever: answer 0. $O(n)$.

14.8 Longest run within budget

apply

Task durations stream in (non-negative). What is the longest run of CONSECUTIVE tasks whose total fits in a time budget? Window state: the running sum.

`longest_within_budget(xs: list[int], budget: int) -> int`

```
>>> longest_within_budget([2, 1, 3, 4], 6)
3
>>> longest_within_budget([5], 3)
0
>>> longest_within_budget([], 10)
0
>>> longest_within_budget([1, 1, 1, 1], 2)
2
```

Solution.

```
def longest_within_budget(xs, budget):
    total = [0]
    def add(x): total[0] += x
    def rem(x): total[0] -= x
    return longest_window(xs, lambda: total[0] <= budget, add, rem)
```

Note. Shrinking is only sound because durations are non-negative (removing can never make the sum worse) – say that invariant out loud; with negatives this window logic breaks.

14.9 Island perimeter

apply

One island of '1' cells sits in a '0' sea. Its perimeter is the number of unit edges touching water or the map border. Count per land cell: 4 minus its LAND neighbours (off-grid sides count as water automatically, since neighbors4 never yields them).

```
island_perimeter(grid: list[str]) -> int
>>> island_perimeter(['0100', '1110', '0100'])
12
>>> island_perimeter(['1'])
4
>>> island_perimeter(['11'])
6
>>> island_perimeter(['00'])
0
```

Solution.

```
def island_perimeter(grid):
    R, C = len(grid), len(grid[0])
    return sum(4 - sum(grid[nr][nc] == '1' for nr, nc in neighbors4(r, c, R, C))
               for r in range(R) for c in range(C) if grid[r][c] == '1')
```

Note. Each land cell contributes 4 edges minus one per land neighbour; the bounds-checked generator makes border edges fall out for free. $O(R \cdot C)$, no flood needed.

14.10 Flood clock

apply

Water enters an open grid (‘ floor, ‘#’ wall) at the top-left and spreads one orthogonal step per minute. Return the minute the LAST reachable floor cell gets wet (0 if only the entrance), or -1 if the entrance itself is a wall.

```
flood_minutes(grid: list[str]) -> int
>>> flood_minutes(['..', '..'])
2
>>> flood_minutes(['.'])
0
>>> flood_minutes(['#'])
-1
>>> flood_minutes(['.#.', '...', '.#'])
4
```

Solution.

```
def flood_minutes(grid):
    R, C = len(grid), len(grid[0])
    if grid[0][0] == '#':
        return -1
    dist = {(0, 0): 0}
    q = deque([(0, 0)])
    while len(q):
        r, c = q.popleft()
        for nr, nc in neighbors4(r, c, R, C):
            if grid[nr][nc] == '.' and (nr, nc) not in dist:
                dist[(nr, nc)] = dist[(r, c)] + 1
                q.append((nr, nc))
    return max(dist.values())
```

Note. BFS eccentricity: the dist map doubles as the seen-set, and the answer is its maximum rather than a target lookup – the same machine as shortest-path, read differently.

14.11 Best uptime with k repairs

challenge

A service log is a list of 1s (up) and 0s (down). You may retroactively excuse at most k downtimes. What is the longest stretch that then reads fully up? Classic interview form: longest window containing at most k zeros.

```
max_up_with_repairs(xs: list[int], k: int) -> int
>>> max_up_with_repairs([1, 1, 0, 0, 1], 1)
3
>>> max_up_with_repairs([1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0], 2)
```

```

6
>>> max_up_with_repairs([1, 1, 0, 1], 0)
2
>>> max_up_with_repairs([], 3)
0

```

Solution.

```

def max_up_with_repairs(xs, k):
    zeros = [0]
    def add(x): zeros[0] += x == 0
    def rem(x): zeros[0] -= x == 0
    return longest_window(xs, lambda: zeros[0] <= k, add, rem)

```

Note. LeetCode 1004 in window form: track zeros in the window, legal while $\leq k$. The insight worth voicing: you never 'use' repairs – the window just tolerates k zeros. $O(n)$.

14.12 Rotting oranges

challenge

A crate map holds 0 (empty), 1 (fresh), 2 (rotten). Every minute, fresh oranges next to a rotten one (orthogonally) rot. Return the minutes until nothing fresh remains, or -1 if some fresh orange can never be reached. Multi-source BFS: seed the queue with EVERY rotten orange at minute 0.

`orange_minutes(grid: list[list[int]]) -> int`

```

>>> orange_minutes([[2, 1, 1], [1, 1, 0], [0, 1, 1]])
4
>>> orange_minutes([[2, 1, 1], [0, 1, 1], [1, 0, 1]])
-1
>>> orange_minutes([[0, 2]])
0
>>> orange_minutes([[1]])
-1

```

Solution.

```

def orange_minutes(grid):
    R, C = len(grid), len(grid[0])
    rotten = [(r, c) for r in range(R) for c in range(C) if grid[r][c] == 2]
    q = deque([(r, c, 0) for r, c in rotten])
    seen = set(rotten)
    fresh = sum(row.count(1) for row in grid)
    minutes = 0
    while len(q):
        r, c, m = q.popleft()
        minutes = max(minutes, m)
        for nr, nc in neighbors4(r, c, R, C):
            if grid[nr][nc] == 1 and (nr, nc) not in seen:
                seen.add((nr, nc))
                fresh -= 1
                q.append((nr, nc, m + 1))
    return -1 if fresh else minutes

```

Note. Multi-source BFS is ordinary BFS with several starting points enqueued at time 0; the distance of the last fresh orange is the answer. Count fresh up front so unreachable ones are detectable – the classic miss is returning minutes without that check.

Chapter 15

Control and Dynamic Programming

The capstone machinery. `memo` turns an honest recurrence into dynamic programming – state the truth, decorate it, and the cache is your DP table, inspectable at `.cache`; the craft is naming the state so “the future only needs X ” comes out in one sentence. `until` handles the loops with no list to fold: iterate a step to a goal or a fixed point, Bellman–Ford being the textbook case.

15.1 Concepts

15.1.1 `memo`

an unbounded memoizer as a plain decorator (no parentheses): results are cached by the argument tuple, so each distinct call computes once. Keyword arguments fold into the key; the cache is exposed as `.cache`.

```
>>> @memo
... def fib(n): return n if n < 2 else fib(n - 1) + fib(n - 2)
>>> fib(30)
832040
>>> fib.cache[(10,)]
55
>>> len(fib.cache)
31                                     # one entry per distinct argument
```

How it works: the decorator wraps `f` with a dict keyed on the arguments. First call computes and stores; every repeat is a lookup. That is the entire mechanism – and why arguments must be `HASHABLE`: lists become tuples before they can be keys.

Why it earns its place: memoized recursion IS dynamic programming. Write the honest recurrence – the answer for state (i, j) in terms of smaller states – decorate it, and the cache is your DP table, filled in dependency order with no index bookkeeping. The exposed `.cache` lets you SEE the table: its size is the state count, exactly the number complexity analysis wants said out loud.

```
>>> @memo
... def ways(r, c):
...     return 1 if r == 0 or c == 0 else ways(r - 1, c) + ways(r, c - 1)
>>> ways(3, 3)
20
>>> len(ways.cache)
15                                     # the 4x4 grid minus (0,0), never needed
```

Versus `functools.lru_cache`: the standard library’s version adds eviction (`maxsize`); `memo` is deliberately unbounded, because DP wants every state kept – and it is nine lines you can retype in any interview that bans imports.

Remember it as: recursion states the truth; the cache makes it affordable; `.cache` is the table.

Roll your own: a dict captured in a closure, keyed by the argument tuple (keyword arguments folded in as a `frozenset` when present – tuples and `frozensets` because keys must be hashable). Compute on miss, look up on hit, and hang the dict on the wrapper as `.cache` so the DP table stays inspectable.

```
def memo(f): # unbounded memoizer; enough for DP (no eviction by design)
    cache = {}
```

```
def wrapped(*args, **kw):
    key = (args, frozenset(kw.items())) if kw else args
    if key not in cache:
        cache[key] = f(*args, **kw)
    return cache[key]
wrapped.cache = cache # peek at the DP table if curious
return wrapped
```

15.1.2 until

iterate f from x until the predicate holds, returning the first satisfying value: a fixed-point loop with no recursion-depth limit. The state can be a tuple carrying whatever the loop must remember.

```
>>> until(lambda x: x > 100, lambda x: x * 2, 1)
128
>>> collatz = lambda s: (s[0] // 2 if even(s[0]) else 3 * s[0] + 1, s[1] + 1)
>>> until(lambda s: s[0] == 1, collatz, (6, 0)) # 8 steps to reach 1
(1, 8)
```

How it works: keep applying f until p approves – iteration to a goal or a FIXED POINT, as a loop rather than recursion, so depth is unbounded. The state can be anything; a tuple lets one loop carry several facts at once, like the collatz example's (value, steps).

Why it earns its place: some processes have no list to fold over – they run "until stable". Newton's isqrt converges; bellman_ford relaxes every edge until the distance map stops changing (the textbook fixed point; g7's hint calls it by name). until states such loops declaratively: here is the step, here is done.

```
>>> until(lambda n: n * n >= 30, succ, 0) # smallest n with n squared >= 30
6
>>> until(lambda xs: xs == sorted(xs), sorted, [3, 1, 2])
[1, 2, 3]
```

Remember it as: while-not-done, promoted to an expression that returns its answer.

Roll your own: while not $p(x)$: $x = f(x)$; return x . Iteration promoted to a value-returning expression – and a loop rather than recursion on purpose, because a fixed point may take unboundedly many steps.

```
def until(cond, f, x): # until cond f x -- loop, not recursion (unbounded depth)
    while not cond(x):
        x = f(x)
    return x
```

15.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

15.1 Tribonacci

drill

The tribonacci sequence starts 0, 1, 1 and every later term is the sum of the previous THREE. Write `trib(n)` with @memo so large n is instant. State the recurrence before coding.

`trib(n: int) -> int`

```
>>> trib(0), trib(1), trib(2)
(0, 1, 1)
>>> trib(4)
4
>>> trib(10)
149
>>> trib(25)
1389537
```

Solution.

```
@memo
def trib(n):
    return n if n < 2 else 1 if n == 2 else trib(n - 1) + trib(n - 2) + trib(n - 3)
```

Note. Direct recurrence plus memo; $O(n)$ states, $O(1)$ work each. Without memo this is $O(3^n)$ – the whole lesson in one decorator.

15.2 First power of two

drill

Using `until`, return the smallest power of two that is $\geq n$ ($n \geq 1$). No loops of your own: state the stop predicate and the step, let `until` iterate.

```
first_pow2(n: int) -> int
```

```
>>> first_pow2(1)
1
>>> first_pow2(100)
128
>>> first_pow2(128)
128
>>> first_pow2(1025)
2048
```

Solution.

```
first_pow2 = lambda n: until(lambda x: x >= n, lambda x: 2 * x, 1)
```

Note. `until(stop, step, seed)` is a while loop as an expression. $O(\log n)$ doublings.

15.3 Digital root

drill

The digital root of n repeatedly replaces n with the sum of its digits until a single digit remains. Write it with `until` – the fixed point is 'already one digit'.

```
digital_root(n: int) -> int
```

```
>>> digital_root(0)
0
>>> digital_root(7)
7
>>> digital_root(9875)
2
>>> digital_root(99999)
9
```

Solution.

```
digital_root = lambda n: until(lambda x: x < 10, lambda x: sum(map(int, str(x))), n)
```

Note. Stop predicate $x < 10$; the step is one digit-sum. Each step shrinks the number fast, so only a handful of iterations ever run.

15.4 Staircase, three strides

drill

You climb a staircase of n steps taking 1, 2 or 3 steps at a time. Count the ways to reach the top (distinct ordered stride sequences) with a memoized recursion; `ways(0)` is 1 (stand still), negative overshoot is 0.

```
steps3(n: int) -> int
```

```
>>> steps3(0)
1
>>> steps3(1)
1
>>> steps3(4)
7
>>> steps3(10)
274
```

Solution.

```
@memo
def steps3(n):
    if n == 0:
        return 1
    if n < 0:
        return 0
    return steps3(n - 1) + steps3(n - 2) + steps3(n - 3)
```

Note. The two base cases ARE the specification: one way to be done, no way to overshoot. $O(n)$ states.

15.5 Reading the DP table

drill

Write `fib` with `@memo` inside a helper, compute `fib(n)`, and return the pair `(fib(n), number_of_cached_states)`. The cache IS the DP table; its size is the state count that complexity analysis asks you to say out loud.

`fib_with_table(n: int) -> tuple[int, int]`

```
>>> fib_with_table(0)
(0, 1)
>>> fib_with_table(10)
(55, 11)
>>> fib_with_table(20)
(6765, 21)
```

Solution.

```
def fib_with_table(n):
    @memo
    def fib(k):
        return k if k < 2 else fib(k - 1) + fib(k - 2)
    value = fib(n)
    return (value, len(fib.cache))
```

Note. `memo` exposes `.cache` precisely so the table is inspectable: $n+1$ states for `fib(n)`. Defining the helper inside gives every call a fresh table.

15.6 House robber

apply

A row of houses holds these amounts of cash. You cannot rob two ADJACENT houses. Return the maximum total you can take. Classic interview DP: at each house, rob it and skip a neighbour, or walk past.

`rob(xs: list[int]) -> int`

```
>>> rob([])
0
>>> rob([5])
5
>>> rob([1, 2, 3, 1])
4
>>> rob([2, 7, 9, 3, 1])
12
```

Solution.

```
def rob(xs):
    @memo
    def go(i):
        if i >= len(xs):
            return 0
        return max(go(i + 1), xs[i] + go(i + 2))
    return go(0)
```

Note. State: index; choice: take (jump 2) or skip (jump 1). $O(n)$ states. The classic mistake is greedy on the largest house – `[2,7,9]` breaks it.

15.7 Decode ways

apply

A message of letters A-Z was encoded as digits (A=1 .. Z=26) and concatenated. Count the ways to decode a digit string: '12' is 2 ('AB' or 'L'). A leading zero at any decision point kills that branch; the empty string counts as one way.

```
decode_ways(s: str) -> int
```

```
>>> decode_ways('')
1
>>> decode_ways('12')
2
>>> decode_ways('226')
3
>>> decode_ways('06')
0
>>> decode_ways('11106')
2
```

Solution.

```
def decode_ways(s):
    @memo
    def go(i):
        if i == len(s):
            return 1
        if s[i] == '0':
            return 0
        two = go(i + 2) if i + 1 < len(s) and 10 <= int(s[i:i + 2]) <= 26 else 0
        return go(i + 1) + two
    return go(0)
```

Note. State: position; two branches (one digit, two digits) guarded by the 10..26 window and the zero rule. O(n) states; the '0' guard is where most attempts die.

15.8 Jump game

apply

xs[i] is the maximum jump length from index i. Starting at index 0, can you reach the last index? Solve it as one fold over enum(xs) carrying the furthest reachable index – poison the accumulator when an index is unreachable.

```
can_jump(xs: list[int]) -> bool
```

```
>>> can_jump([0])
True
>>> can_jump([2, 3, 1, 1, 4])
True
>>> can_jump([3, 2, 1, 0, 4])
False
>>> can_jump([1, 0, 1])
False
```

Solution.

```
def can_jump(xs):
    def step(far, ix):
        i, x = ix
        return -1 if far < 0 or far < i else max(far, i + x)
    return foldl(step, enum(xs), 0) >= 0
```

Note. Greedy reach as a fold; -1 is the poison value that propagates (the g9 trick on a number). O(n), no DP table needed – reach is monotone.

15.9 Perfect squares

apply

Return the fewest perfect squares (1, 4, 9, 16, ...) summing exactly to n. 12 -> 3 (4+4+4), 13 -> 2 (4+9). Memoize on the remaining amount; isqrt bounds the candidates.

```
num_squares(n: int) -> int
```

```
>>> num_squares(0)
0
>>> num_squares(1)
1
>>> num_squares(12)
3
>>> num_squares(13)
2
>>> num_squares(7)
4
```

Solution.

```
def num_squares(n):
    @memo
    def go(m):
        if m == 0:
            return 0
        return min(1 + go(m - k * k) for k in range(1, isqrt(m) + 1))
    return go(n)
```

Note. Coin change with square coins; `isqrt(m)` trims the branch factor. $O(n * \sqrt{n})$. Greedy (largest square first) fails on 12.

15.10 Growth to target

apply

A town of `pop` people grows `rate` percent per year (integer floor) and then gains influx newcomers. Using `until` with a (population, years) state tuple, return how many whole years until the population reaches `target` (0 if already there).

```
years_to_target(pop: int, rate: int, influx: int, target: int) -> int
```

```
>>> years_to_target(500, 5, 10, 400)
0
>>> years_to_target(100, 10, 0, 200)
8
>>> years_to_target(1, 0, 10, 51)
5
```

Solution.

```
def years_to_target(pop, rate, influx, target):
    grow = lambda s: (s[0] * (100 + rate) // 100 + influx, s[1] + 1)
    return snd(until(lambda s: s[0] >= target, grow, (pop, 0)))
```

Note. The tuple state carries the counter alongside the value – `until`’s standard idiom. Integer arithmetic keeps the doctests exact.

15.11 Longest palindromic subsequence

challenge

Return the length of the longest subsequence of `s` (not necessarily contiguous) that reads the same forwards and backwards. 'bbbab' -> 4 ('bbbb'). Memoize on the (i, j) window: matching ends extend the palindrome by two; otherwise drop one end.

```
lps(s: str) -> int
```

```
>>> lps('')
0
>>> lps('a')
1
>>> lps('bbbab')
4
>>> lps('cbbd')
2
>>> lps('abcdef')
1
```

Solution.

```
def lps(s):
    @memo
    def go(i, j):
        if i > j:
            return 0
        if i == j:
            return 1
        if s[i] == s[j]:
            return 2 + go(i + 1, j - 1)
        return max(go(i + 1, j), go(i, j - 1))
    return go(0, len(s) - 1) if s else 0
```

Note. The lcs skeleton on one string against itself, expressed as a shrinking window. $O(n^2)$ states; the $i == j$ base (a single char is a palindrome) is the easy one to forget.

15.12 Word break

challenge

Can s be segmented into a sequence of words from the given list (words reusable)? 'leetcode' with ['leet', 'code'] -> True. Memoize on the start index; isPrefixOf tests each candidate word against the remaining suffix.

word_break(s: str, words: list[str]) -> bool

```
>>> word_break('', ['a'])
True
>>> word_break('leetcode', ['leet', 'code'])
True
>>> word_break('applepenapple', ['apple', 'pen'])
True
>>> word_break('catsandog', ['cats', 'dog', 'sand', 'and', 'cat'])
False
>>> word_break('aaaaaaa', ['aaa', 'aaaa'])
True
```

Solution.

```
def word_break(s, words):
    ws = tuple(words)
    @memo
    def go(i):
        if i == len(s):
            return True
        return any(isPrefixOf(w, s[i:]) and go(i + len(w)) for w in ws)
    return go(0)
```

Note. State: start index – the future only needs where the unbroken suffix begins. any() short-circuits; memo caps the work at $O(\text{len}(s) * \text{len}(\text{words}))$ prefix checks.

Chapter 16

Combinators Capstone

The functions about functions come **FIRST** in the file and **LAST** in a person: you have been using them all course, and now they get named as a discipline. **curry** and **partial** stage arguments; **flip** and **on** adapt argument order and route comparisons through keys; **compose** makes a pipeline a value. The capstone skill is judgement – writing point-free when it clarifies, and refusing to when it does not.

16.1 Concepts

16.1.1 id

the identity function: returns its argument unchanged. Sounds useless; is everywhere. It is the "do nothing" you pass where a transformation is required – a sort key meaning "the value itself", the seed of **compose**.

```
>>> id(42)
42
>>> sortOn(id, [3, 1, 2])
[1, 2, 3]
```

Roll your own — the definition:

```
id = lambda x: x # shadows builtin id() by design
```

16.1.2 const

freezes a value into a function: **const(x)** returns a function that ignores its argument and always answers x. Use it to fill a callback slot with a constant.

```
>>> five = const(5)
>>> five("anything"), five(99)
(5, 5)
>>> map_(const('*'), [1, 2, 3])
['*', '*', '*']
```

Roll your own — the definition:

```
const = lambda x: lambda _: x # always returns x, ignoring the second argument
```

16.1.3 flip

swaps the two arguments of a function. An adapter: the function you have takes (a, b), the slot you are filling supplies (b, a). This is how the prelude derives **foldr** from **foldl**.

```
>>> sub(10, 1), flip(sub)(10, 1)
(9, -9)
>>> flip(pow)(3, 2) # pow(2, 3)
8
```

How it works: `flip(f)` returns a new function that hands its two arguments to `f` in the opposite order. Nothing is computed at flip time – it is pure plumbing, applied when the data arrives in the "wrong" order for the function you already have.

```
>>> pair = lambda a, b: (a, b)
>>> flip(pair)(1, 2)
(2, 1)
```

Why it earns its place: the prelude's own `foldr` is the showcase. `foldl`'s combiner takes (accumulator, element); a right fold's combiner takes (element, accumulator). Rather than build a second fold machine, `foldr = foldl` of the FLIPPED combiner over the reversed list – one adapter turned an existing engine into its mirror image.

```
>>> f = lambda x, acc: '(' + x + acc + ')'
>>> foldr(f, "abc", "*") == foldl(flip(f), list(reversed("abc")), "*")
True
```

Remember it as: when argument order is the only thing wrong, adapt – don't rewrite.

Roll your own — the definition:

```
flip = lambda f: (lambda x, y: f(y, x)) # flip f x y = f y x
```

16.1.4 curry

splits a two-argument function into two one-argument stages, so you can supply the first argument now and the second later.

```
>>> curry(add)(1)(2)
3
>>> add_ten = curry(add)(10)
>>> add_ten(5)
15
```

How it works: read the three lambdas inside-out, then watch a call peel one layer at a time. Each call binds ONE argument by closure – after `curry(add)(10)`, the 10 lives in the returned function's captured environment, and that half-applied function is a value you can name, store, and pass.

```
>>> curry(add)(10)(5)
15
>>> add_ten = curry(add)(10)
>>> add_ten(5), add_ten(90)
(15, 100)
```

Why it earns its place: the prelude's pipelines – `map_`, `filter_`, `find`, `compose` – all want ONE-argument functions in their function slot, while real logic usually takes two ("multiply by k", "is x greater than n?"). Currying is the adapter: close over the configuration argument now, hand the pipeline the unary stage it wants.

```
>>> gt = curry(lambda n, x: x > n)
>>> filter_(gt(10), [4, 11, 8, 40])
[11, 40]
>>> scale = curry(lambda k, x: k * x)
>>> map_(scale(3), [1, 2, 3])
[3, 6, 9]
```

The Haskell connection: in Haskell EVERY function is curried automatically – `add :: Int -> Int -> Int` really means `Int -> (Int -> Int)`, so `add 10` is ordinary code and `map (add 10) xs` falls out for free. Every arrow in a Haskell type is another layer of this doll.

Versus `partial`: both bind early. `partial` binds any number of arguments, all at once, when you have them; `curry` restructures the function so binding happens one call at a time, decided later by whoever holds it. In Python, `partial` is the cheaper everyday tool; `curry` is the pipeline-shaper and the reading key for Haskell signatures.

Remember it as: `curry` turns one two-argument function into two one-argument functions – and one-argument functions are the only thing pipelines eat.

Roll your own: three lambdas, each returning the next – every layer is a closure capturing one more argument until the innermost body finally has both. No state, no storage: the captured environment IS the data structure.

```
curry = lambda f: lambda x: lambda y: f(x, y)
```

16.1.5 uncurry

the reverse adaptation: a two-argument function becomes a one-argument function of a PAIR. Reach for it when your data is already (a, b) tuples and your function is not.

```
>>> uncurry(add)((1, 2))
3
>>> map_(uncurry(lambda a, b: a * b), [(2, 3), (4, 5)])
[6, 20]
```

Roll your own — the definition:

```
uncurry = lambda f: lambda p: f(*p) # inverse of curry: call f with a pair, not two args
```

16.1.6 partial

freezes any number of leading arguments, returning a function that takes the rest. Like curry but takes the arguments now, all at once.

```
>>> partial(pow, 2)(10) # pow(2, 10)
1024
>>> clamp_floor = partial(max, 0) # "never below zero"
>>> clamp_floor(-5), clamp_floor(7)
(0, 7)
```

Roll your own — the definition:

```
partial = lambda f, *bound: lambda *args: f(*bound, *args) # fix leading args of f, wait for the rest
```

16.1.7 on

builds a two-argument function that first sends BOTH arguments through a key function: $\text{on}(f, \text{key})(x, y) == f(\text{key}(x), \text{key}(y))$. Its natural habitat is comparators: compare two records by one field.

```
>>> on(add, len)("ab", "c") # len("ab") + len("c")
3
>>> same_length = on(lambda a, b: a == b, len)
>>> same_length("abc", "xyz"), same_length("ab", "abc")
(True, False)
>>> sortBy(on(sub, snd), [(1, 9), (2, 3)])
[(2, 3), (1, 9)]
```

How it works: $\text{on}(f, \text{key})$ builds a two-argument function that routes BOTH inputs through key before combining them with f: $\text{on}(f, \text{key})(x, y) == f(\text{key}(x), \text{key}(y))$. Combination logic and projection logic stay separate, so each is reusable on its own.

```
>>> shorter = on(lambda a, b: a < b, len)
>>> shorter("ab", "abc"), shorter("abc", "ab")
(True, False)
```

Why it earns its place: comparator factories. Any "compare records by one field" collapses to $\text{on}(\text{compare}, \text{field})$ – Haskell's `sortBy` (compare 'on' `snd`) is exactly `sortBy(on(cmp, snd))` here, and reads as its own documentation.

```
>>> sortBy(on(sub, snd), [('a', 9), ('b', 1), ('c', 5)])
[('b', 1), ('c', 5), ('a', 9)]
```

Versus `sortOn`: when a per-element key exists, `sortOn(key, xs)` is simpler and faster (the key runs once per element). `on` shines when an API demands a genuine two-argument comparator, or when the combining function is something other than comparison.

Remember it as: compare the keys, not the things.

Roll your own — the definition:

```
on = lambda f, g: lambda x, y: f(g(x), g(y)) # Data.Function: combine via a key -- cmp `on` fst
```

16.1.8 compose

fold any number of functions into one; the RIGHTMOST runs first, like mathematical composition $f(g(x))$.

```
>>> compose(str.strip, str.upper)(" hi ")
'HI'
>>> f = compose(succ, succ, succ)
>>> f(0)
3
```

How it works: a fold over the functions themselves – neighbours merge into $\lambda x: f(g(x))$, seeded with the identity, so `compose()` with no arguments is `id`. The RIGHTMOST function runs first, matching mathematical $(f \cdot g)(x) = f(g(x))$.

```
>>> compose()(42) == id(42)
True
>>> compose(len, str.strip)(" hi ")      # strip first, then len
2
```

Why it earns its place: it turns a pipeline into a VALUE. A composed function can be named, mapped, stored in a rules table – processing steps become data you can shuffle. The constraint is the one curry answers: every stage must be unary, which is why `compose`-heavy code leans on `curry` and `partial` to shape its stages.

```
>>> normalize = compose(str.lower, str.strip)
>>> map_(normalize, [" Hi ", " YO "])
['hi', 'yo']
```

Remember it as: `foldr (.) id` – a pipeline you can hold in your hand.

Roll your own: it IS `foldr (.) id` – `foldr` over the functions with `x` as the seed, each stage applied to the accumulated result: `foldr(lambda f, acc: f(acc), fns, x)`. Because the prelude's `foldr` is an iterative loop underneath, stack depth stays constant no matter how many stages you compose.

```
compose = lambda *fns: lambda x: foldr(lambda f, acc: f(acc), fns, x) # foldr (.) id: RIGHTMOST first
```

16.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

16.1 Compose three

drill

Use `compose` to build a single function that strips a string, lowercases it, then reverses it – in that order of effect. Remember `compose` runs its RIGHTMOST function first, so the last stage you write is the first that happens.

```
clean_reverse: str -> str
```

```
>>> clean_reverse(' Hello ')
'olleh'
>>> clean_reverse('ABC')
'cba'
>>> clean_reverse('')
''
```

Solution.

```
clean_reverse = compose(lambda s: s[::-1], str.lower, str.strip)
```

Note. `compose(f, g, h)(x) == f(g(h(x)))`: strip happens first because it is rightmost. Point-free: no lambda parameter of your own.

16.2 Flip subtract

drill

The prelude's `sub(a, b) = a - b`. Use `flip` to build a function that subtracts the FIRST argument from the second. `flip` swaps argument order and nothing else.

```
rsub: (int, int) -> int (rsub(a, b) == b - a)
```

```
>>> rsub(1, 10)
9
>>> rsub(10, 1)
-9
>>> rsub(5, 5)
0
```

Solution.

```
| rsub = flip(sub)
```

Note. flip is a pure adapter – it computes nothing, just reorders. This is exactly how the prelude derives foldr from foldl.

16.3 Add via curry

drill

The prelude's `add(a, b) = a + b`. Curry it, then use the curried form to build `add_ten`, a one-argument function that adds 10. Show that binding the first argument yields a reusable function.

```
add_ten: int -> int
```

```
>>> add_ten(5)
15
>>> add_ten(-10)
0
>>> add_ten(0)
10
```

Solution.

```
| add_ten = curry(add)(10)
```

Note. `curry(add)(10)` captures 10 in a closure and returns a function still waiting for the second argument. That half-applied function is a value you can name.

16.4 Constant column

drill

Use `const` and `map_` to replace every element of a list with the string 'x', regardless of its value. `const(v)` is the function that ignores its argument and always answers `v`.

```
mask(xs: list) -> list[str]
```

```
>>> mask([1, 2, 3])
['x', 'x', 'x']
>>> mask([])
[]
>>> mask(['a', None, 7])
['x', 'x', 'x']
```

Solution.

```
| mask = lambda xs: map_(const('x'), xs)
```

Note. `const` fills a callback slot with a constant. Reaching for a lambda `z: 'x'` would work too, but `const` names the intent.

16.5 Partial power

drill

Use `partial` to freeze the base of `pow`, producing `powers_of_two(exp) == 2 ** exp`. `partial(f, *leading)` fixes the first arguments now and takes the rest later.

```
powers_of_two: int -> int
```

```
>>> powers_of_two(0)
1
>>> powers_of_two(10)
1024
>>> powers_of_two(5)
32
```

Solution.

```
| powers_of_two = partial(pow, 2)
```

Note. `partial` binds any number of leading args in one shot; `curry` binds them one call at a time. Here one frozen base is all we need.

16.6 Sort records by a field*apply*

You have (name, age) tuples. Sort them oldest-first, using `on` to build the comparator from a plain numeric compare and the `snd` accessor, fed through `sortBy`. `on(f, key)(x, y)` routes BOTH arguments through `key` before `f`.

```
by_age_desc(people: list[tuple]) -> list[tuple]
```

```
>>> by_age_desc([('ada', 36), ('bob', 41), ('cy', 19)])
[('bob', 41), ('ada', 36), ('cy', 19)]
>>> by_age_desc([])
[]
>>> by_age_desc([('x', 5)])
[('x', 5)]
```

Solution.

```
| by_age_desc = lambda people: sortBy(on(flip(sub), snd), people)
```

Note. `on` separates the projection (`snd`) from the comparison (`flip(sub)`, `b - a`, for descending). This is Haskell's 'comparing' idiom; a per-element key with `sortOn` would also work here.

16.7 Traffic-light rules table*apply*

Classify an integer as 'neg', 'zero', or 'pos' using a RULES table: a list of (predicate, result) pairs tried in order, with `find` picking the first match. The catch-all arm is `const(otherwise)` – since table guards are APPLIED to the value, the always-true guard must itself be a function of the value.

```
classify(n: int) -> str
```

```
>>> classify(-4)
'neg'
>>> classify(0)
'zero'
>>> classify(9)
'pos'
>>> map_(classify, [-1, 0, 1])
['neg', 'zero', 'pos']
```

Solution.

```
| RULES = [(lambda n: n < 0, const('neg')),
|          (lambda n: n == 0, const('zero')),
|          (const(otherwise), const('pos'))]
| classify = lambda n: snd(find(lambda rule: fst(rule)(n), RULES))(n)
```

Note. An if/elif ladder rewritten as data: rules become appendable and testable in isolation. `const(otherwise)` is the always-true arm; a bare `True` would crash when applied.

16.8 Pipeline from a list of steps*apply*

Given a list of one-argument functions, build a single function that runs them left-to-right (first in the list runs first) and apply it to `x`. Note this is the OPPOSITE order from `compose`, so reverse before composing.

```
pipe(steps: list, x)
```

```
>>> pipe([lambda a: a + 1, lambda a: a * 2], 5)
12
>>> pipe([], 7)
7
>>> pipe([str.strip, str.upper], ' hi ')
```

```
| 'HI'
```

Solution.

```
| pipe = lambda steps, x: compose(*reversed(steps))(x)
```

Note. `compose` is right-to-left; a left-to-right pipeline just reverses the list first. The empty pipeline is the identity, which `compose()` already gives you.

16.9 Group-and-count, point-free key

apply

Count how many words fall into each length bucket. Build the (key, value) pairs with the length as key and 1 as value, then fold them into a dict with `fromListWith` and addition. Use `partial` or a small helper to make the pair-builder read cleanly.

```
length_histogram(words: list[str]) -> dict[int, int]
```

```
>>> length_histogram(['a', 'bb', 'cc', 'ddd'])
{1: 1, 2: 2, 3: 1}
>>> length_histogram([])
{}
>>> length_histogram(['x', 'y', 'z'])
{1: 3}
```

Solution.

```
| pair = lambda w: (len(w), 1)
| length_histogram = lambda words: fromListWith(add, map(pair, words))
```

Note. `fromListWith` is the dict-building fold; addition is commutative so the new/old order is irrelevant here. $O(n)$.

16.10 Equality up to a key

apply

Write `same_by(key, a, b)`: true when `a` and `b` are equal AFTER passing through `key`. Build it with `on` so the projection lives in one place. Then a doctest shows it grouping case-insensitively.

```
same_by(key, a, b) -> bool
```

```
>>> same_by(str.lower, 'ABC', 'abc')
True
>>> same_by(len, 'ab', 'cd')
True
>>> same_by(len, 'ab', 'c')
False
>>> groupBy(partial(same_by, str.lower), ['a', 'A', 'b'])
[['a', 'A'], ['b']]
```

Solution.

```
| same_by = lambda key, a, b: on(lambda x, y: x == y, key)(a, b)
```

Note. `on(eq, key)` is the reusable 'equal through a projection'. `partial(same_by, str.lower)` then feeds `groupBy` a two-argument equivalence – combinators composing with the containers.

16.11 Compose a validation pipeline

challenge

Each validator is a function that returns `None` on success or an error string on failure. Run a list of validators over a value and return the FIRST error, or `None` if all pass. Build the check with `map__` and `find`; a value passes when `find` turns up no error.

```
first_error(validators: list, x)
```

```
>>> pos = lambda n: None if n > 0 else 'not positive'
>>> even_v = lambda n: None if n % 2 == 0 else 'not even'
>>> first_error([pos, even_v], 4) is None
True
>>> first_error([pos, even_v], -3)
'not positive'
>>> first_error([pos, even_v], 3)
```

```
'not even'
>>> first_error([], 0) is None
True
```

Solution.

```
first_error = lambda validators, x: find(lambda e: e is not None,
                                         map_(lambda v: v(x), validators))
```

Note. `map_` runs every validator, `find` returns the first non-`None` (the first failure), else `None`. The `None-is-Nothing` convention makes 'no error' and 'passed' the same value.

16.12 Curried lookup chain

challenge

Build `get_in(keys)` – a function that, given nested dicts, walks the key path and returns the leaf, or `None` if any key is missing. `get_in` is CURRIED: `get_in(['a', 'b'])` returns a function you can reuse on many dicts. Fold the keys with a safe step that stops at `None`.

`get_in(keys: list) -> (dict -> value | None)`

```
>>> deep = {'a': {'b': {'c': 42}}}
>>> get_in(['a', 'b', 'c'])(deep)
42
>>> get_in(['a', 'x'])(deep) is None
True
>>> get_in([])(deep) == deep
True
>>> pluck = get_in(['a', 'b'])
>>> pluck(deep)
{'c': 42}
```

Solution.

```
def get_in(keys):
    def step(d, k):
        return d[k] if isinstance(d, dict) and k in d else None
    return lambda d: foldl(step, keys, d)
```

Note. `get_in` returns a closure over keys – currying by hand – so a path becomes a reusable accessor. The fold threads the dict through each key, poisoning to `None` on any miss.

Part II

Applied Labs: The Nine Problem Classes

About the labs

Seventy-four problems, each a self-contained practice file in the repository (`fpython/g*/`) carrying its needed snippets inline. Statements, contracts, hints and doctests are printed here; solutions follow each problem in this edition.

Chapter 17

Arith And Unfolds

17.1 sign: Sign of a number

Return -1 for negative, 0 for zero, 1 for positive.

Contract:

`sign(x: int | float) -> int`

Hint:

The prelude ships this exact function: `signum`, one comparison subtraction, no branches. Recognize it, don't reinvent it.

```
>>> sign(-7)
-1
>>> sign(0)
0
>>> sign(3)
1
>>> sign(-0.5)
-1
```

Solution.

```
sign = signum
```

17.2 rect_area: Rectangle area

Contract:

`rect_area(w, h) -> w * h` (result type follows the inputs)

Hint:

Deliberate overkill: area is the numeric fold, `product([w, h])`.
See the smallest possible fold once; recognize every fold after.

```
>>> rect_area(3, 4)
12
>>> rect_area(2.5, 4)
10.0
>>> rect_area(0, 99)
0
```

Solution.

```
rect_area = lambda w, h: product([w, h])
```

17.3 reverse_digits: Reverse an integer's digits

Negatives keep their sign; trailing zeros vanish.

Contract:

```
reverse_digits(n: int) -> int
```

Hint:

unfoldr peels the digits with (m % 10, m // 10) -- least significant first, which IS reversed order. Fold them back into a number with acc * 10 + d; signum reapplies the sign.

```
>>> reverse_digits(123)
321
>>> reverse_digits(-45)
-54
>>> reverse_digits(120)
21
>>> reverse_digits(7)
7
```

Solution.

```
def reverse_digits(n):
    digits = unfoldr(lambda m: None if m == 0 else (m % 10, m // 10), abs(n))
    return signum(n) * foldl(lambda a, d: a * 10 + d, digits, 0)
```

17.4 to_base: Render n in base b

Non-negative n, base 2..16, uppercase digits.

Contract:

```
to_base(n: int, b: int) -> str
```

Hint:

unfoldr emits digits low-to-high -- "0123456789ABCDEF"[m % b], seed m // b, stop at 0. Reverse and join; ``or "0"`` covers zero (its unfold is empty).

```
>>> to_base(255, 16)
'FF'
>>> to_base(5, 2)
'101'
>>> to_base(0, 8)
'0'
>>> to_base(100, 10)
'100'
>>> to_base(31, 16)
'1F'
```

Solution.

```
DIGITS = "0123456789ABCDEF"
```

```
def to_base(n, b):
```

```
    ds = unfoldr(lambda m: None if m == 0 else (DIGITS[m % b], m // b), n)
    return "".join(reversed(ds)) or "0"
```

17.5 to_roman: Integer to Roman numeral, 1 ≤ n ≤ 3999

```

Contract:
    to_roman(n: int) -> str

Hint:
    A left fold over the value table (subtractive pairs included:
    900 'CM', 40 'XL', ...) carrying (remaining, out): each step
    appends sym * (remaining // val) and keeps remaining % val.

>>> to_roman(1994)
'MCMXCIV'
>>> to_roman(9)
'IX'
>>> to_roman(58)
'LVIII'
>>> to_roman(3000)
'MMM'
>>> to_roman(1)
'I'

```

Solution.

```

TABLE = [(1000, "M"), (900, "CM"), (500, "D"), (400, "CD"), (100, "C"),
          (90, "XC"), (50, "L"), (40, "XL"), (10, "X"), (9, "IX"),
          (5, "V"), (4, "IV"), (1, "I")]

def to_roman(n):
    step = lambda acc, vs: (acc[0] % vs[0], acc[1] + vs[1] * (acc[0] // vs[0]))
    return foldl(step, TABLE, (n, ""))[1]

```

17.6 from_roman: Parse a valid Roman numeral

```

Contract:
    from_roman(s: str) -> int

Hint:
    pairwise gives each symbol its right neighbor; a value counts
    NEGATIVE when its neighbor is bigger ('IV' = -1 + 5). Pad the
    value list with a trailing 0 so the last symbol gets a neighbor
    too, then sum.

>>> from_roman('MCMXCIV')
1994
>>> from_roman('IX')
9
>>> from_roman('LVIII')
58
>>> from_roman('III')
3
>>> from_roman('MMM')
3000

```

Solution.

```

VAL = {"I": 1, "V": 5, "X": 10, "L": 50, "C": 100, "D": 500, "M": 1000}

def from_roman(s):
    vs = map_(lambda c: VAL[c], s) + [0]
    return sum(map_(lambda p: -p[0] if p[0] < p[1] else p[0], pairwise(vs)))

```


Chapter 18

Lists And Strings

18.1 greet: Greeting string

```
Contract:
    greet(name: str) -> 'Hello, <name>!'

Hint:
    unwords joins words with single spaces; the bang concatenates.
    Trivial on purpose -- the warmup proves the pipeline.

>>> greet("Ada")
'Hello, Ada!'
>>> greet("world")
'Hello, world!'
```

Solution.

```
greet = lambda name: unwords(["Hello,", name]) + "!"
```

18.2 count_vowels: Count vowels, case-insensitive

```
Contract:
    count_vowels(s: str) -> int

Hint:
    len . filter_ (in "aeiou") . lower -- a filter consumed by len is
    the counting idiom.

>>> count_vowels("Programming")
3
>>> count_vowels("AEIOU")
5
>>> count_vowels("xyz")
0
>>> count_vowels("")
0
```

Solution.

```
count_vowels = lambda s: len(filter_(lambda c: c in "aeiou", s.lower()))
```

18.3 fizzbuzz: FizzBuzz as data

```
For 1..n: multiples of 3 -> "Fizz", of 5 -> "Buzz", of both ->
```

```
"FizzBuzz", else the number as a string.
```

Contract:

```
fizzbuzz(n: int) -> list[str]
```

Hint:

Solve it BOTH ways and compare -- the contrast is the lesson:

1. COMPOSITION OF ARMS: map_ one per-number rule over range(1, n+1), building the word by concatenating guards -- "Fizz" * (i % 3 == 0) + "Buzz" * (i % 5 == 0) -- with `or str(i)` catching the empty case. Overlaps compose: 15 earns "FizzBuzz" from two rules for free.
2. FIRST-MATCH SELECTION: a RULES table of (predicate, result) pairs, find picking the first hit; const(otherwise) is the catch-all arm. Overlaps need their own row (15 goes first), but the ladder is now DATA -- appendable, testable rule by rule. Which generalizes better depends on whether cases overlap.

```
>>> fizzbuzz(5)
['1', '2', 'Fizz', '4', 'Buzz']
>>> fizzbuzz(15)[-1]
'FizzBuzz'
>>> fizzbuzz(0)
[]
```

Solution.

```
# way 1 -- composition of arms: overlapping guards concatenate
fizzbuzz = lambda n: map_(
    lambda i: ("Fizz" * (i % 3 == 0) + "Buzz" * (i % 5 == 0)) or str(i),
    range(1, n + 1))

# way 2 -- first-match selection: the ladder as a RULES table (this one wins)
RULES = [(lambda i: i % 15 == 0, const("FizzBuzz")),
          (lambda i: i % 3 == 0, const("Fizz")),
          (lambda i: i % 5 == 0, const("Buzz")),
          (const(otherwise), str)]
speak = lambda i: snd(find(lambda rule: fst(rule)(i), RULES))(i)
fizzbuzz = lambda n: map_(speak, range(1, n + 1))
```

18.4 word_lengths: Map each word to its length

Contract:

```
word_lengths(sentence: str) -> dict[str, int]
First-appearance key order; repeats collapse harmlessly.
```

Hint:

words splits; zip_ the words against map_ len of them; dict() the pairs. Duplicate keys overwrite with the same value -- free dedup.

```
>>> word_lengths("the quick fox")
{'the': 3, 'quick': 5, 'fox': 3}
>>> word_lengths("go go go")
{'go': 2}
>>> word_lengths("")
{}
```

Solution.

```
word_lengths = lambda s: dict(zip_(words(s), map_(len, words(s))))
```

18.5 flatten: Flatten one level of nesting

```
Contract:
    flatten(lists: list[list]) -> list

Hint:
    This IS the prelude's concat. One name, zero work.

>>> flatten([[1, 2], [3], [4, 5]])
[1, 2, 3, 4, 5]
>>> flatten([], [1], [])
[1]
>>> flatten([])
[]
```

Solution.

```
flatten = concat
```

18.6 caesar: Caesar cipher, letters only, case preserved

```
Contract:
    caesar(s: str, k: int) -> str

Hint:
    map_ a per-char shift over the string; the wrap is mod 26 on
    ord() offsets; join the pieces. One helper, one map.

>>> caesar("abc", 2)
'cde'
>>> caesar("XYZ", 3)
'ABC'
>>> caesar("a-b", 1)
'b-c'
>>> caesar("Hello, World!", 13)
'Uryyb, Jbeyq!'
>>> caesar("abc", 0)
'abc'
```

Solution.

```
def caesar(s, k):
    def shift(c):
        if not c.isalpha():
            return c
        base = ord("A") if c.isupper() else ord("a")
        return chr(base + (ord(c) - base + k) % 26)
    return "".join(map_(shift, s))
```

18.7 contains_duplicate: NeetCode: Contains Duplicate

```
Contract:
    contains_duplicate(nums: list[int]) -> bool
    True iff any value appears at least twice.

Hint:
    nub keeps first occurrences; duplicates exist iff nub shrinks the
    list.

>>> contains_duplicate([1, 2, 3, 1])
```

```
True
>>> contains_duplicate([1, 2, 3, 4])
False
>>> contains_duplicate([])
False
```

Solution.

```
contains_duplicate = lambda nums: len(nub(nums)) != len(nums)
```

18.8 chunks: Split a list into pieces of length k

Last piece may be shorter.

Contract:

```
chunks(xs: list, k: int) -> list[list]
```

Hint:

```
splitAt already returns (front, rest) == (value, next_seed), so
chunks IS unfoldr of splitAt; null(rest) is the stop. One line.
(You are deriving the prelude's chunksOf -- deriving a tool once
is why it sticks.)
```

```
>>> chunks([1, 2, 3, 4, 5], 2)
[[1, 2], [3, 4], [5]]
>>> chunks([1, 2, 3, 4], 2)
[[1, 2], [3, 4]]
>>> chunks([], 3)
[]
>>> chunks([1], 5)
[[1]]
```

Solution.

```
chunks = lambda xs, k: unfoldr(
    lambda rest: None if null(rest) else splitAt(k, rest), list(xs))
```

18.9 permutations: All distinct orderings, sorted

Contract:

```
permutations(s: str) -> list[str]
```

Hint:

```
concatMap over the choice of first character: pick s[i], prepend
it to every permutation of the rest. nub kills duplicates from
repeated letters; sort at the end.
```

```
>>> permutations("abc")
['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
>>> permutations("aab")
['aab', 'aba', 'baa']
>>> permutations("x")
['x']
>>> permutations("")
['']
```

Solution.

```
def permutations(s):
    if len(s) <= 1:
        return [s]
```

```

opts = concatMap(
  lambda i: [s[i] + rest for rest in permutations(s[:i] + s[i + 1:]),
    range(len(s))]
  return sorted(nub(opts))

```

18.10 reverse_words: Reverse the word order

Runs of whitespace collapse (that's what a bare split does).

Contract:

```
reverse_words(s: str) -> str
```

Hint:

An unwords . reverse . words sandwich -- three names, no loop.

```

>>> reverse_words("the quick brown fox")
'fox brown quick the'
>>> reverse_words(" hello world ")
'world hello'
>>> reverse_words("solo")
'solo'

```

Solution.

```
reverse_words = lambda s: unwords(list(reversed(words(s))))
```

18.11 transpose_matrix: Flip rows and columns

Contract:

```
transpose_matrix(rows: list[list]) -> list[list]
```

Hint:

This IS the prelude's transpose -- like flatten was concat.
One name, zero work; know WHY zip(*rows) does it.

```

>>> transpose_matrix([[1, 2, 3], [4, 5, 6]])
[[1, 4], [2, 5], [3, 6]]
>>> transpose_matrix([[5]])
[[5]]
>>> transpose_matrix([[7], [8], [9]])
[[7, 8, 9]]

```

Solution.

```
transpose_matrix = transpose
```

18.12 rotate_right: Rotate a list right by k

k may be 0 or larger than the list.

Contract:

```
rotate_right(xs: list, k: int) -> list
```

Hint:

One splitAt at len - k%len, then glue the halves swapped.
The k=0 trap: xs[-0:] thinking sneaks in if you slice by -k.

```
>>> rotate_right([1, 2, 3, 4, 5], 2)
```

```
[4, 5, 1, 2, 3]
>>> rotate_right([1, 2], 0)
[1, 2]
>>> rotate_right([1, 2, 3], 5)
[2, 3, 1]
>>> rotate_right([], 3)
[]
```

Solution.

```
def rotate_right(xs, k):
    if not xs:
        return []
    a, b = splitAt(len(xs) - k % len(xs), list(xs))
    return b + a
```

Chapter 19

Folds Scans Streams

19.1 two_sum: NeetCode: Two Sum

Contract:

```
two_sum(nums: list[int], target: int) -> tuple[int, int]
Return indices (i, j), i < j, with nums[i] + nums[j] == target.
Exactly one solution exists. len(nums) >= 2.
```

Hint:

```
A fold over enum(nums) carrying a {value: index} dict.
The future needs to know: which values have I seen, and where?
```

```
>>> two_sum([2, 7, 11, 15], 9)
(0, 1)
>>> two_sum([3, 2, 4], 6)
(1, 2)
>>> two_sum([3, 3], 6)
(0, 1)
```

Solution.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enum(nums):
        if target - x in seen:
            return (seen[target - x], i)
    seen[x] = i
```

19.2 best_time_stock: NeetCode: Best Time to Buy and Sell Stock

Contract:

```
max_profit(prices: list[int]) -> int
One buy, one later sell. Best achievable profit, 0 if none.
len(prices) >= 1.
```

Hint:

```
scanl1 min gives the running minimum -- the best buy so far.
zipWith (-) prices against it, then maximum. A scan, not a loop.
```

```
>>> max_profit([7, 1, 5, 3, 6, 4])
5
>>> max_profit([7, 6, 4, 3, 1])
0
>>> max_profit([5])
0
```

Solution.

```
def max_profit(prices):
    mins = scanl1(min, prices)
    return max(zipWith(sub, prices, mins))
```

19.3 maximum_subarray: NeetCode: Maximum Subarray (Kadane)

Contract:
 max_subarray(nums: list[int]) -> int
 Largest sum of a non-empty contiguous subarray. len(nums) >= 1.

Hint:
 Kadane IS scanl1: best-ending-here = max(x, best_prev + x).
 The scan is the DP table; maximum reads the answer off it.

```
>>> max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4])
6
>>> max_subarray([1])
1
>>> max_subarray([5, 4, -1, 7, 8])
23
>>> max_subarray([-3, -1, -2])
-1
```

Solution.

```
def max_subarray(nums):
    return max(scanl1(lambda best, x: max(x, best + x), nums))
```

19.4 product_except_self: NeetCode: Product of Array Except Self

Contract:
 product_except_self(nums: list[int]) -> list[int]
 out[i] == product of all nums except nums[i]. No division. O(n).
 len(nums) >= 2.

Hint:
 Prefix products = scanl (*) 1, suffix products = scanr (*) 1.
 out = zipWith (*) prefixes suffixes -- offset by one on each side.

```
>>> product_except_self([1, 2, 3, 4])
[24, 12, 8, 6]
>>> product_except_self([-1, 1, 0, -3, 3])
[0, 0, 9, 0, 0]
```

Solution.

```
def product_except_self(nums):
    pre = scanl(mul, 1, nums)
    suf = scanr(mul, 1, nums)
    return zipWith(mul, pre[:-1], suf[1:])
```

19.5 moving_average: Average of every k-window

Contract:
 moving_average(xs: list, k: int) -> list[float]
 len(xs) >= k >= 1.

Hint:

Prefix sums = scanl (+) 0 -- one scan, then every window sum is
 pref[i+k] - pref[i]: zipWith (-) the two offset views of the same
 scan, divide by k.

```
>>> moving_average([1, 2, 3, 4], 2)
[1.5, 2.5, 3.5]
>>> moving_average([1, 1, 1], 3)
[1.0]
>>> moving_average([5], 1)
[5.0]
```

Solution.

```
def moving_average(xs, k):
    pref = scanl(add, 0, xs)
    return zipWith(lambda hi, lo: (hi - lo) / k, pref[k:], pref[:-k])
```

19.6 pascal: First n rows of Pascal's triangle

Contract:

pascal(n: int) -> list[list[int]]

Hint:

The triangle is a stream: iterate the row step from [1], where
 next = zipWith (+) (0:row) (row:0) -- the row against itself,
 shifted. take n consumes the stream.

```
>>> pascal(4)
[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1]]
>>> pascal(1)
[[1]]
>>> pascal(0)
[]
>>> pascal(6)[-1]
[1, 5, 10, 10, 5, 1]
```

Solution.

```
def pascal(n):
    step = lambda row: zipWith(add, [0] + row, row + [0])
    return take(n, iterate(step, [1]))
```

19.7 climbing_stairs: NeetCode: Climbing Stairs

Contract:

climb_stairs(n: int) -> int
 Ways to reach step n taking 1 or 2 steps. n >= 1.

Hint:

Fibonacci in disguise. iterate the pair step, take the nth --
 an unfold consumed by indexing, no table needed.

```
>>> climb_stairs(1)
1
>>> climb_stairs(2)
2
>>> climb_stairs(5)
8
```

```
>>> climb_stairs(45)
1836311903
```

Solution.

```
def climb_stairs(n):
    step = lambda p: (snd(p), fst(p) + snd(p))
    return fst(last(take(n + 1, iterate(step, (1, 1)))))
```

19.8 primes_up_to: All primes $\leq n$, ascending

```
Contract:
    primes_up_to(n: int) -> list[int]

Hint:
    Streams, Haskell style: candidates = takeWhile (<= n) count(2);
    keep p when no divisor in takeWhile (<= isqrt p) count(2) divides
    it -- all over the remainders. (The array sieve is faster; this
    version is about reading streams.)

>>> primes_up_to(20)
[2, 3, 5, 7, 11, 13, 17, 19]
>>> primes_up_to(2)
[2]
>>> primes_up_to(1)
[]
>>> len(primes_up_to(100))
25
```

Solution.

```
def primes_up_to(n):
    cands = takeWhile(lambda x: x <= n, count(2))
    is_prime = lambda p: all(
        p % d for d in takeWhile(lambda d: d <= isqrt(p), count(2)))
    return filter_(is_prime, cands)
```

19.9 totals_and_deltas: A scan and its inverse

```
Contract:
    running_totals(xs: list) -> list    # cumulative sums
    deltas(xs: list) -> list            # each element minus the previous

Hint:
    running_totals IS scanl1 (+). deltas is the pairwise differences.
    They undo each other: prepend the first element to
    deltas(running_totals(xs)) and xs comes back -- say why.

>>> running_totals([3, 1, 4])
[3, 4, 8]
>>> running_totals([])
[]
>>> deltas([5, 3, 8])
[-2, 5]
>>> deltas([7])
[]
>>> ts = running_totals([2, 5, 1]); [2] + deltas(ts) == [2, 5, 1]
True
```

Solution.

```
running_totals = lambda xs: scanl1(add, xs)
deltas = lambda xs: map_(lambda p: p[1] - p[0], pairwise(xs))
```

19.10 paren_depth: Deepest paren nesting

Input is balanced, only '(' and ')'.
 Contract:

```
max_depth(s: str) -> int
```

Hint:

Depth is a running total: map each paren to +1/-1, scanl (+) 0
 is the entire depth trace, maximum reads off the answer. No
 stack needed -- the scan IS the stack height's history.

```
>>> max_depth("((()()))")
3
>>> max_depth("()()")
1
>>> max_depth("")
0
```

Solution.

```
def max_depth(s):
    steps = map_(lambda c: 1 if c == "(" else -1, s)
    return max(scanl(add, 0, steps))
```


Chapter 20

Sorting And Searching

20.1 `is_anagram_sorted`: Anagrams via canonical form

True when a and b use exactly the same letters, ignoring case and non-letters. (g5 solves the same problem with bags -- do both, argue which you'd ship.)

Contract:

```
is_anagram(a: str, b: str) -> bool
```

Hint:

A canonical form makes equality trivial: sort of the cleaned, lowered characters. `filter_ str.isalpha` does the cleaning.

```
>>> is_anagram("Listen", "Silent")
True
>>> is_anagram("hello", "world")
False
>>> is_anagram("a gentleman", "elegant man")
True
>>> is_anagram("ab", "abb")
False
```

Solution.

```
canon = lambda s: sorted(filter_(str.isalpha, s.lower()))
is_anagram = lambda a, b: canon(a) == canon(b)
```

20.2 `second_largest`: Second largest distinct value

At least two distinct values guaranteed.

Contract:

```
second_largest(xs: list) -> value
```

Hint:

nub then sort; the answer is one from the end.

```
>>> second_largest([1, 5, 3])
3
>>> second_largest([5, 1, 5, 3])
3
>>> second_largest([2, 1])
1
>>> second_largest([-1, -5, -3])
```

```
| -3
```

Solution.

```
| second_largest = lambda xs: sorted(nub(xs))[-2]
```

20.3 `binary_search`: Index of target in a sorted list, else -1

```
Contract:
    binary_search(xs: list, target) -> int

Hint:
    bisect_left finds the leftmost slot where the target COULD be;
    one bounds-check plus one equality test finishes the job. Know
    why bisect_left(xs, x) == len(xs) is possible.

>>> binary_search([1, 3, 5, 7], 5)
2
>>> binary_search([1, 3, 5, 7], 1)
0
>>> binary_search([1, 3, 5, 7], 7)
3
>>> binary_search([1, 3], 2)
-1
>>> binary_search([], 1)
-1
```

Solution.

```
def binary_search(xs, x):
    i = bisect_left(xs, x)
    return i if i < len(xs) and xs[i] == x else -1
```

20.4 `merge_intervals`: Merge overlapping (and touching) intervals

```
Intervals are inclusive [s, e]: [1,2] and [2,3] merge to [1,3].

Contract:
    merge_intervals(intervals: list[list[int]]) -> list[list[int]]
    Result sorted by start.

Hint:
    sortOn fst, then one fold: each interval either extends the
    accumulator's last entry or starts a new one.

>>> merge_intervals([[1, 3], [2, 6], [8, 10], [15, 18]])
[[1, 6], [8, 10], [15, 18]]
>>> merge_intervals([[1, 2], [2, 3]])
[[1, 3]]
>>> merge_intervals([[5, 7], [1, 2], [3, 6]])
[[1, 2], [3, 7]]
>>> merge_intervals([[4, 4]])
[[4, 4]]
```

Solution.

```
def merge_intervals(intervals):
    def step(acc, iv):
        s, e = iv
        if acc and s <= acc[-1][1]:
            acc[-1][1] = max(acc[-1][1], e)
```

```

    else:
        acc.append([s, e])
    return acc
return foldl(step, sortOn(fst, [list(iv) for iv in intervals]), [])

```

20.5 photo_lineup: The reported Anduril prompt, prelude-sized

Back row and front row, same length; can the columns be arranged so every back person is STRICTLY taller than the front person?

Contract:

```
can_lineup(back: list[int], front: list[int]) -> bool
```

Hint:

Sort both rows; all over zipWith (>). Be ready to argue why sorted-vs-sorted decides it (the exchange argument).

```

>>> can_lineup([5, 1, 3], [2, 4, 0])
True
>>> can_lineup([1, 2, 3], [1, 2, 3])
False
>>> can_lineup([10, 9], [1, 2])
True
>>> can_lineup([2], [3])
False

```

Solution.

```

can_lineup = lambda back, front: all(
    zipWith(lambda b, f: b > f, sorted(back), sorted(front)))

```

20.6 merge_sort: A real $O(n \log n)$ sort, by hand

No sorted(), no .sorted().

Contract:

```
merge_sorted(xs: list) -> list (new list; stable)
```

Hint:

splitAt the midpoint, recurse on both halves, and the prelude's merge does the two-pointer join. Also retype merge itself from memory once -- it's the loop interviews ask for.

```

>>> merge_sorted([5, 2, 8, 1])
[1, 2, 5, 8]
>>> merge_sorted([])
[]
>>> merge_sorted([3, 3, 1])
[1, 3, 3]
>>> merge_sorted([1])
[1]

```

Solution.

```

def merge_sorted(xs):
    if len(xs) <= 1:
        return list(xs)
    a, b = splitAt(len(xs) // 2, list(xs))
    return merge(merge_sorted(a), merge_sorted(b))

```

20.7 lis: Longest strictly increasing subsequence (length)

```
Contract:
    lis(xs: list[int]) -> int

Hint:
    Patience sorting: fold the sequence into `tails`, where tails[i]
    is the smallest possible tail of an increasing run of length i+1.
    bisect_left says which pile each element lands on; a landing past
    the end grows the answer. O(n log n) where the obvious DP is
    O(n^2) -- say both.

>>> lis([10, 9, 2, 5, 3, 7, 101, 18])
4
>>> lis([5, 4, 3])
1
>>> lis([1, 2, 3])
3
>>> lis([])
0
>>> lis([7, 7, 7])
1
```

Solution.

```
def lis(xs):
    def place(tails, x):
        i = bisect_left(tails, x)
        return tails + [x] if i == len(tails) else tails[:i] + [x] + tails[i + 1:]
    return len(foldl(place, xs, []))
```

20.8 russian_doll: LC 354: Russian Doll Envelopes

Envelope (w, h) fits inside another only when BOTH are strictly smaller. No rotation. Longest nesting chain.

```
Contract:
    max_dolls(envelopes: list[tuple[int, int]]) -> int

Hint:
    Sort by (width asc, height DESC), then plain LIS on the heights
    -- g4_07 verbatim. The descending tie-break is the whole trick:
    equal-width envelopes arrive tallest-first, so no two of them can
    ever chain in a strictly-increasing height run. Sorting
    linearized one dimension; LIS handles the other. O(n log n).
```

```
>>> max_dolls([(5, 4), (6, 4), (6, 7), (2, 3)])
3
>>> max_dolls([(1, 1), (1, 1), (1, 1)])
1
>>> max_dolls([(3, 4), (3, 5), (3, 6)])
1
>>> max_dolls([])
0
>>> max_dolls([(2, 3)])
1
```

Solution.

```
def max_dolls(envelopes):
    hs = [h for _, h in sortOn(lambda e: (e[0], -e[1]), envelopes)]
    def place(tails, x):
```

```

    i = bisect_left(tails, x)
    return tails + [x] if i == len(tails) else tails[:i] + [x] + tails[i + 1:]
return len(foldl(place, hs, []))

```

20.9 pairs_within_budget: Count pairs whose sum fits a budget

Pairs are (i, j) with i < j; values may be negative; the count does not depend on input order -- that sentence licenses sorting.

Contract:

```
pairs_within_budget(t: int, xs: list[int]) -> int
```

Hint:

Sort first. Then each element asks: how many LATER elements keep the sum <= t? bisect_right(ys, t - x) answers in O(log n) -- subtract the i+1 elements at or before x itself, clip at 0.

```

>>> pairs_within_budget(5, [3, 1, 4, 2])
4
>>> pairs_within_budget(-1, [-2, -3, 5])
1
>>> pairs_within_budget(100, [1, 2, 3])
3
>>> pairs_within_budget(0, [1, 2])
0

```

Solution.

```

def pairs_within_budget(t, xs):
    ys = sorted(xs)
    return sum(max(0, bisect_right(ys, t - x) - (i + 1)) for i, x in enum(ys))

```

20.10 sensor_pairing: The screen's domain problem, function form

Two sensor feeds watch the same airspace. A detection is (id, t, bearing_degrees); ids are sortable strings. Detections a, b are COMPATIBLE when |a.t - b.t| <= t_tol and their bearing difference is <= b_tol -- and bearings wrap: 359.5 and 0.3 are 0.8 apart.

Contract:

```

pair_detections(a, b, t_tol, b_tol)
    -> (pairs, unpaired_a_ids, unpaired_b_ids)
Greedy: repeatedly take the lowest-cost compatible pair of still-
unused detections, cost = |dt| + wrapped bearing diff (exact ties
by ids). pairs as (a_id, b_id); all three outputs sorted.

```

Hint:

Build the compatible candidates as one comprehension over both feeds (cost, a_id, b_id, i, j), sort -- the fold that follows takes each candidate whose indices are still unused. The wrap is min(d, 360 - d) in ONE helper. This is mock_phased.py distilled.

```

>>> a = [('A0', 10.0, 90.0), ('A1', 20.0, 180.0)]
>>> b = [('B0', 10.2, 91.0), ('B1', 20.1, 179.5)]
>>> pair_detections(a, b, 0.5, 2.0)
([('A0', 'B0'), ('A1', 'B1')], [], [])
>>> pair_detections([('A0', 10.0, 359.5)], [('B0', 10.1, 0.3)], 0.5, 2.0)
([('A0', 'B0')], [], [])
>>> pair_detections([('A0', 0.0, 10.0)], [('B0', 900.0, 200.0)], 0.5, 2.0)
([], ['A0'], ['B0'])

```

```
>>> a = [('A0', 10.0, 45.0), ('A1', 30.0, 90.0)]
>>> b = [('B0', 10.5, 44.5), ('B1', 10.6, 46.0), ('B2', 50.0, 200.0)]
>>> pair_detections(a, b, 1.0, 2.0)
([('A0', 'B0')], ['A1'], ['B1', 'B2'])
```

Solution.

```
def pair_detections(a, b, t_tol, b_tol):
    wrapd = lambda x, y: min(abs(x - y) % 360, 360 - abs(x - y) % 360)
    candids = sorted([(abs(ta - tb) + wrapd(ba, bb), ai, bi, i, j)
                      for i, (ai, ta, ba) in enum(a)
                      for j, (bi, tb, bb) in enum(b)
                      if abs(ta - tb) <= t_tol and wrapd(ba, bb) <= b_tol])
    used_a, used_b, pairs = set(), set(), []
    for _cost, ai, bi, i, j in candids:
        if i not in used_a and j not in used_b:
            used_a.add(i)
            used_b.add(j)
            pairs.append((ai, bi))
    ua = [d[0] for i, d in enum(a) if i not in used_a]
    ub = [d[0] for j, d in enum(b) if j not in used_b]
    return sorted(pairs), sorted(ua), sorted(ub)
```

Chapter 21

Bags And Grouping

21.1 valid_anagram: NeetCode: Valid Anagram

```
Contract:
    is_anagram(s: str, t: str) -> bool
    True iff t is a permutation of s. Lowercase ASCII.

Hint:
    Two Counter folds; anagram == equal multisets. (Compare with
    g4's sorted-canonical version: same problem, different algebra.)

>>> is_anagram("anagram", "nagaram")
True
>>> is_anagram("rat", "car")
False
>>> is_anagram("", "")
True
```

Solution.

```
is_anagram = lambda s, t: Counter(s) == Counter(t)
```

21.2 ransom_note: HackerRank: Ransom Note

```
Contract:
    can_make(note: list[str], magazine: list[str]) -> bool
    True iff every word in note is available in magazine with at
    least the required multiplicity. Case-sensitive whole words.

Hint:
    Multiset inclusion: Counter(note) <= Counter(magazine), spelled
    as an `all` over the note's counts.

>>> can_make("give me one grand today night".split(), "give me one grand today".split())
False
>>> can_make("two times three is not four".split(), "two times three is not four".split())
True
>>> can_make(["give"], ["Give"])
False
```

Solution.

```
def can_make(note, magazine):
    have = Counter(magazine)
    need = Counter(note)
    return all(have[w] >= n for w, n in need.items())
```

21.3 most_frequent: Most frequent value; ties -> smallest

```
Contract:
    most_frequent(xs: list) -> value

Hint:
    Counter, then minOn the tuple key (-count, value). The
    two-level tie-break lives entirely in the key, and minOn is
    O(n) where sortOn + head pays O(n log n) for the same answer.

>>> most_frequent([1, 2, 2, 3, 3])
2
>>> most_frequent([5, 5, 1])
5
>>> most_frequent([3, 1, 2])
1
>>> most_frequent(["b", "a", "b", "a"])
'a'
```

Solution.

```
most_frequent = lambda xs: minOn(
    lambda kv: (-kv[1], kv[0]), list(Counter(xs).items()))[0]
```

21.4 top_k_words: The k most frequent words

Ordered by count descending, ties by word ascending. At least k distinct words guaranteed.

```
Contract:
    top_k_words(k: int, text: str) -> list[tuple[str, int]]

Hint:
    Counter over the words, sortOn (-count, word), take k. The whole
    pipeline is three prelude names deep.

>>> top_k_words(2, "the cat the dog the cat bird")
[('the', 3), ('cat', 2)]
>>> top_k_words(3, "b a a b c c")
[('a', 2), ('b', 2), ('c', 2)]
```

Solution.

```
top_k_words = lambda k, text: take(
    k, sortOn(lambda kv: (-kv[1], kv[0]), list(Counter(text.split()).items())))
```

21.5 group_words: Group words by first letter

First-appearance order for both keys and members.

```
Contract:
    group_words(words: list[str]) -> dict[str, list[str]]

Hint:
    THE fromListWith showcase: pairs (first letter, [word]). The
    combiner receives f(new, old) -- Haskell's order -- so KEEP
    arrival order by combining old + new; a bare (+) would reverse
    each group (the classic Haskell gotcha). Keys keep first-seen
    order.
```

```
>>> group_words(["ant", "bee", "ape"])
{'a': ['ant', 'ape'], 'b': ['bee']}
>>> group_words(["cat"])
{'c': ['cat']}
>>> group_words([])
{}
```

Solution.

```
group_words = lambda ws: fromListWith(flip(add), [(w[0], [w]) for w in ws])
```

21.6 group_anagrams: NeetCode: Group Anagrams

Contract:

```
group_anagrams(words: list[str]) -> list[list[str]]
Partition words into anagram classes; classes in first-seen order,
members in input order.
```

Hint:

Key = canonical form (sorted letters). Two prelude routes, do both: `Tree(1, list)` with a bare `append` (autovivification aids the write-heavy access), or one `fromListWith` over `(key, [w])` pairs -- combining old + new, since `f` receives `(new, old)`; a bare `(+)` would reverse each group. Compare.

```
>>> group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"])
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
>>> group_anagrams([""])
[['']]
>>> group_anagrams(["a"])
[['a']]
```

Solution.

```
def group_anagrams(words):
    return list(fromListWith(flip(add),
                             ["".join(sorted(w)), [w]) for w in words]).values()
```

21.7 merge_sum: Merge two dicts of numbers, summing shared keys

Don't mutate the inputs. Key order: a's keys, then b's new ones.

Contract:

```
merge_sum(a: dict, b: dict) -> dict
```

Hint:

This IS `unionWith` with `(+)`: merge, combining shared keys by addition. The prelude's `bag_union` is the SAME merge with `max` -- one combinator, different monoid. Spot the difference and say it. (`fromListWith` over the two items() lists concatenated gets there too; decide which reads better.)

```
>>> merge_sum({'a': 1, 'b': 2}, {'b': 3, 'c': 4})
{'a': 1, 'b': 5, 'c': 4}
>>> merge_sum({}, {'x': 1})
{'x': 1}
>>> merge_sum({'x': 1}, {})
{'x': 1}
```

Solution.

```
merge_sum = lambda a, b: unionWith(add, a, b)
```

21.8 rle_encode: Run-length encode

```
"aaabbc" -> "a3b2c1"; every run gets a count, even runs of one.
```

Contract:

```
rle_encode(s: str) -> str
```

Hint:

The prelude's group gathers runs of CONSECUTIVE equals as bare lists. map_ each run to run[0] + str(len(run)), join.

```
>>> rle_encode("aaabbc")
'a3b2c1'
>>> rle_encode("abc")
'a1b1c1'
>>> rle_encode("aaaa")
'a4'
>>> rle_encode("")
''
```

Solution.

```
rle_encode = lambda s: "".join(
    map_(lambda run: run[0] + str(len(run)), group(s)))
```

21.9 lru_class: An LRU cache class, dict-only

Capacity at construction; put(key, value); get(key) -> value or None. Both get and put count as use; puts beyond capacity evict the least recently used entry.

Contract:

```
class LRU(capacity)
    .put(key, value) -> None
    .get(key) -> value | None
```

Hint:

The container lesson with no snippet to lean on: dicts remember insertion order, so pop + re-insert IS move-to-most-recent, and next(iter(d)) is the least recent. That's the whole machine.

```
>>> c = LRU(2)
>>> c.put('a', 1); c.put('b', 2)
>>> c.get('a')
1
>>> c.put('c', 3)
>>> c.get('b') is None
True
>>> c.get('c')
3
>>> c.get('a')
1
>>> c.put('a', 99); c.get('a')
99
```

Solution.

```
class LRU:
```

```

def __init__(self, capacity):
    self.capacity = capacity
    self.data = {}
def get(self, key):
    if key not in self.data:
        return None
    val = self.data.pop(key)
    self.data[key] = val
    return val
def put(self, key, value):
    if key in self.data:
        self.data.pop(key)
    elif len(self.data) >= self.capacity:
        self.data.pop(next(iter(self.data)))
    self.data[key] = value

```

21.10 first_unique: First non-repeating character

Contract:

```
first_unique(s: str) -> str    # the char, or '_' if none
```

Hint:

Three prelude names, one line: Counter the string, find the first char whose count is 1, fromMaybe the '_' default. Two passes, O(n) -- and find stops at the first hit.

```

>>> first_unique("swiss")
'w'
>>> first_unique("aabb")
'_'
>>> first_unique("")
'_'

```

Solution.

```

def first_unique(s):
    counts = Counter(s)
    return fromMaybe('_', find(lambda c: counts[c] == 1, s))

```

21.11 log_report: Error report from messy logs

A well-formed line is "<ts> <LEVEL> <service>: <message...>": LEVEL in {INFO, WARN, ERROR}, service nonempty and glued to its colon, message may be empty. Blank lines are ignored; any other non-blank line is malformed.

Contract:

```

error_report(loglines: list[str]) -> (report, malformed_count)
report = [(service, error_count)] for services with >= 1 ERROR,
sorted by count desc, then name asc.

```

Hint:

THE mapMaybe shape: parse(line) -> (level, service) or None.
The Nones ARE the malformed count; catMaybes + a filter feed Counter; sortOn (-count, name) ranks. Messy input, no raw loops.

```

>>> logs = ["t1 ERROR api: down", "t2 INFO web: ok", "garbage line",
...         "t3 ERROR api: db", "t4 ERROR web: 500", "t5 WARN api: slow",
...         "t6 DEBUG api: no", "t7 ERROR db missing"]
>>> error_report(logs)

```

```
([('api', 2), ('web', 1)], 3)
>>> error_report(["t1 INFO web: ok", "half a line"])
([], 1)
>>> error_report([])
([], 0)
```

Solution.

```
LEVELS = {"INFO", "WARN", "ERROR"}

def parse(line):                                # (level, service) or None
    parts = line.split(maxsplit=3)
    if (len(parts) < 3 or parts[1] not in LEVELS
        or not parts[2].endswith(":") or parts[2] == ":"):
        return None
    return (parts[1], parts[2][:-1])

def error_report(loglines):
    parsed = map_(parse, filter_(lambda l: l.strip(), loglines))
    malformed = len(filter_(lambda p: p is None, parsed))
    errs = [svc for lv, svc in catMaybes(parsed) if lv == "ERROR"]
    report = sortOn(lambda kv: (-kv[1], kv[0]), list(Counter(errs).items()))
    return report, malformed
```

21.12 invert__dict: Swap a dict's keys and values

Values are unique (so the inversion is lossless).

Contract:

```
invert(d: dict) -> dict
```

Hint:

A dict is a list of pairs wearing a hat: `items()`, swap each pair, `dict()` the result. The swap showcase.

```
>>> invert({'a': 1, 'b': 2})
{1: 'a', 2: 'b'}
>>> invert({})
{}
```

Solution.

```
invert = lambda d: dict(map_(swap, list(d.items())))
```

Chapter 22

Tries And Paths

22.1 deepest_directory: Challenge: deepest path in a directory forest

Contract:

deepest(dirs: list[str]) -> str
Each string is a '/'-separated path ("a/b/c"). Return the deepest full path (unique deepest guaranteed). Paths may share prefixes.

Hint:

Building IS walking: foldl indexing over the parts autovivifies the whole branch -- no setpath needed since there is no leaf value. Then one paths() + maxOn keypath length. Note why setpath would be WRONG here: writing "a/b" after "a/b/c" would clobber the subtree.

```
>>> deepest(["a/b/c", "a/d", "e"])
'a/b/c'
>>> deepest(["x"])
'x'
>>> deepest(["a/b", "a/b/c/d", "a/b/c"])
'a/b/c/d'
```

Solution.

```
def deepest(dirs):
    t = ITree()
    for d in dirs:
        foldl(lambda node, k: node[k], d.split('/'), t)
    return '/'.join(maxOn(lambda pl: len(pl[0]), paths(t))[0])
```

22.2 autocomplete: Challenge: prefix trie with completion

Contract:

make_trie(words: list[str]) -> ITree
Letter-by-letter trie; each word terminated by '\$' whose leaf is the word itself.
complete(trie, prefix: str) -> list[str]
All stored words starting with prefix, sorted. A word counts as its own completion.

Hint:

setpath writes each word; getpath descends to the prefix's subtree WITHOUT autovivifying (reading an ITree with [] would plant ghost branches); paths() harvests every '\$' leaf below.
(No-trie baseline: filter_ + isPrefixOf over the word list, O(n*m))

```
| per query -- the trie is what repeated queries buy.)

|>>> make_trie(["at"]) == {'a': {'t': {'$': 'at'}}}
|True
|>>> t = make_trie(["cat", "car", "card", "dog", "do"])
|>>> complete(t, "ca")
|['car', 'card', 'cat']
|>>> complete(t, "do")
|['do', 'dog']
|>>> complete(t, "z")
|[]
|>>> 'z' in t
|False
```

Solution.

```
def make_trie(words):
    t = ITree()
    for w in words:
        setpath(t, list(w) + ['$'], w)
    return t

def complete(trie, prefix):
    sub = getpath(trie, list(prefix), default={})
    return sorted([leaf for p, leaf in paths(sub) if p and p[-1] == '$'])
```

22.3 subset_sum_paths: Challenge: all paths in the decision tree summing to a total

Contract:

```
build_tree(nums: list[int], total: int) -> tree
    The take/skip decision tree over nums. Node = dict with keys
    ('take', x) and ('skip', x); leaf = bool (True iff the takes
    along that path sum to total).

tree_paths(tree) -> list[list[int]]
    The taken values along every path ending in True, take-first
    DFS order.

sum_paths(nums: list[int], total: int) -> list[list[int]]
    Every subsequence of nums summing to total. Negatives allowed,
    so a path only ends at full depth -- rem == 0 mid-path proves
    nothing.
```

Hint:

```
An unfold builds the tree from seed (nums, remaining); a fold over
the tree collects paths ending in True. unfold + fold = hylomorphism:
fuse them and the tree never exists -- but here it is the exhibit.
2^n leaves: every subset gets judged, exactly once.
(The prelude now also ships subsequences -- filtering the powerset
by sum is the same 2^n with the tree left implicit.)
```

```
>>> build_tree([], 0)
True
>>> build_tree([5], 5) == {('take', 5): True, ('skip', 5): False}
True
>>> tree_paths(True)
[[]]
>>> tree_paths(False)
[]
>>> tree_paths(build_tree([1, 2], 3))
[[1, 2]]
>>> sum_paths([1, 2, 3], 3)
```

```
[[1, 2], [3]]
>>> sum_paths([2, 4, 6, 10], 16)
[[2, 4, 10], [6, 10]]
>>> sum_paths([1, -1, 2], 2)
[[1, -1, 2], [2]]
>>> sum_paths([], 0)
[[]]
>>> sum_paths([1, 1], 2)
[[1, 1]]
```

Solution.

```
def build_tree(nums, total):
    if null(nums):
        return total == 0
    x = head(nums)
    return {('take', x): build_tree(tail(nums), total - x),
            ('skip', x): build_tree(tail(nums), total)}

def tree_paths(tree):
    if tree is True:
        return [[]]
    if tree is False:
        return []
    return concat([[x] + p if tag == 'take' else p for p in tree_paths(sub)]
                  for (tag, x), sub in tree.items())

def sum_paths(nums, total):
    return tree_paths(build_tree(nums, total))
```

22.4 subset_sum_trie: Challenge: subset-sum, winners recorded in a solution trie

Contract:

```
solution_trie(nums: list[int], total: int) -> ITree
    Search the take/skip space; write only the WINNING paths (the
    taken values, terminated by '$') into an ITree.
sum_paths(nums: list[int], total: int) -> list[list[int]]
    The winning paths read back off the trie via paths().
NB: a trie stores a SET of solutions -- duplicates arising from
different index choices (e.g. [1, 1] with total 1) collapse to one.
That is a semantic change from 03's multiset.
```

Hint:

```
The full trio: unfold searches, ITree records, paths() reads.
Why the '$' terminator? One solution can be a prefix of another
([2] inside [2, 4, -4]) -- without an end marker the shorter one
vanishes into the longer one's interior.
```

```
>>> solution_trie([1, 2], 3) == {1: {2: {'$': True}}}
True
>>> sum_paths([1, 2, 3], 3)
[[1, 2], [3]]
>>> sum_paths([2, 4, -4], 2)
[[2, 4, -4], [2]]
>>> sum_paths([1, 1], 1)
[[1]]
>>> sum_paths([], 0)
[[]]
```

Solution.

```
def solution_trie(nums, total):
    t = ITree()
    def go(i, rem, picked):
        if i == len(nums):
            if rem == 0:
                setpath(t, picked + ['$'], True)
            return
        go(i + 1, rem - nums[i], picked + [nums[i]])
        go(i + 1, rem, picked)
    go(0, total, [])
    return t

def sum_paths(nums, total):
    return [p[:-1] for p, _ in paths(solution_trie(nums, total)) if p[-1] == '$']
```

Chapter 23

Graphs Grids Heaps

23.1 longest_unique: Longest substring without repeats

```
Contract:
    longest_unique(s: str) -> int

Hint:
    The longest_window skeleton does the two-pointer bookkeeping; you
    supply the state as closures: per-char counts (Tree(1, int)) plus
    a duplicate tally. valid() == "no duplicates in the window".

>>> longest_unique("abcabcbb")
3
>>> longest_unique("bbbbbb")
1
>>> longest_unique("")
0
>>> longest_unique("abba")
2
```

Solution.

```
def longest_unique(s):
    counts, dups = Tree(1, int), [0]
    def add(c):
        counts[c] += 1
        dups[0] += counts[c] == 2
    def rem(c):
        counts[c] -= 1
        dups[0] -= counts[c] == 1
    return longest_window(s, lambda: dups[0] == 0, add, rem)
```

23.2 grid_path: Shortest path through a grid

Steps from top-left to bottom-right through '.' cells, around '#' walls, moving orthogonally. -1 if unreachable (or an endpoint is a wall); a 1x1 open grid is 0.

```
Contract:
    grid_path(grid: list[str]) -> int
```

```
Hint:
    BFS: the prelude deque is the frontier, neighbors4 the edges.
    Mark seen when you ENQUEUE, not when you pop.
```

```
>>> grid_path(["..", ".."])
2
>>> grid_path([".#", "#."])
-1
>>> grid_path(["."])
0
>>> grid_path(["..#", "#.#", "#.."])
4
```

Solution.

```
def grid_path(grid):
    R, C = len(grid), len(grid[0])
    if grid[0][0] == "#" or grid[R - 1][C - 1] == "#":
        return -1
    q, seen = deque([(0, 0, 0)]), {(0, 0)}
    while len(q):
        r, c, d = q.popleft()
        if (r, c) == (R - 1, C - 1):
            return d
        for nr, nc in neighbors4(r, c, R, C):
            if grid[nr][nc] == "." and (nr, nc) not in seen:
                seen.add((nr, nc))
                q.append((nr, nc, d + 1))
    return -1
```

23.3 rate_limiter: Sliding-window rate limiter

A request at time `ts` is denied when its client already has `limit` ALLOWED requests in `(ts - window, ts]`. Denied requests consume no budget. Timestamps arrive non-decreasing.

Contract:

```
rate_limiter(window: int, limit: int,
             events: list[tuple[int, str]]) -> list[tuple[int, str]]
The denied (ts, client) pairs, in stream order.
```

Hint:

Tree(1, deque): each client's allowed timestamps in a deque. The prelude deque has no peek -- evict by popleft, and appendleft the survivor back. Each stamp enters and leaves once: $O(1)$ amortized.

```
>>> rate_limiter(10, 2, [(1, 'alice'), (2, 'alice'), (3, 'alice'),
...                      (5, 'bob'), (12, 'alice'), (13, 'alice')])
[(3, 'alice')]
>>> rate_limiter(5, 3, [(1, 'a'), (2, 'a'), (3, 'a')])
[]
>>> rate_limiter(5, 1, [(1, 'a'), (2, 'b'), (3, 'a'), (7, 'a'), (8, 'b')])
[(3, 'a')]
```

Solution.

```
def rate_limiter(window, limit, events):
    allowed = Tree(1, deque)
    denied = []
    for ts, client in events:
        q = allowed[client]
        while len(q):
            t = q.popleft()
            if t > ts - window:
                q.appendleft(t)
                break
        if len(q) == limit:
            denied.append((ts, client))
```

```

    if len(q) >= limit:
        denied.append((ts, client))
    else:
        q.append(ts)
    return denied

```

23.4 running_median: Median after each arrival

Even counts average the two middle values; one decimal is the caller's problem -- return floats.

Contract:

```
running_median(xs: list[int]) -> list[float]
```

Hint:

Two heaps: lo holds the lower half NEGATED (the prelude heap is min-only), hi the upper half. Push through lo into hi, rebalance so lo keeps the extra; the median reads off the tops.

```

>>> running_median([12, 4, 5, 3, 8, 7])
[12.0, 8.0, 5.0, 4.5, 5.0, 6.0]
>>> running_median([10])
[10.0]
>>> running_median([1, 2, 3, 4])
[1.0, 1.5, 2.0, 2.5]

```

Solution.

```

def running_median(xs):
    lo, hi, out = [], [], []
    for x in xs:
        heappush(lo, -x)
        heappush(hi, -heappop(lo))
        if len(hi) > len(lo):
            heappush(lo, -heappop(hi))
        m = -lo[0] if len(lo) > len(hi) else (-lo[0] + hi[0]) / 2
        out.append(float(m))
    return out

```

23.5 dijkstra: Challenge: shortest paths from weighted (from, to, dist) edges

Contract:

```

dijkstra(edges: list[tuple], src) -> (dist: dict, parent: dict)
    Directed edges (u, v, w), w >= 0. dist[n] = shortest distance
    src -> n; unreachable nodes absent. parent[n] = predecessor on
    one shortest path (parent[src] = None).
shortest_path(edges, src, dst) -> list | None
    The node sequence of a shortest src -> dst path, None if
    unreachable.

```

Hint:

Adjacency map = Tree(1, list) fold over edges. Frontier = the heap.
 Lazy deletion: skip a popped entry unless it matches the current
 best -- the heap may hold stale distances. Path reconstruction =
 an unfold: iterate the parent chain, takeWhile not None, reverse.
 Graphs are not trees -- cycles make tree unfolds diverge, so fold
 over STATES to a fixed point instead.

```
>>> dist, _ = dijkstra([('a','b',1), ('b','c',2), ('a','c',4)], 'a')
```

```
>>> dist == {'a': 0, 'b': 1, 'c': 3}
True
>>> dist, _ = dijkstra([('a','b',5), ('a','c',1), ('c','b',1)], 'a')
>>> dist['b']
2
>>> dijkstra([('x','y',1)], 'y')[0]
{'y': 0}
>>> shortest_path([('a','b',5), ('a','c',1), ('c','b',1)], 'a', 'b')
['a', 'c', 'b']
>>> shortest_path([('a','b',1)], 'a', 'a')
['a']
>>> shortest_path([('a','b',1)], 'b', 'a') is None
True
```

Solution.

```
INF = float("inf")

def dijkstra(edges, src):
    adj = Tree(1, list)
    for u, v, w in edges:
        adj[u].append((w, v))
    dist, parent, h = {src: 0}, {src: None}, [(0, src)]
    while h:
        d, u = heappop(h)
        if d == dist.get(u, INF):
            for w, v in adj[u]:
                if d + w < dist.get(v, INF):
                    dist[v] = d + w
                    parent[v] = u
                    heappush(h, (d + w, v))
    return dist, parent

def shortest_path(edges, src, dst):
    dist, parent = dijkstra(edges, src)
    if dst not in dist:
        return None
    back = takeWhile(lambda n: n is not None, iterate(lambda n: parent[n], dst))
    return list(reversed(back))
```

23.6 dag_longest_path: Challenge: longest path in a weighted DAG

Contract:

```
dag_longest(edges: list[tuple]) -> (total: int, path: list)
Directed acyclic edges (u, v, w). The maximum-weight path anywhere
in the DAG and its node sequence. Ties broken toward the
lexically-first end node. Input guaranteed acyclic.
```

Hint:

Longest path is NP-hard on general graphs; acyclicity is what makes it linear. Kahn's topo sort (indegree fold + deque), then a fold over topo order relaxing $best[v] = \max(best[v], best[u] + w)$. Same skeleton as Dijkstra's relax with max for min and topo order replacing the heap -- the LCS-vs-edit-distance move on graphs. `maxOn` best picks the end node; feed it the SORTED nodes so ties break lexically.

```
>>> dag_longest([('a','b',3), ('b','c',4), ('a','c',5)])
(7, ['a', 'b', 'c'])
>>> dag_longest([('s','a',1), ('s','b',5), ('a','t',6), ('b','t',1)])
(7, ['s', 'a', 't'])
>>> dag_longest([('u','v',2)])
```

```
| (2, ['u', 'v'])
```

Solution.

```
def dag_longest(edges):
    adj, indeg, nodes = Tree(1, list), Tree(1, int), set()
    for u, v, w in edges:
        adj[u].append((w, v))
        indeg[v] += 1
        nodes |= {u, v}
    q = deque([n for n in sorted(nodes) if indeg[n] == 0])
    topo = []
    while len(q):
        u = q.popleft()
        topo.append(u)
        for w, v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    best = {n: 0 for n in nodes}
    parent = {n: None for n in nodes}
    for u in topo:
        for w, v in adj[u]:
            if best[u] + w > best[v]:
                best[v] = best[u] + w
                parent[v] = u
    end = maxOn(lambda n: best[n], sorted(nodes))
    back = takeWhile(lambda n: n is not None, iterate(lambda n: parent[n], end))
    return best[end], list(reversed(back))
```

23.7 bellman_ford: Challenge: shortest paths as a fixed point (negative weights ok)

Contract:

```
bellman_ford(edges: list[tuple], src) -> dict | None
Directed edges (u, v, w); w may be negative. dist[n] = shortest
distance src -> n, unreachable nodes absent. Returns None if a
negative cycle is reachable from src (distances then have no
fixed point -- they diverge to -inf).
```

Hint:

Shortest paths ARE a fixed point: dist is the least map satisfying $\text{dist}[v] \leq \text{dist}[u] + w$ for every edge. So: relax ALL edges as one step function, `until` the map stops changing. Convergence within $|V| - 1$ steps is guaranteed (a shortest path has at most $|V| - 1$ edges); a change on step $|V|$ proves a negative cycle. Dijkstra is the same relax with a clever ORDER; Bellman-Ford orders nothing and pays for it in passes.

```
>>> bellman_ford([('a','b',1), ('b','c',2), ('a','c',4)], 'a') == {'a': 0, 'b': 1, 'c': 3}
True
>>> bellman_ford([('a','b',5), ('b','c',-4), ('a','c',2)], 'a')['c']
1
>>> bellman_ford([('a','b',1), ('b','c',-3), ('c','b',1), ('b','d',1)], 'a') is None
True
>>> bellman_ford([('x','y',7)], 'y')
{'y': 0}
>>> bellman_ford([('a','b',-2), ('b','a',3)], 'a') == {'a': 0, 'b': -2}
True
```

Solution.

```

INF = float("inf")

def bellman_ford(edges, src):
    n_nodes = len({n for u, v, _ in edges for n in (u, v)} | {src})

    def relax(state):
        steps, dist = state
        new = dict(dist)
        for u, v, w in edges:
            if new.get(u, INF) + w < new.get(v, INF):
                new[v] = new[u] + w
        return (steps + 1, new)

    def settled(state):
        steps, dist = state
        return steps >= n_nodes or relax(state)[1] == dist

    steps, dist = until(settled, relax, (0, {src: 0}))
    return None if relax((steps, dist))[1] != dist else dist

```

23.8 course__schedule: NeetCode: Course Schedule (cycle detection via Kahn)

Contract:

```

can_finish(n: int, prereqs: list[tuple]) -> bool
    Courses 0..n-1; (a, b) means b must precede a. True iff every
    course can be taken, i.e. the prerequisite graph is acyclic.
find_order(n: int, prereqs: list[tuple]) -> list | None
    A valid course order (smallest-numbered course first on ties),
    or None if impossible.

```

Hint:

Kahn's insight inverted: topo sort doesn't FAIL on a cycle, it just leaves the cycle behind -- nodes on a cycle never reach indegree 0. So: run 06's Kahn, then len(topo) == n is the whole cycle test. Heap instead of deque only to make ties deterministic.

```

>>> find_order(2, [(1, 0)])
[0, 1]
>>> find_order(4, [(1, 0), (2, 0), (3, 1), (3, 2)])
[0, 1, 2, 3]
>>> find_order(2, [(1, 0), (0, 1)]) is None
True
>>> find_order(3, [])
[0, 1, 2]
>>> can_finish(2, [(1, 0)])
True
>>> can_finish(2, [(1, 0), (0, 1)])
False
>>> can_finish(5, [(1, 4), (2, 4), (3, 1), (3, 2)])
True

```

Solution.

```

def find_order(n, prereqs):
    adj, indeg = Tree(1, list), Tree(1, int)
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    heap = []
    for c in range(n):
        if indeg[c] == 0:

```

```
        heappush(heap, c)
    topo = []
    while heap:
        u = heappop(heap)
        topo.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                heappush(heap, v)
    return topo if len(topo) == n else None

def can_finish(n, prereqs):
    return find_order(n, prereqs) is not None
```


Chapter 24

Dp And Control

24.1 coin_change: NeetCode: Coin Change

Contract:

```
coin_change(coins: list[int], amount: int) -> int
Fewest coins summing to amount (unlimited supply), -1 if impossible.
amount >= 0, coins positive.
```

Hint:

```
Knapsack, dp[remaining]. Top-down with memo on the single
index `amount` -- the future only needs what remains.
```

```
>>> coin_change([1, 2, 5], 11)
3
>>> coin_change([2], 3)
-1
>>> coin_change([1], 0)
0
>>> coin_change([186, 419, 83, 408], 6249)
20
```

Solution.

```
INF = float("inf")

def coin_change(coins, amount):
    @memo
    def go(rem):
        if rem == 0:
            return 0
        return min([1 + go(rem - c) for c in coins if c <= rem],
                    default=INF)
    best = go(amount)
    return -1 if best == INF else best
```

24.2 lcs: NeetCode: Longest Common Subsequence

Contract:

```
lcs(a: str, b: str) -> int
Length of the longest common subsequence (not substring).
```

Hint:

```
Alignment pattern, dp[i][j] -- same skeleton as 03's edit
distance, but max instead of min. Index-based args for the cache.
```

```
>>> lcs("abcde", "ace")
3
>>> lcs("abc", "abc")
3
>>> lcs("abc", "def")
0
>>> lcs("", "abc")
0
```

Solution.

```
def lcs(a, b):
    @memo
    def go(i, j):
        if i == len(a) or j == len(b):
            return 0
        if a[i] == b[j]:
            return 1 + go(i + 1, j + 1)
        return max(go(i + 1, j), go(i, j + 1))
    return go(0, 0)
```

24.3 edit_distance: Levenshtein distance

Minimum single-character insertions, deletions, or substitutions turning a into b.

Contract:

```
edit_distance(a: str, b: str) -> int
```

Hint:

The alignment DP again -- 02's skeleton with min-of-three plus one. Matching heads cost nothing; otherwise pay 1 and recurse on the three repairs. memo over the index pair.

```
>>> edit_distance("kitten", "sitting")
3
>>> edit_distance("", "abc")
3
>>> edit_distance("same", "same")
0
>>> edit_distance("flaw", "lawn")
2
>>> edit_distance("a", "b")
1
```

Solution.

```
def edit_distance(a, b):
    @memo
    def go(i, j):
        if i == len(a):
            return len(b) - j
        if j == len(b):
            return len(a) - i
        if a[i] == b[j]:
            return go(i + 1, j + 1)
        return 1 + min(go(i + 1, j), go(i, j + 1), go(i + 1, j + 1))
    return go(0, 0)
```

24.4 knapsack: 0/1 knapsack

Each (weight, value) item usable at most once; maximize value within capacity.

Contract:

```
knapsack(capacity: int, items: list[tuple[int, int]]) -> int
```

Hint:

memo over (i, room): take item i or skip it. Two integers of state -- the cache keys on the args tuple, so keep them hashable.

```
>>> knapsack(10, [(5, 10), (4, 40), (6, 30), (3, 50)])
90
>>> knapsack(0, [(1, 100)])
0
>>> knapsack(5, [])
0
>>> knapsack(3, [(4, 100), (2, 7)])
7
```

Solution.

```
def knapsack(capacity, items):
    items = list(items)
    @memo
    def go(i, room):
        if i == len(items) or room == 0:
            return 0
        w, v = items[i]
        skip = go(i + 1, room)
        if w > room:
            return skip
        return max(skip, v + go(i + 1, room - w))
    return go(0, capacity)
```

24.5 calculator: Integer expression evaluator (capstone)

+, -, * and parentheses, optional spaces, normal precedence, left-associative. No unary minus, no division.

Contract:

```
evaluate(expr: str) -> int
```

Grammar (one function per rule, shared position cursor):

```
expression := term (('+' | '-') term)*
term       := factor ('*' factor)*
factor     := NUMBER | '(' expression ')'
```

Hint:

Recursive descent. Lex numbers with takeWhile str.isdigit on the remaining input -- span IS the lexer. And remember the trap you certified earlier: ' ' in "+-" is True; test membership against a tuple.

```
>>> evaluate("2*(3+4)")
14
>>> evaluate("2*(3+4)-5")
9
>>> evaluate("10-2-3")
5
```

```
>>> evaluate("2+3*4")
14
>>> evaluate("((1+2))*3")
9
>>> evaluate(" 12 * 2 ")
24
```

Solution.

```
def evaluate(expr):
    s = expr.replace(" ", "")
    pos = [0]

    def peek():
        return s[pos[0]] if pos[0] < len(s) else ""

    def number():
        digits = takeWhile(str.isdigit, s[pos[0]:])
        pos[0] += len(digits)
        return int("".join(digits))

    def factor():
        if peek() == "(":
            pos[0] += 1
            v = expression()
            pos[0] += 1
            return v
        return number()

    def term():
        v = factor()
        while peek() == "*":
            pos[0] += 1
            v *= factor()
        return v

    def expression():
        v = term()
        while peek() in ("+", "-"):
            op = peek()
            pos[0] += 1
            v = v + term() if op == "+" else v - term()
        return v

    return expression()
```

24.6 max_height_cuboids: LC 1691: Maximum Height by Stacking Cuboids

Any rotation allowed; cuboid A sits on B only when ALL THREE of A's dims are $\leq$ B's, compared side to side. Each cuboid used at most once. Maximize stack height.

Contract:

```
max_height(cuboids: list[tuple[int, int, int]]) -> int
```

Hint:

The rotation freedom COLLAPSES: sort each cuboid's own dims ascending -- the biggest dim as height is always at least as good, and sorted-vs-sorted is the only comparison that can succeed (the photo_lineup exchange argument, twice). Then sort the cuboids and run weighted LIS: best ending at $i = h_i + \text{best over } j \text{ fitting under } i$. Fold-with-history, $O(n^2)$ -- the g4_07 shape with sum

```
instead of length. (g8_06's mask version is the ground truth
oracle for THIS file: same >= 17-style surprises, different fit
rule -- all three dims here, base only there.)
```

```
>>> max_height([(50, 45, 20), (95, 37, 53), (45, 23, 12)])
190
>>> max_height([(38, 25, 45), (76, 35, 3)])
76
>>> max_height([(7, 11, 17), (7, 17, 11), (11, 7, 17), (11, 17, 7), (17, 7, 11), (17, 11, 7)])
102
>>> max_height([(1, 1, 1)])
1
```

Solution.

```
def max_height(cuboids):
    cs = sorted([sorted(c) for c in cuboids])
    fits = lambda a, b: all(zipWith(lambda x, y: x <= y, a, b))    # a under-or-equal b, all three dims
    best = []
    for i, c in enumerate(cs):
        best.append(c[2] + max([best[j] for j in range(i) if fits(cs[j], c)],
                               default=0))
    return max(best, default=0)
```

24.7 guillotine__cut: Max value cut from a sheet, guillotine cuts only

A W x H sheet; piece types (pw, ph, value), unlimited supply, fixed orientation (add rotated copies yourself if rotation is allowed). Every cut runs edge to edge, so each cut splits one rectangle into two rectangles. Unused offcuts score 0.

Contract:

```
guillotine(W: int, H: int, pieces: list[tuple[int, int, int]]) -> int
```

Hint:

Guillotine is what re-admits DP: free-form 2D packing has no small state (the frontier goes irregular), but edge-to-edge cuts keep every subproblem a RECTANGLE -- (w, h), two ints. Interval pattern: best(w,h) = max of a piece that fits the sheet EXACTLY, every vertical split x, every horizontal split y. A smaller piece never needs a direct case: the cut that isolates it is one of the splits. memo over (w, h); O(W*H*(W+H)) -- say it.

```
>>> guillotine(4, 4, [(1, 1, 1)])
16
>>> guillotine(4, 4, [(3, 3, 7), (1, 1, 1)])
16
>>> guillotine(4, 4, [(3, 3, 20), (1, 1, 1)])
27
>>> guillotine(4, 4, [(4, 2, 10), (2, 2, 3)])
20
>>> guillotine(3, 3, [(2, 2, 5)])
5
>>> guillotine(2, 2, [(3, 1, 99)])
0
```

Solution.

```
def guillotine(W, H, pieces):
    pieces = list(pieces)

    @memo
```

```
def best(w, h):
    exact = [v for pw, ph, v in pieces if (pw, ph) == (w, h)]
    vsplit = [best(x, h) + best(w - x, h) for x in range(1, w // 2 + 1)]
    hsplit = [best(w, y) + best(w, h - y) for y in range(1, h // 2 + 1)]
    return max(exact + vsplit + hsplit, default=0)

return best(W, H)
```

24.8 partition_k_subsets: LC 698: Partition to K Equal Sum Subsets

Contract:

```
can_partition(nums: list[int], k: int) -> bool
True iff nums splits into k non-empty groups of equal sum.
Positive ints, small n (say the bound: 2^n states).
```

Hint:

The mask family, pure form: the future needs (which numbers remain, how much room is left in the group being filled). When a group closes (room hits 0) the room resets to target -- so the state is just (used, room): the number of CLOSED groups is implied by the used-sum. memo over that pair; fill greedily into one group at a time to avoid counting group orderings. (g8_06's go is this skeleton plus geometry.)

```
>>> can_partition([4, 3, 2, 3, 5, 2, 1], 4)
True
>>> can_partition([1, 2, 3, 4], 3)
False
>>> can_partition([2, 2, 2, 2], 2)
True
>>> can_partition([1], 1)
True
>>> can_partition([1, 1], 3)
False
```

Solution.

```
def can_partition(nums, k):
    nums = list(nums)
    total, n = sum(nums), len(nums)
    if k <= 0 or n < k or total % k:
        return False
    target = total // k

    @memo
    def go(used, room):
        if used == (1 << n) - 1:
            return room == target          # last group closed cleanly
        return any(go(used | (1 << i),
                     target if room == nums[i] else room - nums[i])
                   for i in range(n)
                   if not used & (1 << i) and nums[i] <= room)

    return go(0, target)
```

24.9 min_path_sum: Cheapest right/down path through a grid

Contract:

```
min_path_sum(grid: list[list[int]]) -> int
```

Top-left to bottom-right, moving only right or down.

Hint:

Grid DP: state (r, c) -- the future only cares where you stand.
memo over the index pair; base case is the goal cell.
Compare g7_02: uniform steps there needed BFS; weighted cells
with only right/down moves make a DAG, so plain DP suffices.

```
>>> min_path_sum([[1, 3, 1], [1, 5, 1], [4, 2, 1]])
7
>>> min_path_sum([[5]])
5
>>> min_path_sum([[1, 2], [3, 4]])
7
```

Solution.

```
def min_path_sum(grid):
    R, C = len(grid), len(grid[0])

    @memo
    def go(r, c):
        if (r, c) == (R - 1, C - 1):
            return grid[r][c]
        opts = ([go(r + 1, c)] if r + 1 < R else []) + \
            ([go(r, c + 1)] if c + 1 < C else [])
        return grid[r][c] + min(opts)

    return go(0, 0)
```


Chapter 25

Stack Folds

25.1 `balanced_brackets`: Matching pairs: valid bracket nesting

Only `()[]{}` appear.

Contract:

```
balanced(s: str) -> bool
```

Hint:

The stack fold, purest form: a plain list IS a stack, and the fold's accumulator is one. The twist worth learning: use `None` as the `FAILED` state -- once a closer mismatches, the poison propagates through the rest of the fold untouched (Maybe as accumulator). `Balanced ==` the fold ends on an empty stack.

```
>>> balanced("([{}])")
True
>>> balanced("([])")
False
>>> balanced("(((")
False
>>> balanced("")
True
>>> balanced("]")
False
```

Solution.

```
MATCH = {"(": ")": "(", ")": "[", "]": "["}: "{}": "{}"
```

```
def balanced(s):
    def step(stack, c):
        if stack is None:
            return None                # poison: already failed
        if c in "([{":
            return stack + [c]
        if not stack or stack[-1] != MATCH[c]:
            return None
        return stack[:-1]
    return foldl(step, s, []) == []
```

25.2 `eval_rpn`: Evaluate Reverse Polish notation

Integer operands, operators `+ - *`.

```
Contract:
    eval_rpn(tokens: list[str]) -> int

Hint:
    One fold over the tokens: operands push; an operator pops two and
    pushes the result. The classic slip: stack[-2] is the LEFT
    operand. The answer is whatever the fold leaves on the stack.

>>> eval_rpn(["2", "3", "+", "4", "*"])
20
>>> eval_rpn(["5", "1", "2", "+", "4", "*", "-"])
-7
>>> eval_rpn(["7"])
7
```

Solution.

```
OPS = {"+": add, "-": sub, "*": mul}

def eval_rpn(tokens):
    def step(stack, tok):
        if tok in OPS:
            return stack[:-2] + [OPS[tok](stack[-2], stack[-1])]
        return stack + [int(tok)]
    return foldl(step, tokens, [])[0]
```

25.3 next_greater: First later element that's larger

For each element, the next strictly greater element to its right;
-1 if none.

```
Contract:
    next_greater(xs: list[int]) -> list[int]

Hint:
    The MONOTONIC stack: fold over enum(xs); the newcomer pops every
    stacked index whose value it beats (that newcomer is their
    answer), then pushes itself. Each index pushes and pops at most
    once -- O(n) despite the inner while. The pro tier of stacks.
```

```
>>> next_greater([2, 1, 2, 4, 3])
[4, 2, 4, -1, -1]
>>> next_greater([5, 4, 3])
[-1, -1, -1]
>>> next_greater([])
[]
```

Solution.

```
def next_greater(xs):
    out = [-1] * len(xs)
    def step(stack, ix):
        i, x = ix
        while stack and xs[stack[-1]] < x:
            out[stack.pop()] = x
        return stack + [i]
    foldl(step, enum(xs), [])
    return out
```

Part III

Appendices

Appendix A

prelude.py, Complete

```
1  """prelude.py -- Haskell-style Prelude in pure Python, in pedagogical order.
2
3  Pascal discipline: strict define-before-use. Read top to bottom and every name is already defined
4  when you meet it. Sections build on each other:
5
6      1 combinators          2 pairs          3 arithmetic
7      4 list basics          5 higher-order lists  6 folds
8      7 scans                8 streams (lazy)  9 slicing & spans
9      10 sorting & searching 11 strings      12 containers
10     13 nodes               14 grids & windows 15 control
11     16 monoids & monads
12
13  Within a section names run alphabetically, except that a definition is hoisted ahead of the first
14  name that uses it (foldl before compose, span before break_), so the discipline holds inside
15  sections too.
16
17  Import qualified (import prelude as P): Haskell names shadow builtins by design. Beyond the Prelude
18  proper: the working parts of Data.List, Data.Map, Data.Maybe, Data.Function and friends are folded
19  into the same sections -- grouped by what they do, not by the module they came from. None is
20  Nothing throughout. foldr = foldl(flip(f), reversed(xs), base): iterative, stack-safe, valid only
21  because Python is strict.
22  """
23
24  # ===== 1. combinators ===== (functions about functions)
25
26  const      = lambda x: lambda _: x                # always returns x, ignoring the second argument
27  # curry: converts a two-argument function into a chain of two one-argument functions, so instead
28  # of calling f(x, y), you call curry(f)(x)(y).
29  curry      = lambda f: lambda x: lambda y: f(x, y)
30  flip       = lambda f: (lambda x, y: f(y, x))      # flip f x y = f y x
31  NOTHING    = object()                            # Maybe's Nothing: "no arg given"; test with `is`
32                                                    # hoisted ahead of foldl, which needs it as a sentinel
33
34  def foldl(f, xs, base=NOTHING): # THE left fold; Python buried its own in functools as reduce
35      it = iter(xs)
36      if base is NOTHING:
37          try:
38              acc = next(it)
39          except StopIteration:
40              raise TypeError("fold of empty sequence with no initial value")
41      else:
42          acc = base
43      for x in it:
44          acc = f(acc, x)
45      return acc
46
47  foldr      = lambda f, xs, base: foldl(flip(f), reversed(list(xs)), base) # THE right fold, via foldl
48  compose    = lambda *fns: lambda x: foldr(lambda f, acc: f(acc), fns, x)  # foldr (.) id: RIGHTMOST first
49  fromMaybe = lambda d, x: d if x is None else x      # Data.Maybe: default for every None-returning tool
50  id         = lambda x: x                          # shadows builtin id() by design
51  isJust     = lambda x: x is not None                # Data.Maybe: None test as a handable predicate
52  isNothing  = lambda x: x is None                    # Data.Maybe: true when x is Nothing (None)
53  on         = lambda f, g: lambda x, y: f(g(x), g(y)) # Data.Function: combine via a key -- cmp `on` fst
```

```

54 partial    = lambda f, *bound: lambda *args: f(*bound, *args)    # fix leading args of f, wait for the rest
55 uncurry    = lambda f: lambda p: f(*p)                          # inverse of curry: call f with a pair, not two args
56
57 # ===== 2. pairs =====
58
59 fst         = lambda p: p[0]                                     # first element of a pair
60 snd         = lambda p: p[1]                                     # second element of a pair
61 swap        = compose(tuple, reversed)    # Data.Tuple
62
63 # ===== 3. arithmetic & logic =====
64
65 add         = lambda a, b: a + b                                # (+) as a value: scanl1(add, xs)
66 div         = lambda a, b: a // b                                # floors, like Haskell div
67 even        = lambda n: n % 2 == 0                               # true for even numbers
68 gcd         = lambda a, b: abs(a) if b == 0 else gcd(b, a % b)    # >= 0 like Haskell; depth <= 90
69 halve       = lambda n: n // 2                                  # e.g. iterate(halve, 1024) -> 1024, 512, 256, ...
70 hypot       = lambda x, y: (x*x + y*y) ** 0.5                   # straight-line distance
71
72 def isqrt(n): # floor sqrt without floats (Newton)
73     if n < 0:
74         raise ValueError("isqrt of negative")
75     x = n
76     y = halve(x + 1)
77     while y < x:
78         x, y = y, halve(y + n // y)
79     return x if n else 0
80
81 lcm         = lambda a, b: abs(a // gcd(a, b) * b) if a and b else 0 # >= 0 like Haskell
82 mod         = lambda a, b: a % b                                  # remainder, sign follows b
83 mul         = lambda a, b: a * b                                  # (*) as a value: Product, zipWith(mul, a, b)
84 not_        = lambda b: not b                                     # `not` is a Python keyword
85 odd         = lambda n: n % 2 == 1                               # true for odd numbers
86 otherwise   = True                                              # guard fallback: always true
87 pred        = lambda x: chr(ord(x) - 1) if isinstance(x, str) else x - 1 # Enum: numbers and Chars
88 quot        = lambda a, b: -(a // b) if (a < 0) != (b < 0) else a // b # truncates, like Haskell quot
89 rem         = lambda a, b: a - b * quot(a, b)                    # remainder, sign follows a
90 quotRem     = lambda a, b: (quot(a, b), rem(a, b))                # (quot, rem) in one call
91 signum      = lambda x: (x > 0) - (x < 0)                         # -1, 0, or 1 by the sign of x
92 sub         = lambda a, b: a - b                                  # (-) as a value: zipWith(sub, a, b)
93 succ        = lambda x: chr(ord(x) + 1) if isinstance(x, str) else x + 1 # next: increments, advances chars
94
95 # ===== 4. list basics =====
96
97 drop        = lambda n, xs: list(xs)[max(n, 0):]                # finite lists; n<0 drops none
98 elem        = lambda x, xs: x in xs                             # true if x is in xs
99 zip_        = lambda a, b: list(zip(a, b))                      # shortest wins, any iterable
100 enum        = lambda xs, start=0: zip_(list(range(start, start + len(xs))), xs) # pairs each x with its index
101 head        = lambda xs: xs[0]                                   # first element
102 init        = lambda xs: xs[:-1]                                 # all but the last element
103 isPrefixOf  = lambda p, xs: list(xs[:len(p)]) == list(p)        # Data.List; strings or lists
104 isSuffixOf  = lambda p, xs: list(xs[len(xs) - len(p):]) == list(p) # not [-len(p):]: [-0:] is ALL
105 last        = lambda xs: xs[-1]                                  # last element
106 lookup      = lambda k, pairs: next((v for kk, v in pairs if kk == k), None) # Nothing -> None
107 notElem     = lambda x, xs: x not in xs                          # true if x is not in xs
108 nub         = compose(list, dict.fromkeys)                       # unique; first seen wins
109 null        = lambda xs: len(xs) == 0                           # true for an empty list
110 pairwise    = lambda xs: zip_(xs, xs[1:])                       # zip xs (tail xs)
111 replicate   = lambda n, x: [x] * n                               # x repeated n times
112 splitAt     = lambda n, xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs) # n<0 splits at the front
113 stripPrefix = lambda p, xs: xs[len(p):] if isPrefixOf(p, xs) else None # Just the rest, or Nothing
114 tail        = lambda xs: xs[1:]                                  # all but the first element
115 def transpose(xss): # Data.List: rows <-> cols, RAGGED-SAFE like Haskell
116     xss = [list(xs) for xs in xss] # any iterables in (rows may be one-shot); materialise once
117     n = max(map(len, xss), default=0) # short rows just drop out of later columns (no truncation)
118     return [[xs[i] for xs in xss if i < len(xs)] for i in range(n)]
119 unzip       = lambda ps: tuple(map(list, zip(*ps))) if ps else ([], []) # inverse of zip: pairs -> two lists
120 zip3        = lambda a, b, c: list(zip(a, b, c))                 # zip three lists together
121
122 # ===== 5. higher-order lists =====
123
124 filter_     = lambda cond, xs: [x for x in xs if cond(x)]        # keep elements where cond holds
125 catMaybes   = partial(filter_, isJust)                           # mapMaybe id
126 concat      = lambda xss: [x for xs in xss for x in xs]          # flatten one level of nesting

```

```

127 concatMap = lambda f, xs: [y for x in xs for y in f(x)]           # map f over xs, then flatten
128 cross     = lambda a, b: [(x, y) for x in a for y in b]          # cartesian; `product` is the fold
129 # Data.List find: lazy first-match -- folds can't stop early, find can
130 find      = lambda cond, xs: next((x for x in xs if cond(x)), None) # first match, else None
131 map_      = lambda f, xs: [f(x) for x in xs]                     # apply f to every element
132 # Data.Maybe mapMaybe: map and drop the Nothings in ONE pass -- the parse-and-filter shape
133 mapMaybe = lambda f, xs: [y for y in (f(x) for x in xs) if y is not None]
134 def partition(cond, xs):                                         # (keepers, rest) in ONE pass; cond called once per element
135     yes, no = [], []
136     for x in xs:
137         (yes if cond(x) else no).append(x)
138     return (yes, no)
139 starmap    = lambda f, pairs: [f(*p) for p in pairs]             # map . uncurry
140 zipWith    = lambda f, a, b: [f(x, y) for x, y in zip(a, b)]      # combine two lists elementwise
141 zipWith3    = lambda f, a, b, c: [f(x, y, z) for x, y, z in zip(a, b, c)] # combine three lists elementwise
142
143 # ===== 6. folds ===== (collapse a list to one value)
144 # foldl, foldr and compose are hoisted up to section 1 (combinators): compose needs foldr, foldr needs
145 # foldl, and swap (section 2) needs compose -- so all three must exist before section 2 even begins.
146
147 foldl1 = foldl                                                    # foldl seeded by the first element
148 foldr1 = lambda f, xs: foldl(flip(f), reversed(list(xs)))        # foldr seeded by the last element
149 product = lambda xs: foldl(mul, xs, 1)                            # the numeric fold
150
151 # enumeration folds -- brute-force licenses for small n (say the bound out loud):
152 # Control.Monad replicateM: every length-n word over xs -- cross, n times; |xs|^n
153 replicateM = lambda n, xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs], range(n), [[]])
154 # Data.List subsequences: the powerset; each element DOUBLES acc -- 2^n, fine to n ~ 20
155 subsequences = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs), [[]])
156
157 # ===== 7. scans ===== (a fold that keeps its history)
158
159 def accumulate(xs, f=None, initial=NOTHING):                      # scanl / scanl1
160     if f is None:
161         f = add
162     it = iter(xs)
163     if initial is NOTHING:
164         try:
165             acc = next(it)
166         except StopIteration:
167             return
168     else:
169         acc = initial
170     yield acc
171     for x in it:
172         acc = f(acc, x)
173     yield acc
174
175 scanl = lambda f, z, xs: list(accumulate(xs, f, initial=z))       # foldl, keeping every accumulator
176 scanl1 = lambda f, xs: list(accumulate(xs, f))                   # scanl with no explicit base
177 scanr = lambda f, z, xs: list(reversed(scanl(flip(f), z, list(reversed(list(xs)))))) # foldr, keeping every acc
178 scanr1 = lambda f, xs: list(reversed(scanl1(flip(f), list(reversed(list(xs))))))    # scanr with no base
179
180 # ===== 8. streams ===== (lazy, possibly infinite)
181
182 chain = lambda *iterables: (x for it in iterables for x in it)   # concat, lazily
183
184 def count(start=0, step=1):                                       # [start, start+step ..]
185     n = start
186     while True:
187         yield n
188         n += step
189
190 def cycle(xs):                                                    # repeat xs forever
191     xs = list(xs)
192     if not xs:
193         raise ValueError("cycle: empty list") # Haskell errors too; the alternative is a silent hang
194     while True:
195         yield from xs
196
197 def iterate(f, x):                                                # iterate f x = [x, f x, f (f x), ..]
198     while True:
199         yield x

```

```

200     x = f(x)
201
202 def repeat(x, n=None):                # yield x forever, or n times if given
203     if n is None:
204         while True:
205             yield x
206     else:
207         for _ in range(n):
208             yield x
209
210 def unfoldr(f, seed):                 # Data.List unfoldr: f(seed) -> None | (value, seed')
211     out = []                          # iterate's finite twin: grows a list until f says stop --
212     step = f(seed)                   # no more threading (state, acc) tuples through until
213     while step is not None:
214         val, seed = step
215         out.append(val)
216         step = f(seed)
217     return out
218
219 # ===== 9. slicing & spans ===== (finite views of streams)
220
221 def span(cond, xs):                  # split at first failure, ONE pass; safe on one-shot iters
222     xs = list(xs)
223     i = next((i for i, x in enumerate(xs) if not cond(x)), len(xs))
224     return (xs[:i], xs[i:])
225 break_ = lambda cond, xs: span(compose(not_, cond), xs)          # split at first hit (opposite of span)
226 chunksOf = lambda n, xs: [xs[i:i + n] for i in range(0, len(xs), n)] # Data.List.Split; short last ok
227
228 def dropwhile(cond, xs):             # generator: drop the leading run where cond holds
229     it = iter(xs)
230     for x in it:
231         if not cond(x):
232             yield x
233             break
234     yield from it
235
236 dropWhile = lambda cond, xs: list(dropwhile(cond, xs))           # drop the leading run where cond holds, as a list
237 inits = lambda xs: [xs[:i] for i in range(len(xs) + 1)]         # Data.List: every prefix, [] first
238
239 def islice(iterable, stop):          # take
240     it = iter(iterable)
241     for _ in range(stop):
242         try:
243             yield next(it)
244         except StopIteration:
245             return
246
247 tails = lambda xs: [xs[i:] for i in range(len(xs) + 1)]         # every suffix; substrings start here
248 take = lambda n, xs: list(islice(iter(xs), n))                  # works on infinite streams
249
250 def takeuntil(cond, xs):             # like takewhile(not . cond), but INCLUDES the first hit
251     for x in xs:
252         yield x
253         if cond(x):
254             return
255
256 def takewhile(cond, xs):             # generator: yield the leading run where cond holds
257     for x in xs:
258         if not cond(x):
259             return
260     yield x
261
262 takeWhile = lambda cond, xs: list(takewhile(cond, xs))           # take the leading run where cond holds, as a list
263
264 # ===== 10. sorting & searching =====
265
266 def bisect_left(a, x):               # first index where a[i] >= x
267     lo, hi = 0, len(a)
268     while lo < hi:
269         mid = halve(lo + hi)
270         if a[mid] < x:
271             lo = mid + 1
272     else:

```

```

273         hi = mid
274     return lo
275
276 def bisect_right(a, x):                # first index where a[i] > x
277     lo, hi = 0, len(a)
278     while lo < hi:
279         mid = halve(lo + hi)
280         if a[mid] <= x:
281             lo = mid + 1
282         else:
283             hi = mid
284     return lo
285
286 def cmp_to_key(cmp):                  # resurrect Python 2's cmp for sortBy
287     class K:
288         def __init__(self, x): self.x = x
289         def __lt__(self, other): return cmp(self.x, other.x) < 0
290     return K
291
292 maxOn = lambda f, xs: max(xs, key=f)   # maximumBy (comparing f) in O(n); first wins ties
293
294 def merge(a, b):                      # merge two sorted lists, stable; the heart of merge sort
295     out, i, j = [], 0, 0              # (section 13's merge_lists is this same loop on ListNodes)
296     while i < len(a) and j < len(b):
297         if a[i] <= b[j]:
298             out.append(a[i]); i += 1
299         else:
300             out.append(b[j]); j += 1
301     return out + a[i:] + b[j:]
302
303 minOn = lambda f, xs: min(xs, key=f)   # minimumBy (comparing f): sortOn + head without the sort
304 sortBy = lambda cmp, xs: sorted(xs, key=cmp_to_key(cmp)) # comparator sort, Python 2 style
305 sortOn = lambda f, xs: sorted(xs, key=f) # sortOn (schwartzian, f called once per element)
306
307 # ===== 11. strings =====
308
309 lines = str.splitlines                # split on newlines
310 unlines = lambda ls: "".join(l + "\n" for l in ls) # join lines back with trailing newlines
311 unwords = " ".join                   # join words with single spaces
312 words = str.split                     # split on whitespace
313
314 # ===== 12. containers =====
315
316 # multiset ops on Counters (use .get, not [] -- indexing a Counter autovivifies zeros):
317 bag_diff = lambda a, b: {k: a[k] - b[k] for k in a if a[k] > b.get(k, 0)} # clipped at 0
318 bag_inter = lambda a, b: {k: v for k in a.keys() & b.keys() if (v := min(a.get(k, 0), b.get(k, 0)))} # min count per shared key
319 bag_sub = lambda a, b: {k: a.get(k, 0) - b.get(k, 0) for k in a.keys() if a.get(k, 0) > b.get(k, 0)} # a <= b (inclusion)
320 bag_union = lambda a, b: {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() | b.keys()}
321
322 class defaultdict(dict):              # dict that manufactures a value via factory() on first miss
323     def __init__(self, factory=None, *args, **kw):
324         super().__init__(*args, **kw)
325         self.factory = factory
326     def __missing__(self, key):
327         if self.factory is None:
328             raise KeyError(key)
329         self[key] = self.factory()
330     return self[key]
331
332 def Counter(xs):                      # count-by-value; a defaultdict(int) fold
333     d = defaultdict(int)
334     for x in xs:
335         d[x] += 1
336     return d
337
338 class deque:                          # two-stack; all four ends amortized O(1)
339     def __init__(self, xs=()):
340         self._in, self._out = [], list(reversed(list(xs)))
341     def append(self, x): self._in.append(x)
342     def appendleft(self, x): self._out.append(x)
343     def pop(self):
344         if not self._in:
345             self._in, self._out = list(reversed(self._out)), []

```

```

346     return self._in.pop()
347 def popleft(self):
348     if not self._out:
349         self._out, self._in = list(reversed(self._in)), []
350     return self._out.pop()
351 def __len__(self): return len(self._in) + len(self._out)
352 def __iter__(self): return iter(self._out[::-1] + self._in)
353
354 def dsu(n):                                     # union-find, path halving; union -> False if already joined
355     parent = list(range(n))
356     def find(x):
357         while parent[x] != x:
358             parent[x] = parent[parent[x]]
359             x = parent[x]
360         return x
361     def union(a, b):
362         ra, rb = find(a), find(b)
363         if ra == rb:
364             return False
365         parent[ra] = rb
366         return True
367     return find, union
368
369 def fromListWith(f, pairs):                     # Data.Map fromListWith: THE dict-building fold; f(new, old)
370     d = {}                                     # like insertWith -- so grouping with concat REVERSES each
371     for k, v in pairs:                         # group (the classic Haskell gotcha); use add for counts
372         d[k] = f(v, d[k]) if k in d else v
373     return d                                  # Counter == fromListWith(add) over (x, 1);
374                                           # grouping == fromListWith(++) over (k, [v])
375
376 def getpath(t, ks, default=None):              # read along ks WITHOUT autovivifying
377     for k in ks:
378         if not isinstance(t, dict) or k not in t:
379             return default
380         t = t[k]
381     return t
382
383 def groupBy(eq, xs):                          # Data.List groupBy: runs of CONSECUTIVE elements; each new
384     out = []                                  # element is compared via eq against the run's FIRST element,
385     for x in xs:                              # exactly like Haskell (span-based) -- not its neighbor
386         if out and eq(out[-1][0], x):
387             out[-1].append(x)
388         else:
389             out.append([x])
390     return out
391
392 group = partial(groupBy, lambda a, b: a == b)  # Data.List group: runs of equals, [[a]] -- no keys
393
394 def heappop(h):                               # min-heap on a plain list: sift-down
395     h[0], h[-1] = h[-1], h[0]
396     top = h.pop()
397     i, n = 0, len(h)
398     while True:
399         s, l, r = i, 2*i + 1, 2*i + 2
400         if l < n and h[l] < h[s]: s = l
401         if r < n and h[r] < h[s]: s = r
402         if s == i:
403             return top
404         h[i], h[s] = h[s], h[i]
405         i = s
406
407 def heappush(h, x):                           # min-heap on a plain list: sift-up
408     h.append(x)
409     i = len(h) - 1
410     while i and h[halve(i - 1)] > h[i]:
411         h[halve(i - 1)], h[i] = h[i], h[halve(i - 1)]
412         i = halve(i - 1)
413
414 insertWith = lambda f, k, v, d: {**d, k: f(v, d[k]) if k in d else v} # Data.Map insertWith: PERSISTENT
415                                           # (fresh dict); f(new, old)
416 ITree = lambda: defaultdict(ITree)           # infinitely-nestable defaultdict: a tree that grows on access
417
418 def paths(t):                                 # cata for Tree/ITree: [(keypath, leaf)]

```

```

419     # recursion depth = tree height (word length / len(nums)) -- well under the ~1000 limit
420     if not isinstance(t, dict) or not t:      # leaf = non-dict value, or empty node
421         return [[], t]
422     return [[k] + p, leaf] for k, sub in t.items() for p, leaf in paths(sub)]
423
424 leaves = compose(partial(map_, snd), paths)    # every leaf value in a Tree/ITree, in path order
425
426 def setpath(t, ks, v):                        # write leaf v at key path ks (autovivifies interior)
427     for k in ks[:-1]:
428         t = t[k]
429     t[ks[-1]] = v                             # NB: clobbers any subtree already at ks
430
431 Tree = lambda depth, leaf: (defaultdict(leaf) if depth == 1      # fixed-depth nested defaultdict;
432                             else defaultdict(lambda: Tree(depth - 1, leaf))) # leaves default to leaf()
433
434 unionWith = lambda f, a, b: {**a, **{k: f(a[k], v) if k in a else v for k, v in b.items()}}
435                             # Data.Map unionWith: f(a's value, b's value) on shared keys;
436                             # bag_union == unionWith(max), merge-sum == unionWith(add)
437
438 # ===== 13. nodes ===== (the LeetCode givens: binary trees & linked lists)
439
440 class ListNode:                                # the LeetCode linked list
441     def __init__(self, val=0, nxt=None):
442         self.val, self.next = val, nxt
443
444 class TreeNode:                                # the LeetCode binary tree
445     def __init__(self, val=0, left=None, right=None):
446         self.val, self.left, self.right = val, left, right
447
448 from_list = lambda xs: foldr(ListNode, xs, None)    # build; foldr of ListNode
449
450 def has_cycle(node):                            # Floyd: fast laps slow iff a cycle exists
451     slow = fast = node
452     while fast and fast.next:
453         slow, fast = slow.next, fast.next.next
454         if slow is fast:
455             return True
456     return False
457
458 # tfold: the tree cata -- f(val, l_acc, r_acc), z for the empty tree; recursion depth = tree height
459 tfold = lambda f, z, t: z if t is None else f(t.val, tfold(f, z, t.left), tfold(f, z, t.right))
460 inorder = partial(tfold, lambda v, l, r: l + [v] + r, [])    # left, root, right
461
462 def levelorder(t):                              # BFS: the deque earning its keep
463     out, q = [], deque([t] if t else [])
464     while len(q):
465         n = q.popleft()
466         out.append(n.val)
467         if n.left: q.append(n.left)
468         if n.right: q.append(n.right)
469     return out
470
471 def merge_lists(a, b):                          # merge two sorted lists; dummy-head idiom
472     dummy = tail_ = ListNode()
473     while a and b:
474         if a.val <= b.val:
475             tail_.next, a = a, a.next
476         else:
477             tail_.next, b = b, b.next
478         tail_ = tail_.next
479     tail_.next = a or b
480     return dummy.next
481
482 def middle(node):                               # fast/slow: second middle for even lengths
483     slow = fast = node
484     while fast and fast.next:
485         slow, fast = slow.next, fast.next.next
486     return slow
487
488 postorder = partial(tfold, lambda v, l, r: l + r + [v], [])    # left, right, root
489 preorder = partial(tfold, lambda v, l, r: [v] + l + r, [])    # root, left, right
490
491 def reverse_list(node):                         # three-pointer; prev is a fold accumulator

```

```

492     prev = None
493     while node:
494         node.next, prev, node = prev, node, node.next
495     return prev
496
497 tdepth    = partial(tfold, lambda _, l, r: 1 + max(l, r), 0)    # height of the tree
498 to_list   = compose(list, partial(unfoldr, lambda n: None if n is None else (n.val, n.next))) # ListNode chain -> list
499 tsize     = partial(tfold, lambda _, l, r: 1 + l + r, 0)        # number of nodes in the tree
500
501 # ===== 14. grids & windows =====
502
503 def longest_window(xs, valid, push, shed): # sliding-window skeleton; state lives in the closures
504     lo = best = 0
505     for hi in range(len(xs)):
506         push(xs[hi])
507         while not valid(): # shrink until legal again
508             shed(xs[lo])
509             lo += 1
510         best = max(best, hi - lo + 1)
511     return best
512
513 def neighbors4(r, c, R, C): # in-bounds orthogonal neighbors
514     for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
515         if 0 <= r + dr < R and 0 <= c + dc < C:
516             yield r + dr, c + dc
517
518 def neighbors8(r, c, R, C): # ... plus diagonals
519     for dr in (-1, 0, 1):
520         for dc in (-1, 0, 1):
521             if (dr or dc) and 0 <= r + dr < R and 0 <= c + dc < C:
522                 yield r + dr, c + dc
523
524 # ===== 15. control =====
525
526 def memo(f): # unbounded memoizer; enough for DP (no eviction by design)
527     cache = {}
528     def wrapped(*args, **kw):
529         key = (args, frozenset(kw.items())) if kw else args
530         if key not in cache:
531             cache[key] = f(*args, **kw)
532         return cache[key]
533     wrapped.cache = cache # peek at the DP table if curious
534     return wrapped
535
536 def until(cond, f, x): # until cond f x -- loop, not recursion (unbounded depth)
537     while not cond(x):
538         x = f(x)
539     return x
540
541 # ===== 16. monoids & monads ===== (the algebra of folds; failure as a value)
542
543 # ---- monoids: an associative op with an identity, reified as the pair (mempty, mappend) ----
544 Monoid = lambda empty, op: (empty, op) # a monoid IS its identity and its combiner
545 All = Monoid(True, lambda a, b: a and b) # mconcat(All, bs) == all(bs)
546 Any = Monoid(False, lambda a, b: a or b) # mconcat(Any, bs) == any(bs)
547 both = lambda m1, m2: Monoid((m1[0], m2[0]), # tuple monoid: TWO folds in ONE pass
548                             lambda a, b: (m1[1](a[0], b[0]), m2[1](a[1], b[1])))
549 First = Monoid(None, lambda a, b: b if a is None else a) # first non-None wins
550 mconcat = lambda m, xs: foldl(m[1], xs, m[0]) # fold a whole list with the monoid
551 foldMap = lambda f, m, xs: mconcat(m, [f(x) for x in xs]) # map into the monoid, then mconcat
552 Last = Monoid(None, lambda a, b: a if b is None else b) # last non-None wins
553 ListM = Monoid([], add) # concat as a monoid
554 MaxM = Monoid(float("-inf"), max) # identity -inf: max of nothing loses to all
555 MinM = Monoid(float("inf"), min) # identity inf: min of nothing loses to all
556 Product = Monoid(1, mul) # mconcat(Product, xs) == product(xs)
557 Sum = Monoid(0, add) # mconcat(Sum, xs) == sum(xs)
558
559 # ---- Maybe monad: chain None-propagating steps without if-ladders -----
560 # None is Nothing (file header), so a step cannot return None as a SUCCESS -- the standing caveat.
561 bind = lambda x, f: None if x is None else f(x) # >= : feed x onward unless it already failed
562 chainM = lambda *fs: lambda x: foldl(bind, fs, x) # Kleisli >= : LEFTMOST runs first (not compose)
563 sequenceM = lambda xs: foldr(lambda x, acc: None if x is None or acc is None
564                             else [x] + acc, xs, []) # [Maybe a] -> Maybe [a]: all-or-nothing

```

```

565 traverseM = lambda f, xs: sequenceM([f(x) for x in xs])      # map a failable f, demand every success
566
567 # ---- Either: a Maybe that carries WHY it failed -----
568 bindE      = lambda x, f: x if x[0] == "err" else f(x[1])      # short-circuit, but keep the message
569 Ok, Err    = lambda v: ("ok", v), lambda e: ("err", e)        # tagged pairs; hoisted ahead of chainE
570 chainE     = lambda *fs: lambda x: foldl(bindE, fs, Ok(x))    # compose Ok/Err-returning steps

```

